

OS(Operating Systems)

교수님 강의 + THREE_EASY_PIECES 번역본 독학

2020/09/01 ~

박지원

목차

1. 2강_OS Overview
2. 과제 1번 공부
3. 4강_Process Abstraction
4. 과제 2번 공부 + 5강_프로세스 API
5. 6강_Limited Direct Execution
6. 7강_Scheduling
7. 과제 3번 공부
8. 13강_Address Spaces & Memory Management
9. 15강_Address Translation
10. 21강_Page Replacement
11. 26강_Concurrency
12. 27강_Thread API
13. 28강_Mutual Exclusion
14. 31강_Semaphore
15. 32강_Common Concurrency Problems : Deadlock

- 16. 과제 5번 공부
- 17. 39강_Interlude : Files and Directories
- 18. 40강_File System Implementation
- 19. 37강_HDD Drives
- 20. 과제 6번 공부
- 21. 38강_RAID
- 22. 41강_Locality and Fast File System
- 23. 42강_Crash Consistency : FSCK and Journaling

<2강_OS Overview> 09/01 ~ 09/13

1. 운영체제 개요

1) OS = 운영체제 = 자원관리자 = 자원 관리하는 놈 = 컴퓨터 시스템 자원을 관리하는 놈 = 응용프로그램에게 시스템 콜을 제공합니다.

= 하드웨어를 쉽게 사용하기 위해 추상화 한 시스템 소프트웨어 = 때때로 가상머신(Virtual Machine)이라고 불립니다.

- OS 덕분에 하드웨어에 직접 접근하지 않아도 됩니다.

- OS는 시스템이 유저가 사용하기 쉬운 방식으로 정확하고 효율적으로 작동하는지 검사도 해줍니다.

- 거의 다 C로 구현했고, C++로 구현한 OS도 조금 있습니다. 그러므로 모두 구조체로 구현했다고 생각해도 됩니다.

2) 자원 = 컴퓨터 시스템 자원 = 하드웨어(메모리 + CPU + 디스크 = 3대 하드웨어 구성요소) 자원

- 자 또는 놈이 들어가는 용어는 소프트웨어입니다.

- 자원을 관리하면 소프트웨어입니다.

- 하드웨어를 과거(애니악)에는 연구원들이 직접 컴퓨터 선을 이용하여 사용하였습니다. 현대에는 인간이 직접 물리적으로 관리하는 어려운 과정 대신에 쉽게 하드웨어를 관리하기 위해 OS라는 소프트웨어를 이용하게 되었습니다.

3) CPU는 명령어를 초당 수십억 번 반입하고 해석(무슨 명령어 인지 파악)하고 실행(덧셈, 메모리 접근, 분기)합니다. 그리고 다음 명령어로 이동하는 과정을 무한 반복합니다.

4) OS의 주요 역할 3가지

3 Easy Piece (UW)	Abstracted by struct(class)	HW
Virtualization	Process Structure (task_struct) Memory Management (mm_struct -> vm_area struct ->)	CPU Memory
Concurrency	Process (semaphore, thread...)	
Persistent	File System (files_struct -> file struct -> inode struct)	DISK

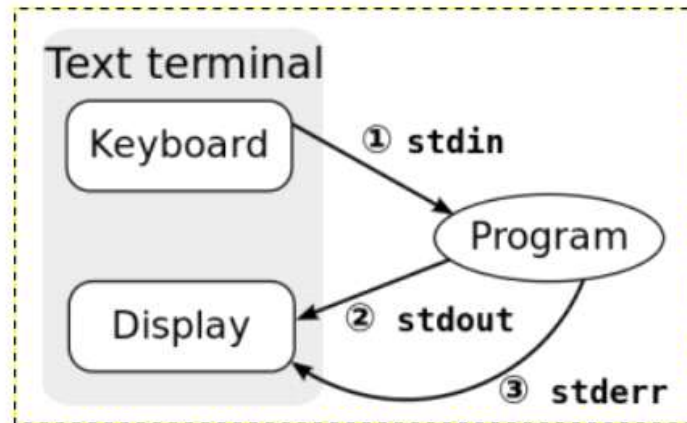
2. 중요 기초 용어 정리

1) Kernel - 알맹이(Core)

- = 운영체제 내부에 존재하는 핵심적인 소프트웨어 입니다. 커널 자체를 운영체제라고 부를 정도로 핵심 입니다.
- = 메모리에 상주하는 소프트웨어 입니다.
- 운영체제가 하는 역할 중에 자원관리가 있습니다. 이 자원관리를 커널이 담당합니다.
- 수업시간의 OS는 커널을 의미합니다.

2) Shell - 껍데기

- = 사용자(명령을 입력함)와 운영체제(사용자의 명령을 받아 연산을 하고 하드웨어에 명령을 내림) 간에 대화를 가능하게 해주는 중계자 입니다.
- 사용자가 터미널을 통해 명령어를 입력하면 셸이 stdin을 통해 입력 받고 운영체제로 명령어를 전달합니다. 운영체제는 입력 받은 명령어를 수행하고 결과를 셸에게 전달하고 셸은 stdout또는 stderr를 통해 명령어를 터미널에서 사용자에게 보여줍니다.



- Default Shell은 Windows 환경에서 cmd 이고 Linux 환경에서는 bash 입니다.
- 리눅스에는 sh , csh 셸이 더 존재 합니다.

3) Terminal

- = 셸을 실행하기 위해 명령어를 입력할 수 있는 GUI 환경의 소프트웨어 입니다. (리눅스 검은 창 , windows cmd , PowerShell)
- = 터미널은 TV이고 셸은 TV 프로그램이라고 생각하면 됩니다.
- = 터미널의 \$은 셸이 사용자의 명령어를 받아드릴 준비가 되었다는 뜻입니다.

4) Console

- = 터미널의 한 종류입니다.

5) System Call(시스템 콜)

- = 추상화 된 장치(cpu , memory)들을 사용하기 위해 함수(커널함수)가 필요합니다. 사용자가 커널함수를 쓰려면 System Call을 이용해야 합니다.
 - 이러한 시스템 콜은 직접 사용하기에 많은 어려움이 있습니다. 때문에 프로그래밍 언어들은 시스템 콜을 편리하게 사용하기 위한 수단으로 API를 제공합니다. 다시 말해 Windows API와 시스템 콜은 비슷하지만 범위가 다릅니다. 시스템 콜은 커널함수를 호출하고 , API는 시스템 콜을 호출합니다.
 - 500개 정도가 존재합니다.
- ex) open , fork , sleep

6) Memory

- 메모리에는 자신이 만든 모든 자료구조가 존재하고 명령어가 존재합니다.
- 물리 모델은 바이트의 배열입니다. (n바이트씩 읽음 = 1 Word)

3. Abstraction(추상화)

= 사용자가 직접 사용하기 어려운 하드웨어를 사용자가 직접 사용하기 쉬운 소프트웨어를 통해 사용할 수 있게 하는 방법

ex) 프린터를 쉽게 사용하기 위해 프린터 마스터 같은 소프트웨어를 이용합니다.

- 하드웨어의 cpu는 매우 복잡하므로 논리적인 타입(int,double...)으로 만드는 것이 불가능 합니다. 그러므로 Struct로 선언해줍니다.

하드웨어	OS에서 추상화 한 소프트웨어(Struct)
CPU	PCB(Process Control Block)라고 부르며 리눅스에서는 task_struct 유닉스(xv6)에서는 proc.h
메모리	mm(=Memory Management) = task_struct의 멤버입니다.
디스크	File System(=disk)
네트워크(NIC)	Protocol or Socket

Each process has a PCB (process control block)
defined by `struct proc` in xv6

4. Virtualization(가상화)

- 가상화와 추상화는 다른 개념이므로 완벽하게 구분해야 합니다. CPU , 메모리 , 디스크는 모두 추상화를 하지만 가상화는 CPU와 Memory만 수행합니다.

- 추상화를 먼저 하고 가상화가 진행 되는 것입니다.

- 컴퓨터는 더 많은 프로세스를 실행 시키는 것이 목표(Utilize)이므로 가상화 기법을 이용합니다.

1) CPU Virtualization

= 하나의 CPU 또는 다중 CPU가 존재하는 환경에서 매우 짧은 시간 동안 하나의 프로세스에게 사용할 권한을 주고 시간이 끝나면 다른 하나의 프로세스에게 CPU 사용 권한을 주는 과정을 고속으로 반복 처리하여 마치 CPU가 무한개인 것처럼 보이는 기법입니다.(=Illusion)

- Illusion은 시분할 기법과 연관 된 개념입니다.

- 하나의 프로세스는 자신의 차례가 되면 약 10ms(0.01초)만 cpu를 사용하고 시간이 지나면 양도합니다.

- 10ms후에 다른 프로세스에게 CPU를 양도하면 switch2(a,b) method를 이용해서 프로세스 a(현재 수행되고 있는 프로세스)와 프로세스 b(스케줄 함수를 통해 찾아 다음에 수행할 프로세스)를 교환합니다. 교환 하는 과정을 Context Switch(문맥교환)라 합니다.

- 다음에 어떤 프로세스를 처리 할 지를 정해 놓는 기법을 Scheduling(=dispatch)이라고 합니다.
- 반드시 10ms를 지켜야 하지 않고 , 프로세스가 그 전에 종료되거나 예기치 못한 일이 발생하면 즉시 문맥 교환이 일어날 수 있습니다.
- 운영체제의 자원관리자 적 측면이 가장 부각되는 기능입니다.

2) Memory Virtualization

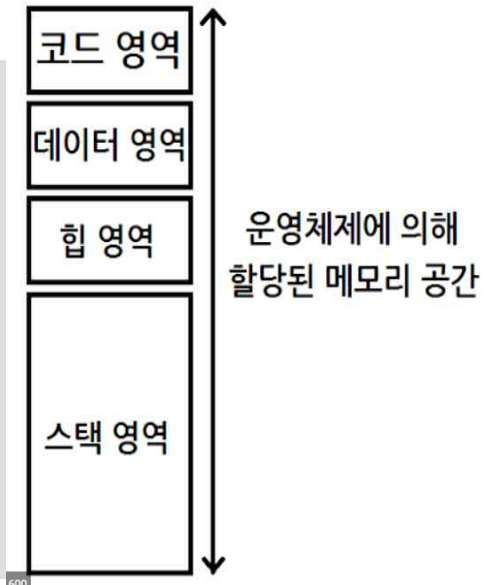
*** 13강에서 자세하게 배웁니다. ***

① 사전 지식

<Virtual Address Space(가상 주소 공간)>

여담: 당신이 보는 모든 주소는 가상 주소이다

포인터를 출력하는 C 프로그램을 작성한 적이 있는가? 당신이 보는 값은(좀 큰 변수, 종종 16진수로 출력되는) 가상 주소(virtual address)이다. 프로그램 코드가 탑재된 주소가 어디인지 궁금한 적은 없는가? 그 주소를 출력할 수 있다. 그러나 주소를 출력했다면, 그 주소는 가상 주소이다. 사용자 프로그램(user level program)이 볼 수 있는 주소는 모두 가상 주소이다. 메모리 가상화의 기술 때문에, 명령어와 데이터가 탑재되어 있는 물리 메모리 주소를 알 수 있는 것은 오직 운영체제뿐이다. 결코 잊지 말아야 할 사실: 프로그램에서 주소를 출력하면, 그 주소는 가상 주소, 즉 메모리 배치에 대한 환상이다. 오직 운영체제와 하드웨어만이 진실을 알고 있다.

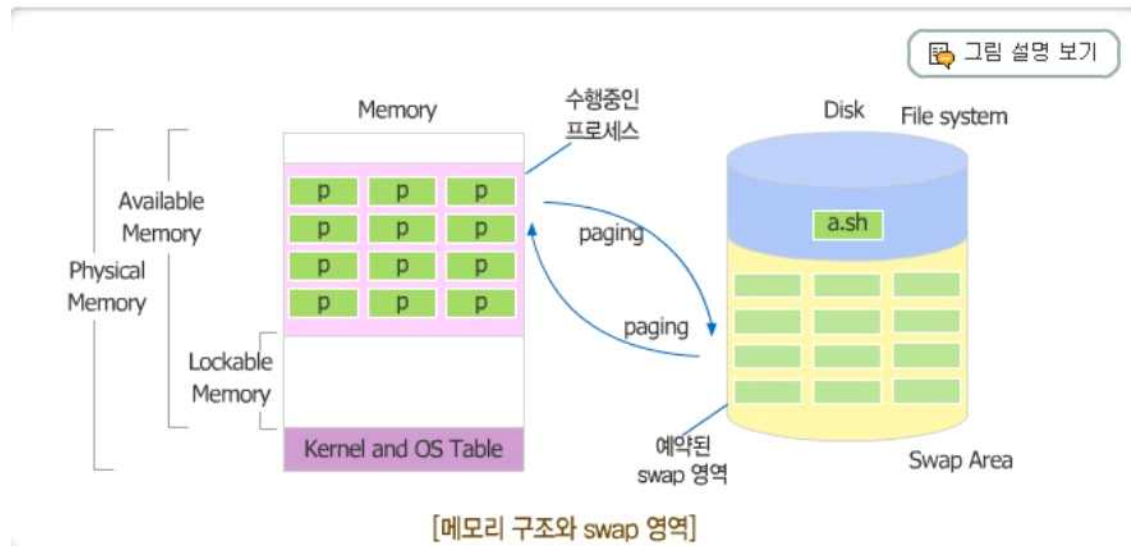


- = OS가 하드웨어인 메모리(실제 메모리)를 쉽게 관리하기 위해 만든 가상의 주소 공간 입니다.
- 지금까지 알고 있던 메모리의 주소는 사실 다 가상 주소였고 , **가상 주소와 실제 주소는 완전 다른 개념**입니다.
- 32비트 컴퓨터는 레지스터가 32비트로 이루어진 컴퓨터 입니다. 레지스터에 저장할 수 있는 값의 범위는 0x00000000 ~ 0xffffffff(Hex값)이므로

하나당 4비트)입니다. 현대에는 64비트를 이용하지만 OS가 그 능력을 따라가지 못해 아직 48비트만 이용합니다.

- 가상 메모리의 가상 주소공간은 0x00000000 ~ 0xffffffff까지 존재하므로 총 4GB를 할당 받습니다. 리눅스에서는 이 중 1GB는 커널 메모리 공간이고, 나머지 3GB는 유저 공간이며, 유저 공간은 흔히 알고 있는 코드, 데이터, 힙, 스택 영역으로 이루어져 있습니다.

<Virtual Memory 와 Swap 영역>



<Swap>

- = 메모리 영역을 넘어서도 프로그램을 사용할 수 있도록 해야 하므로 **디스크의 일부 영역을 메모리처럼 사용하는 영역**입니다.
- 현대에는 메모리가 16GB를 넘어가므로 점점 필요 없는 영역이 되어가고 있습니다.
- Swap 영역도 결국은 디스크이므로, Process가 Swap 영역에 존재하면 실행할 수 없기 때문에 I/O가 발생합니다.

<Virtual Memory(가상 메모리)>

= Physical Memory(실제 메모리) + Swap 영역

= 속도를 따지지 않고 어쨌든 프로세스가 올라갈 수 있는 공간

- 프로그램은 메모리로의 로딩 과정을 느리게 수행하고 , 그 과정에서 **일부분이 로딩 되면 바로 실행해버립니다.**

<CPU Utilization>

= CPU가 놀지 않고 많은 일을 처리하기 바라는 os의 정책

- 프로세스는 사실 대부분 i/o 작업이 많고 이 **i/o작업은 CPU가 담당하지 않고 i/o controller가 담당합니다.** 다시 말해 , 메모리가 프로세스에 많이 올라가면 CPU를 많이 사용하여 과부하가 걸릴 것 같으나 실제로는 i/o만 많이 하게 됩니다. 그러므로 os는 최대한 메모리에 프로세스를 올려서 CPU를 많이 사용할 확률을 늘려야 합니다.

② 개념 설명

= 가상 메모리의 가상 주소 공간(0x00000000 ~ 0xffffffff)에서 하나의 가상 주소를 프로세스 에게 할당합니다. 가상주소는 서로 다른 프로세스 끼리 같을 수 있습니다. 가상 주소를 CPU에 있는 MMU(Memory Management Unit , 현대의 대부분의 고사양 CPU는 가지고 있습니다.)에 전달하고 MMU는 **가상 주소와 실제 주소를 mapping하여 실제 주소로 변환시켜 줍니다.** 이 과정을 모든 프로세스에서 진행하면 **프로세스마다 1대1로 메모리가 존재하는 것처럼 보입니다.** 이를 **Memory Virtualization**이라고 합니다.

- 프로그래밍을 하다보면 서로 다른 프로세스가 같은 가상 주소를 가질 때가 있습니다. 하지만 MMU가 실제 주소로 변화 시켜주면 실제 주소는 절대 같지 않습니다.

- 만일 프로그래머가 실제 메모리의 실제 주소를 기반으로 프로그래밍을 하려면 매우 어려워집니다. 그러므로 가상 메모리로 쉽게 프로그래밍하고 실제 메모리로의 mapping은 CPU MMU에게 맡깁니다.

5. Persistence(영속성)

= 사용자에게 공유가 중요하기에 만들어진 기능입니다.

- /tmp/file.txt = 'file.txt를 tmp폴더 안에 넣어라' = 다른 사람과 공유하겠다는 의미입니다.

- 추후에 깊게 설명하겠습니다.

6. File Descriptor

1) 사전지식

<Descriptor>

= 설명해주는 놈

<Stream>

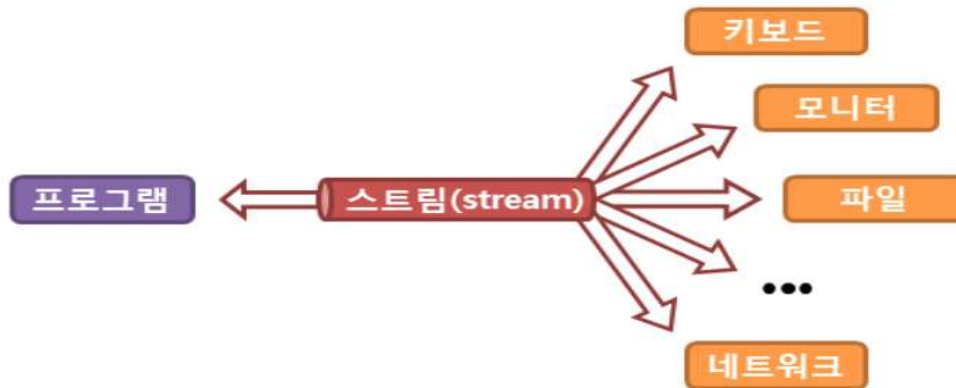
= 파일들의 이동(입력과 출력)을 일관된 방식으로 다루기 위한 가상의 연결 통로

- 사전적 의미는 흐르는 시냇물

- Buffer라는 임시 저장 공간을 갖고 있어서 일정량의 데이터를 모아두었다가 한 번에 처리합니다.

ex) 디스크는 느리고 메모리는 빠르므로 그 사이에 버퍼를 두어 임시 저장하고 한 번에 데이터를 교환하면 disk i/o를 줄일 수 있습니다.

- 스트림의 관점에서 데이터가 어떤 곳으로 흘러 들어가는 것을 입력이라고 하며 , 어떤 곳에서 나오는 것을 출력이라 합니다.



<리눅스의 입출력>

-리눅스는 컴퓨터의 모든 것을 파일로 모델링 합니다. 사용자가 터미널에 적은 것도 어떤 파일에 저장 되고 그 내용을 디스플레이에 띄워서 보여주는 것도 어떤 파일에 저장된 것 입니다.

2) 정의

- = 파일 디스크립터는 어떤 정수 값 입니다.
- = 리눅스 환경에서 3가지의 Default 입출력 스트림이 존재합니다. 이들은 각자 특정한 파일 디스크립터를 갖고 있습니다.
 - Standard Input(STDIN) -> 파일디스크립터 정수 값 0을 갖고 있습니다.
 - Standard Output(STDOUT) -> 파일디스크립터 정수 값 1을 갖고 있습니다.
 - Standard Error(STDERR) -> 파일디스크립터 정수 값 2을 갖고 있습니다.
- Default 이외에 파일 디스크립터는 3이상의 정수 값을 갖습니다.

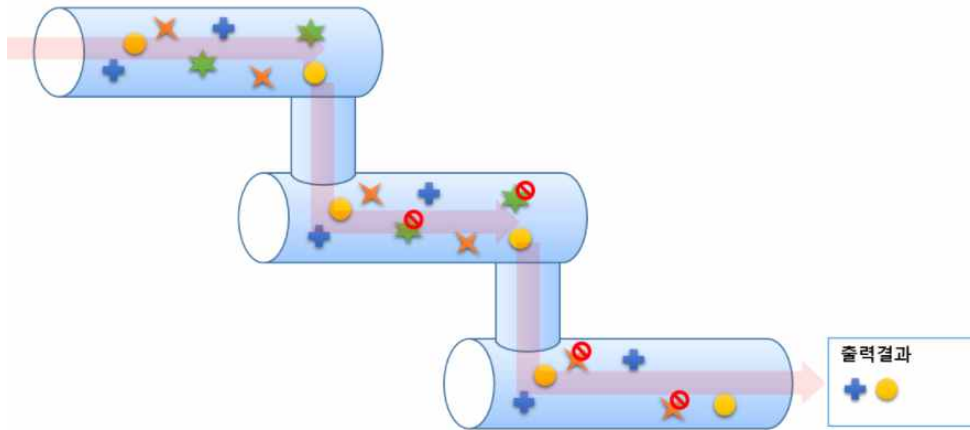
3) 데이터 흐름



- 키보드로 명령을 터미널에 적으면 셸은 stdin을 통해 입력을 받습니다.
- 셸은 운영체제가 처리한 명령어의 결과를 stdout 또는 stderr를 통해 터미널에 데이터를 출력시킵니다.
- 0 ~ 2는 파일디스크립터 정수 값을 나타 냅니다.
- 데이터의 입력과 관련된 파일 디스크립터는 stdin이고 데이터의 출력과 관련된 파일 디스크립터는 stdout 과 stderr 입니다.

4) 파이프('|')

- = 터미널에서 여러 개의 명령어를 실행할 때 **이전 명령어의 결과를 다음 명령어의 입력 값으로 사용하고 싶을 때** 파이프를 사용합니다.
- 리눅스 터미널에서 명령어로 사용할 수 있습니다.



<사용법>

`cat /etc/passwd | grep mail`

cat 명령어의 결과를 grep 명령어의 입력으로 사용 할 수 있습니다.

- 터미널에서 한 번에 사용 가능한 파이프의 개수는 제한이 없습니다.
- 파이프 기준으로 양쪽 다 바이너리 실행 파일이 들어갑니다.

5) I/O Redirection (방향 수정)

= 보통의 입출력 과정은 stdout 또는 stderr로 출력 되는 데이터를 터미널에 표시합니다. 하지만 Redirection을 하면 **출력 되는 데이터를 다른 장치로 보내게 됩니다.**

① >

> file

표준 출력을 파일로 재지향 합니다. 파일이 없으면 새로 만들고, 파일이 있으면 덮어씁니다.

ex) test.txt 파일의 출력을 test.out 파일로 저장

```
cat test.txt > test.out
```

② >>

>> file

표준 출력을 파일로 재지향 합니다. 파일이 없으면 새로 만들고, 파일이 있으면 파일의 끝에 덧붙입니다.

ex) today.log 파일의 출력을 week.log 파일의 끝에 덧붙임

```
cat today.log >> week.log
```

③ >&

2>&1

표준 에러를 표준 출력으로 재지향합니다. 표준 에러도 표준 출력의 자격으로 보내집니다.

ex) cat 명령어의 에러를 표준 출력으로 재지향하고 표준 출력을 out 파일로 재지향

```
[root@peterdev test]# cat nofile > out 2>&1
[root@peterdev test]# cat out
cat: nofile: No such file or directory
```

에러가 표준 출력으로 재지향 되었기 때문에 화면에 에러가 출력되는 대신에 out 파일에 기록됩니다.

에러 메시지는 반드시 2를 붙여주어야 합니다. 파일디스크립터가 2인 stderr이기 때문입니다.

④ <

< file

파일로부터 표준 입력을 받도록 재지향합니다.

ex) cat 명령어의 에러를 표준 출력으로 재지향하고 표준 출력을 out 파일로 재지향

```
[root@peterdev test]# cat nofile > out 2>&1
[root@peterdev test]# cat out
cat: nofile: No such file or directory
```

에러가 표준 출력으로 재지향 되었기 때문에 화면에 에러가 출력되는 대신에 out 파일에 기록됩니다.

<과제 1번 공부 > 09/04 ~ 09/08

과제에도 학습 내용이 있으므로 반드시 봅니다.

1. Ubuntu 설치 과정

1) VirtualBox 설치 <https://www.virtualbox.org/>

2) VirtualBox 내부에 Ubuntu 설치 : <https://m.blog.naver.com/wideeyed/220960764870>

- 최소 기능만 설치 합니다.
- 한글 100/104
- 하드디스크는 우분투 자체가 13GB이므로 최소 30GB할당 , 메모리는 4096MB
- 프로세서 사용 2개
- 가속 -> HyperV 이용합니다.
- 디스플레이 메모리 128MB
- 그래픽 컨트롤러 VMSVGA + 3차원 가속 이용합니다.
- 저장소 -> 구글에서 Ubuntu 18.04 iso 파일(2GB)을 다운 받고 컨트롤러에서 설치합니다.
- 일반 -> 고급에서 클립보드 공유를 양방향으로 변경합니다. (윈도우에서 복사한 것을 우분투에서 붙여넣을 수 있습니다.)



3) 공유폴더

- 공유 폴더 탭에서 윈도우의 폴더를 선택
- 우분투에 접속해서 장치 -> 게스트 확장 CD 삽입 설치
- `sudo gpasswd -a pjw960316 vboxsf`로 나의 계정을 vboxsf에 추가합니다.

4) 시스템 설정에서 원하는 대로 설정을 합니다.

2. 용어

1) fork()

- 실행 중인 프로세스(부모)가 자신과 완전한 동일한 복사본(자식)을 생성합니다. 두 프로세스를 구분하기 위해 부모 프로세스의 리턴 값은 자식 프로세스의 PID이고 자식프로세스의 리턴 값은 0입니다.
- fork()는 부모프로세스가 하는 것이고 자식 프로세스가 fork()되는 것입니다.

2) PID(Process Identifier)

- 프로세스 식별자입니다.
- 프로세스 관련 식별할 일이 있으면 반드시 PID를 이용하므로 매우 중요한 개념입니다.
- PID 1(init 프로세스)이 모든 프로세스의 조상입니다.

3) PPID

= 부모 프로세스의 PID

3. 리눅스 명령어

리눅스의 명령어 옵션은 서로이어서 쓰면 모두 작동합니다.

ex) -e , -f 가 있다면 -ef

1) more

확인하고자 하는 파일의 내용이 너무 길 경우 한 화면에 다 출력할 수 없으므로 파이프()를 이용해서 more 명령어를 사용한다.

```
$ ls -al | more
```

2) ps

- 현재 실행중인 프로세스의 목록을 보는 명령어입니다.
- ef를 옵션으로 이용하면 해당 프로세스의 부모 프로세스의 PID를 볼 수 있습니다.
- aux를 옵션으로 이용하면 더욱 상세하게 볼 수 있습니다.

```
pjw960316@pjw960316-VirtualBox:~$ ps -aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.2 160112  9316 ?        Ss   06:25   0:08 /sbin/init splash
```

USER	프로세스 소유주
PID	프로세스 아이디
%CPU , %MEM	CPU , 메모리 사용률
VSZ(Virtual Memory Size)	가상 메모리의 할당량
RSS(Resident Set Size)	실제 메모리의 할당량
TTY	프로세스의 소유자가 사용중인 터미널 포트
STAT or S	프로세스 상태
START	프로세스 시작 시간
TIME	총 사용된 CPU time
COMMAND	실행된 명령어의 이름

- STAT의 옵션

코드분류	의미
D	IO 와 같이 중지(interrupt)시킬 수 없는 잠자고 있는(휴지) 프로세스 상태
R	현재 동작중이거나 동작할 수 있는 상태
S	잠자고 있지만 중지시킬 수 있는 상태
T	작업 제어 시그널로 정지되었거나 추적중에 있는 프로세스 상태
X	완전히 죽어 있는 프로세스
Z	죽어 있는 좀비 프로세스

두번째 세번째 필드 코드

코드분류	의미
<	프로세스의 우선 순위가 높은 상태
N	프로세스의 우선 순위가 낮은 상태
L	실시간이나 기존 IO를 위해 메모리 안에 잠겨진 페이지를 가진 상태
s	세션 리더(주도 프로세스)
I	멀티 쓰레드
+	포어그라운드 상태로 동작하는 프로세스

① VSZ(Virtual Memory Size)

= 스왑 아웃 된 메모리, 할당되었지만 사용되지 않은 메모리 및 공유 라이브러리의 메모리를 포함하여 프로세스가 액세스 할 수 있는 모든 메모리를 포함합니다.

② RSS(Resident Set Size)

= 해당 프로세스에 할당되고 RAM에 있는 메모리 양을 표시하는 데 사용됩니다. 스왑 아웃 된 메모리는 포함되지 않습니다. 해당 라이브러리의 페이지가 실제로 메모리에 있는 한 공유 라이브러리의 메모리가 포함됩니다. 모든 스택 및 힙 메모리가 포함됩니다.

RSS는 Resident Set Size이며 해당 프로세스에 할당되고 RAM에있는 메모리 양을 표시하는 데 사용됩니다. 스왑 아웃 된 메모리는 포함되지 않습니다. 해당 라이브러리의 페이지가 실제로 메모리에있는 한 공유 라이브러리의 메모리가 포함됩니다. 모든 스택 및 힙 메모리가 포함됩니다.

VSZ는 가상 메모리 크기입니다. 스왑 아웃 된 메모리, 할당되었지만 사용되지 않은 메모리 및 공유 라이브러리의 메모리를 포함하여 프로세스가 액세스 할 수있는 모든 메모리를 포함합니다.

- 윈도우에서는 tasklist라는 명령어를 쓰면 프로세스 기록을 볼 수 있습니다.

3) sudo

- 뒤따르는 모든 명령을 슈퍼유저 권한으로 실행합니다.

4) top

PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND

- * PID : 프로세스 ID (PID)
- * USER : 프로세스를 실행시킨 사용자 ID
- * PRI : 프로세스의 우선순위 (priority)
- * NI : NICE 값. 일의 nice value값이다. 마이너스를 가지는 nice value는 우선순위가 높음.
- * VIRT : 가상 메모리의 사용량(SWAP+RES)
- * RES : 현재 페이지가 상주하고 있는 크기(Resident Size)
- * SHR : 분할된 페이지, 프로세스에 의해 사용된 메모리를 나눈 메모리의 총합.
- * S : 프로세스의 상태 [S(sleeping), R(running), W(swapped out process), Z(zombies)]
- * %CPU : 프로세스가 사용하는 CPU의 사용율
- * %MEM : 프로세스가 사용하는 메모리의 사용율
- * TIME+ : 프로세스 시작된 이후 경과된 총 시간
- * COMMAND : 실행된 명령어

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1251	pjw9603+	20	0	3666156	376024	124348	S	18.2	9.3	3:40.06	gnome-shell
1053	pjw9603+	20	0	865484	153084	76784	R	15.5	3.8	1:35.44	Xorg
19886	pjw9603+	20	0	824764	41816	31444	R	6.4	1.0	0:00.51	gnome-terminal-
1	root	20	0	160112	9312	6688	S	1.8	0.2	0:07.70	systemd
19914	pjw9603+	20	0	50308	4008	3276	R	1.8	0.1	0:00.03	top
1185	pjw9603+	20	0	189568	2856	2500	S	0.9	0.1	1:41.20	VBoxClient
1481	pjw9603+	20	0	1012848	125780	64904	S	0.9	3.1	0:12.51	nautilus-deskto
2	root	20	0	0	0	0	S	0.0	0.0	0:00.02	kthreadd

5) grep

= 입력으로 전달된 파일의 내용에서 특정 문자열을 찾고자 할 때 사용합니다.

4. sh파일

.sh파일은 쉘 명령어 언어입니다. 터미널에서 명령어를 통해 실행 시킬 수 있습니다.

```
$ sudo chmod 755 test.sh
```

실행

test.sh파일이 /home/ 아래에 있다고 가정

```
$ cd /home
```

```
$ ./test.sh # 실행
```

또는

```
$ /home/test.sh # 절대경로로 실행
```

<4강_Process Abstraction> 09/13 ~ 09/16

1. Process(프로세스)

1) 정의

- = 프로그램은 디스크(SSD OR HDD) 상에 존재하며 명령어와 정적 데이터의 묶음으로 이루어진 실행 대기 상태의 파일(.exe)이며 생명이 없는 존재입니다. 이를 **메모리에 올리면(생명을 넣어주면) 이를 프로세스**라고 합니다.
- 프로세스는 하드웨어(메모리, 레지스터, 디스크)와 밀접한 관련이 있습니다.
- 현대의 OS는 프로그램을 실행하면서 코드나 데이터가 필요할 때 필요한 부분만 메모리에 탑재합니다.
- OS는 code와 data 영역을 우선 메모리에 탑재하고, Stack과 Heap을 할당하고 초기화 하고, 입출력과 관계된 다른 작업을 모두 마치면 프로그램을 프로세스로 실행시키기 위한 준비가 모두 완료됩니다.

2) Process Structure(프로세스 구성)

- = **레지스터에 저장될 값(연산할 값 & PC 값 & etc) + 메모리에 올라갈 4가지 영역(Code(=Text) + Data + Stack + (Heap))**

3) Time Sharing(시분할 기법)

- = 하나의 프로세스를 실행하고, 얼마 후 중단시키고 다른 프로세스를 실행하는 작업을 빠르게 반복하면서 여러 개의 CPU가 존재하는 듯이 환상을 만들어 내는 기술입니다. (실행을 하드웨어 자원 사용과 같은 개념으로 생각합니다.)
- 현대의 모든 운영체제들이 채택하고 있습니다.

4) Scheduling Policy(스케줄링 정책)

- = 운영체제가 여러 프로그램들 중에 어느 프로그램을 실행시켜야 하는 지 결정 하는 정책입니다.

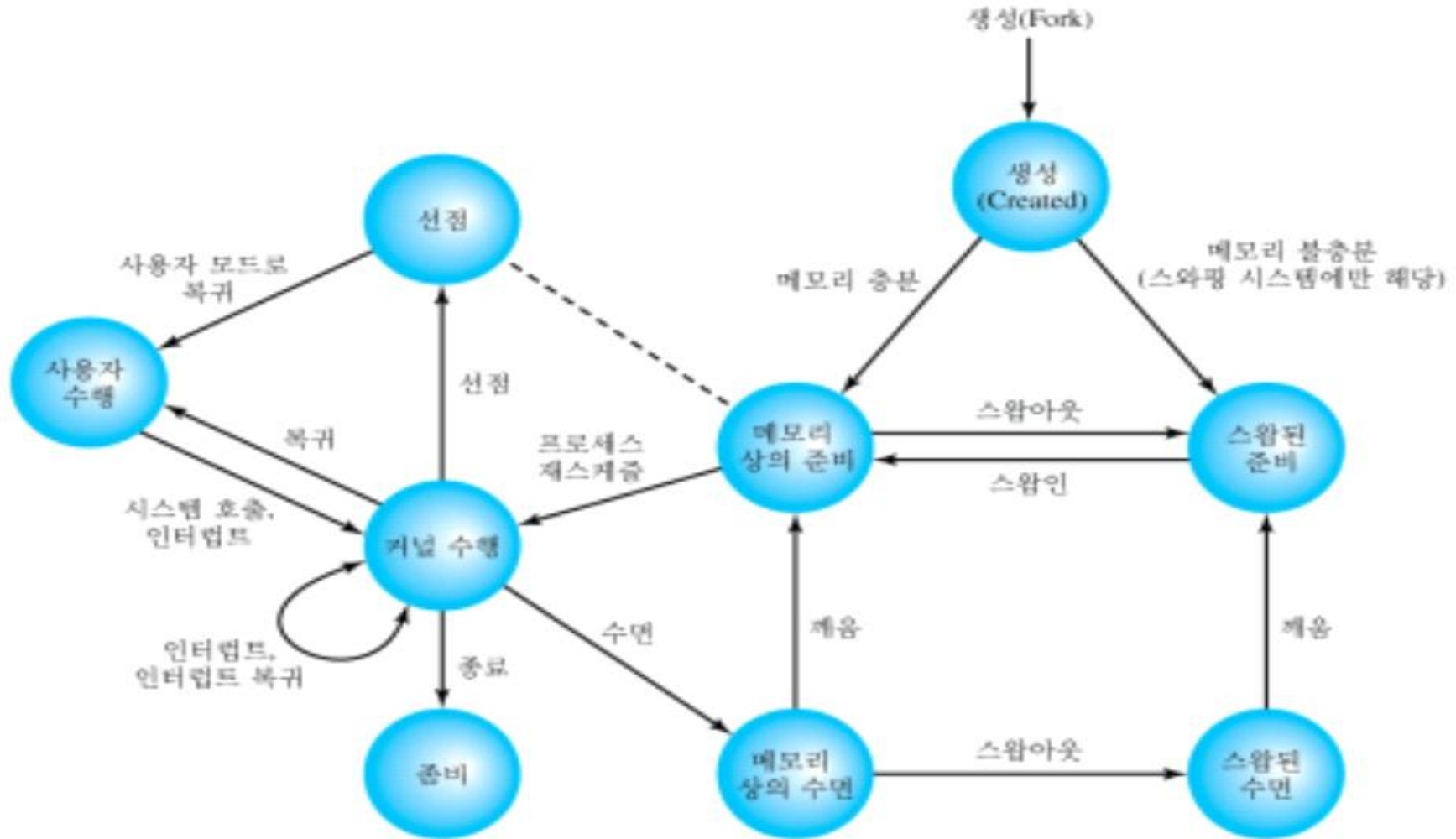
5) 프로그램이 프로세스가 되는 과정

- ① 디스크에 존재하는 프로그램의 프로그램 코드와 정적 데이터를 메모리에 load 합니다. 현대에는 Paging과 Swapping 기술로 이 과정을 늦췄습니다.
- ② 메모리의 스택 공간에 일정량의 메모리를 할당해야 합니다.
- ③ 메모리의 힙 공간에 일정량의 메모리를 할당해야 합니다.
- ④ 입출력 기능과 관련된 세 개의 file descriptor를 이용합니다.
- ⑤ 프로그램 실행을 대기 합니다.
- os는 로딩 작업(프로그램 -> 프로세스)을 느리게 수행합니다. -> os는 실행을 빨리하기 위해 code부터 올립니다.

6) 프로세스의 종료

- 어떤 OS에서는 프로세스가 자신을 생성한 프로세스에 의해서 종료되거나 부모 프로세스가 종료될 때 같이 종료합니다.
- 시분할 시스템에서 특정 사용자 프로세스는 해당 사용자가 로그오프 하거나 터미널을 끝 때 종료됩니다.
- 여러 종류의 오류 및 Fault 상태로 인해 프로세스는 종료됩니다.
- 중지 명령어는 프로세스가 완료되었다는 사실을 OS 에게 통보하기 위해 인터럽트를 발생시킵니다.

7) Process State(프로세스 상태)



*** PEARSON 책도 공부했습니다. ***

① Create(생성 상태)

- = 프로세스가 막 생성된 상태이며 프로세스를 메모리의 어느 주소 공간에 저장할 지 결정하는 단계입니다.
- 생성되면 프로세스에 pcb를 생성하고 기본적으로 Ready상태로 존재합니다.

② Ready(준비 상태)

- = Ready in Memory와 Preemption 상태를 합친 상태입니다.
- = Runnable 상태라고도 합니다.
- = 프로세스가 메모리에서 자신에게 CPU가 할당되기를 기다리고 있는 상태입니다.
- 프로세스가 Time Quantum이 만료되어서 다른 프로세스에게 CPU를 사용할 권한을 뺏기면 Preemption 상태가 됩니다. Preemption 상태도 CPU를 사용할 권한을 뺏겨서 실행상태는 아니지만 Ready 상태 입니다.
- Preemption 상태가 교재에서는 선점으로 나왔지만 우리는 CPU를 점유하는 권한을 뺏긴 Ready 상태로 기억합시다.

③ Block (수면 상태)

- = 프로세스가 OS에게 지금 즉시 수행할 수 없는 서비스(즉시 이용할 수 없는 파일 OR 가상 메모리의 공유 영역 요구)를 요청할 때 Block 상태가 됩니다.
- = I/O가 발생하여 자신은 CPU를 사용하지 않고(스케줄링에서 배제), 다른 프로세스가 CPU를 사용하도록 하는 상태입니다.
- = Sleep 상태 또는 I/O 상태라고도 합니다.
- 프로세스 수행 도중 I/O 과정이 발생하면 Block 상태가 됩니다.
- I/O가 끝나서 깨우거나 , 이벤트가 발생하면 Ready 상태가 됩니다. 이를 Wake Up 이라고 합니다.
- Block 상태에서는 절대 Running 상태가 될 수 없습니다.

④ Running(수행 상태)

= 프로세스가 CPU를 점유하여 실행되고 있는 상태 입니다.

- 사용자 수행 과 커널 수행을 합친 상태 입니다.

<사용자 수행>

= 시스템 콜 또는 인터럽트가 아니면 사용자 수행 입니다.

<커널 수행>

= 시스템 콜 또는 인터럽트가 발생하면 커널 영역에서 수행 입니다.

⑤ Zombie

= 프로세스가 종료되었지만 아직 메모리에 있으므로 다시 살릴 수 있는 상태입니다.

- 부모 프로세스가 메모리의 흔적을 없애야 합니다.

- 만약 부모 프로세스가 먼저 죽었다면 조상 프로세스가 Zombie 프로세스의 메모리의 흔적을 없애 줍니다.

⑥ Suspend(보류 상태)

= 프로세스가 메모리 공간을 뺏기는 Swap-Out이 일어나 디스크로 보내진 상태 입니다.

- 보류 상태의 프로세스는 즉시 수행될 수 없으며 , 메모리에 더 많은 프로세스를 수용하기 위해 잠시 디스크로 보내집니다.

<Ready in Swap = Ready Suspend>

= 메모리가 충분하지 못해 디스크의 Swap 영역에 저장되어 있지만 , 메모리에 올라가면 즉시 수행될 수 있는 상태입니다.

- swap에서는 프로세스를 바로 실행 할 수 없습니다.

<Block in Swap = Block Suspend>

= 대기 상태일 때 메모리 공간을 잃은 상태를 말합니다. 보류 대기의 프로세스는 특별한 경우가 아니면 입출력이나 기다리던 사건의 종류 시 보류 준비 상태로 바뀌게 됩니다.

8) 프로세스 상태 추가 개념

① 수행 상태에서 3가지 상황 발생 시에 처리 방법

상황 발생	프로세스의 상태 변화
Time Quantum Interrupt(CPU 점유 시간 초과)	CPU 사용 권한을 뺏겨서 Ready(Preemption) 상태가 됩니다.
I/O Interrupt(I/O 발생)	I/O가 발생하므로 Block 상태가 됩니다.
Exit()	프로세스를 종료시키므로 Zombie 상태가 됩니다.

② Swapping

= 프로세스의 일부나 전체를 메모리에서 디스크의 Swap 영역으로 옮기는 방법입니다.

- Swap In + Swap Out을 합쳐서 부릅니다.
- Swap In은 디스크에서 메모리로 올리는 과정이고 Swap Out은 메모리에서 디스크로 저장시키는 과정입니다.
- Swap Out 되면 프로세스는 Suspend(보류) 상태가 됩니다. Suspend 상태는 Ready in Memory , Ready in Swap은 Suspend 상태입니다.
- 시스템에서 대부분 프로세스들은 CPU 위주의 작업보다는 I/O 위주의 작업이 많아 CPU는 대부분 시간 동안 유휴 상태가 되기 때문에 더 많은 프로세스들을 준비 상태로 만들어 CPU가 더 많은 프로세스들을 실행시킬 수 있도록 해주어야 합니다.

③ ps 명령어에서의 프로세스 상태

코드분류	의미
D	IO 와 같이 중지(interrupt)시킬 수 없는 잠자고 있는(휴지) 프로세스 상태
R	현재 동작중이거나 동작할 수 있는 상태
S	잠자고 있지만 중지시킬 수 있는 상태
T	작업 제어 시그널로 정지되었거나 추적중에 있는 프로세스 상태
X	완전히 죽어 있는 프로세스
Z	죽어 있는 좀비 프로세스

코드분류	의미
<	프로세스의 우선 순위가 높은 상태
N	프로세스의 우선 순위가 낮은 상태
L	실시간이나 기존 IO를 위해 메모리 안에 잠겨진 페이지를 가진 상태
s	세션 리더(주도 프로세스)
l	멀티 쓰레드
+	포어그라운드 상태로 동작하는 프로세스

2. PCB(Process Control Block)

= OS가 CPU를 추상화한 구조체 입니다.

= 현재 프로세스의 상태와 메모리 내에서의 위치 등 OS에게 필요한 프로세스의 모든 정보를 담은 구조체 입니다.

- Linux 에서는 task_struct , 유닉스(xv6)에서는 proc.h 구조체로 존재합니다.

- 리눅스의 모든 프로세스는 각각 한 개의 task_struct를 갖고 있습니다.

<실제 Linux 환경에서 찾아 본 PCB>

```
pjw960316@pjw960316-VirtualBox: /usr/src/linux-hwe-5.4-headers-5.4.0-42/include/linux$
```

```
624 struct task_struct {
625 #ifdef CONFIG_THREAD_INFO_IN_TASK
626     /*
627      * For reasons of header soup (see current_thread_info()), this
628      * must be the first element of task_struct.
629      */
630     struct thread_info      thread_info;
631 #endif
632     /* -1 unrunnable, 0 runnable, >0 stopped: */
633     volatile long           state;
634
635     /*
636      * This begins the randomizable portion of task_struct. Only
637      * scheduling-critical items should be added above here.
638      */
639     randomized_struct_fields_start
640
641     void                    *stack;
642     refcount_t              usage;
643     /* Per task flags (PF_*), defined further below: */
644     unsigned int            flags;
645     unsigned int            ptrace;
```

```
727     struct mm_struct        *mm;
728     struct mm_struct        *active_mm;
```

- 우분투 18.04에서 실제로 task_struct를 찾아 보았습니다.

- 해당 경로에서 sched.h에 들어가면 task_struct 구조체를 찾을 수 있습니다.

- 그림에서는 극히 일부분만 캡처한 것 이고 , 실제 task_struct는 500줄이 넘어 가는 긴 구조체 입니다.

- task_struct 내부에서 mm_struct를 찾았습니다.

<실제 UNIX(xv6)환경에서 찾아 본 PCB>

```
pjw960316@pjw960316-VirtualBox:~$ cd xv6-public
pjw960316@pjw960316-VirtualBox:~/xv6-public$ ls
BUGS                elf.h               helloname_test.o    mp.d                stressfs.d
LICENSE             entry.S             helloname_test.sym  mp.h                stressfs.o
Makefile            entry.o             ide.c                mp.o                stressfs.sym
Notes               entryother          ide.d                mytest.c            string.c
README              entryother.S        ide.o                param.h             string.d
TRICKS              entryother.asm      init.asm             picirq.c            string.o
_cat                entryother.d        init.c               picirq.d            swtch.S
_echo               entryother.o        init.d               picirq.o            swtch.o
_firstsyscall       exec.c              init.o               pipe.c              syscall.c
_forktest           exec.d              init.sym             pipe.d              syscall.d
_get_prio_test      exec.o              initcode             pipe.o              syscall.h
_getmaxpid_test     fcntl.h             initcode.S           pr.pl               syscall.o
_getnumproc_test    file.c              initcode.asm         printf.c             sysfile.c
_getprocinfo_test   file.d              initcode.d           printf.d            sysfile.d
_grep               file.h              initcode.o           printf.o            sysfile.o
_hello_test         file.o              initcode.out         printrcs            sysproc.c
_helloname_test     firstsys.c          ioapic.c             proc.c              sysproc.d
_init               firstsyscall.asm    ioapic.d             proc.d              sysproc.o
_kill               firstsyscall.c      ioapic.o             proc.h              toc.ftr
_ln                 firstsyscall.d      kalloc.c             proc.o              toc.hdr
```

```

37 // Per-process state
38 struct proc {
39     uint sz;                // Size of process memory (bytes)
40     pde_t* pgdir;          // Page table
41     char *kstack;          // Bottom of kernel stack for this process
42     enum procstate state;   // Process state
43     int pid;               // Process ID
44     struct proc *parent;    // Parent process
45     struct trapframe *tf;   // Trap frame for current syscall
46     struct context *context; // swtch() here to run process
47     void *chan;            // If non-zero, sleeping on chan
48     int killed;            // If non-zero, have been killed
49     struct file *ofile[NOFILE]; // Open files
50     struct inode *cwd;     // Current directory
51     char name[16];         // Process name (debugging)
52
53     //my_memeber_value
54     int context_switch;    // Context_Swtich count
55     int priority;          // Schedule Priority
56     int first_prio;        // when I use set_prio() , this value initialized & using at
    Schedule
57 };
58

```

- 과제 3번을 하면서 공부하고 구현한 xv6에서의 PCB인 proc.h 입니다.
- 학습용으로 만들어진 OS이기 때문에 task_struct에 비해 매우 간단합니다.

<과제 2번 공부 + 5강_Process API> 09/13 ~ 09/25

1. 질의 응답

<질문>

과제 명세서상에 fork(2) wait(2) system(3)등의 괄호 안의 숫자가 의미하는 바를 찾아본 결과 명령어 section을 뜻하는 것임을 알았습니다. man을 확인해본 결과 fork(2)와 fork(3)의 명세가 분리되어있다는 사실도 알았습니다. exec의 man을 확인해보니 exec(3)으로 표기됨을 알 수 있었습니다.

<답변>

man은 온라인 메뉴얼임..여기서 번호는 본래 책으로 만들 때 볼륨 (섹션) 번호임. 요즘은 시스템 마라 다르지만 보통 8권의 책이 있었다고 생각하면 되고...

1번 사용자 명령어 (셸에서 바로 실행시키는) 2번.. 시스템 콜 3번. 사용자 라이브러리, 4번. 디바이스 등 특수 파일..5번 파일 포맷에 관한 것...6... 게임.... 8. 디몬 등...이라고 생각하면 되고 이중 중요한 것은 1/2/3번 임

질문한 것 중 요즘 fork() 도 라이브러리 함수로 있는 것 처럼 리눅스가 되어 있으나 2번으로 봐야 하고..즉 시스템 콜이고..exec 류 함수 들 6개 중 중 execve()만 시스템 콜이고 나머지는 라이브러리임. 그리고 추가로 exec() 도 리눅스는 3번으로 정의만 되어 있음....

질문에 대한 답.. exec(3)으로 구현하겠다..상관없으나..exec(3)을 쓰려면 어차피 6개 exec 류 함수. execl, execlp, ...등을 써야 함..6개 중 알아서...어떤 것을 골라 써도 됨.

2. 과제에서 사용한 시스템 콜

1) fork()

= 현재 프로세스(코드를 작성하고 있는 소스 프로그램)와 완벽하게 동일한 프로세스를 만듭니다. 현재 프로세스는 부모 프로세스가 되고 복제된 프로세스는 자식 프로세스가 됩니다.

- 보통 부모프로세스의 pid가 1233이면 자식 프로세스의 pid는 1234가 됩니다. 둘은 ps-aux를 실행해보면 완벽하게 동일한 프로세스 이며 pid만 다르다는 사실을 알게 됩니다.

```
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    int pid;

    pid = fork();
    if (pid > 0)
    {
        printf("부모 프로세스 %d : %d\n", getpid(), pid);
        pause();
    }
    else if (pid == 0)
    {
        printf("자식 프로세스 %d\n", getpid());
        pause();
    }
    else if (pid == -1)
    {
        perror("fork error : ");
        exit(0);
    }
}
```

pid > 0 의 조건문은 부모 프로세스에서의 코드를 나타냅니다.

pid == 0 의 조건문은 자식 프로세스에서의 코드를 나타냅니다.

pid < 0 은 에러입니다.

<fork() 와 exec()의 관계>

```
int pid;
pid = fork();
```

 ---> 해당 그림은 ssu_shell.c에서 fork()하는 부분입니다.

- pid = fork()를 통해 부모 프로세스인 ssu_shell.c는 자식 프로세스를 생성합니다. 이 때 자식 프로세스의 task_struct가 생성됩니다.
- ssu_shell.c를 실행해서 ./a.out을 했을 때 부모 프로세스인 ssu_shell의 pid가 101이라 치면 , 자식 프로세스의 pid는 102가 됩니다.
- 부모 프로세스는 메모리를 할당 받습니다. 교수님은 메모리를 집이라고 표현하시고 , 부모님은 집이 있다고 표현하셨습니다. 자식 프로세스는 fork()만 되어서 task_struct만 생성되었지 메모리에 할당되지 않았습니다. 자식은 집이 없고 부모님의 집에 얹혀 삽니다. 즉 , 자식 프로세스는 부모님의 메모리를 공유합니다.
- exec()을 하면 자식 프로세스가 독립하여서 자신만의 메모리를 갖고 인자로 받은 프로그램을 실행합니다.

<fork() 와 wait()의 관계>

① 자식이 종료되지 않을 때

```
5 int main()
6 {
7     int pid = fork();
8
9     for(int i=0; i<3; i++)
10    {
11        if(pid==0)
12        {
13            printf("i am child %d\n" , i);
14            while(1)
15            {}
16        }
17
18        else
19        {
20            int status;
21            wait(&status);
22            printf("i am parent\n");
23        }
24    }
25    return 0;
26 }
```

```
pjw960316@pjw960316-VirtualBox:~$ ./t
i am child 0
```

- fork()를 한번 호출하여 자식을 1개 만들어주고 반복문을 돌립니다. 하지만 자식에서 무한루프가 발생하고 자식 프로세스는 종료되지 않습니다. 원래 같으면 부모는 이를 무시한 채 'I am parent'를 출력하고 종료 되나 , wait()을 명령함으로 자식 프로세스가 끝나기 전까지 sleep 상태가 되기 때문에 'I am parent'를 출력시키지 못하고 부모도 종료 되지 않습니다.
- wait()이 없다면 자식프로세스가 무한 루프든 말든 'I am parent'를 실행시키고 부모 프로세스는 종료가 됩니다. 이때 자식은 좀비 프로세스가 됩니다.
- 자식 프로세스는 종료되기 위해서는 반드시 부모 프로세스가 자식 프로세스의 리턴 값을 읽어 들어야합니다.(wait을 써서)

② 자식이 종료 될 때

```
pjw960316@pjw960316-VirtualBox:~$ ./t
i am child 0
i am child 1
i am child 2
i am parent
i am parent
i am parent
pjw960316@pjw960316-VirtualBox:~$ cat tt.c
#include <stdio.h>
#include <unistd.h>
#include <wait.h>

int main()
{
    int pid = fork();

    for(int i=0; i<3; i++)
    {
        if(pid==0)
        {
            printf("i am child %d\n" , i);
            sleep(1);
        }
        else
        {
            int status;
            wait(&status);
            printf("i am parent\n");
        }
    }
    return 0;
}
```

자식프로세스는 sleep(1)이 3번 실행되므로 각각 1초간 실행이 되고 종료됩니다. 부모프로세스도 3번 실행이 되지만 , 모두 자식프로세스가 종료될 때 까지 기다립니다. 그러므로 3초 뒤에 'I am parent'를 3번 출력합니다.

2) wait(&state)

= 자식 프로세스가 종료할 때까지 부모 프로세스가 sleep()모드로 기다리게 됩니다.

- 부모 프로세스가 자식 프로세스 보다 먼저 종료되는 것을 방지하기 위한 기능입니다.
- 자식이 종료되면 비로소 부모 프로세스가 종료 됩니다.
- 매개변수로 자식 프로세스의 상태가 들어갑니다. ex) int state; wait(&state);

3) exec 계열 System Call

= 현재 프로세스를 새로운 프로그램으로 대체해버립니다.

ex) fork()한 자식 프로세스에서 exec를 사용하면 해당 인자에 들어간 프로그램을 실행해버립니다.

- fork()함수와 자주 같이 사용합니다.
- 6가지 계열이 존재합니다.

1. l, v: argv인자를 넘겨줄 때 사용합니다. (l일 경우는 char *로 하나씩 v일 경우에는 char *[]로 배열로 한번에 넘겨줌)
2. e: 환경변수를 넘겨줄 때 사용합니다. (e는 위에서 v와 같이 char *[]로 배열로 넘겨줍니다.)
3. p: p가 있는 경우에는 환경변수 PATH를 참조하기 때문에 절대경로를 입력하지 않아도 됩니다. (위에서 system함수 처럼)

- exec 뒤에 위의 4가지 알파벳이 오는데 이번과제에서 경험한 내용을 적으면

l을 붙이면 인자 하나 하나를 읽고 , v를 붙이면 인자 하나가 배열 형태입니다.

e는 사용하지 않았고 p를 붙이면 원래는 절대경로를 지정해 주었지만 PATH 덕분에 사용하지 명령어만 적어 주어도 됩니다.

- 개인적으로는 vp를 붙인 **execvp**가 가장 간편한 명령어 인듯합니다.

```
if(execvp(tokens[pipe_index[j]+1] , tokens+pipe_index[j]+1) == -1)
```

= tokens는 명령어 ls , -al , cat 같은 token(문자열)들로 이루어진 긴 문자열입니다. tokens[0] = ls , tokens[1] = -al , tokens[2] = cat
execvp는 NULL이 나올 때 까지 읽으므로 해당 명령어를 처리하면 ls -al cat 이므로 이상한 명령어라 에러가 됩니다. 그러므로 cat만 처리하기 위해 위의 형식처럼 tokens[2] , tokens+2를 인자로 넣어 2번째 token부터 읽도록 하여 cat을 읽고 다음은 NULL 이므로 종료됩니다.

4) exit()

① 정의 및 특징

= 해당 프로세스를 완전히 종료시킵니다.

- 자식 프로세스가 I가 0부터 1000000000까지 반복문을 돌린다고 가정합니다. 부모 프로세스는 wait()을 통해 그 긴 시간을 기다리고요. 이런 상황에서 I가 2000쯤 되었을 때 exit()을 통해 자식 프로세스를 종료시켜 자원을 해제시키고 부모 프로세스 또한 자식 프로세스의 종료 상태를 wait()의 인자로 받아서 종료합니다.

② exit() vs return;

- exit()은 프로세스 자체를 종료시켜 버리고 os에게 권한을 넘겨 버립니다.
- return은 일반 함수에서는 함수만 종료시키고 , main 함수 속에서는 프로그램 전체를 중지시킵니다.
- 기본적으로 return 이므로 return 이후의 코드를 당연히 반환 시키고 종료시킵니다.
- main 함수의 return 과 exit은 결국 하는 역할은 비슷하지만 시스템 내부적으로는 좀 다를 것 같습니다.

exit(0) = 정상 종료
exit(1) = 비정상 종료

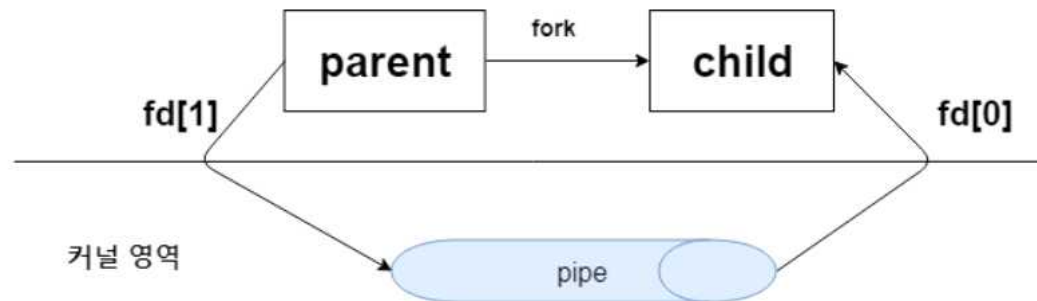
보통 문제가 없으면 0을
문제가 있으면 1을 리턴하도록 작성합니다.

참고로 /usr/include/stdlib.h 를 보면

```
/* We define these the same for all machines.  
Changes from this to the outside world should be done in `_exit'. */  
#define EXIT_FAILURE 1 /* Failing exit status. */  
#define EXIT_SUCCESS 0 /* Successful exit status. */
```

5) pipe()

- = 서로 독립된 프로세스들이 데이터를 주고 받기 위해 os가 제공하는 함수
- return 값이 0이면 성공 , return 값이 -1이면 실패입니다.



- fd[0] = 프로세스가 파이프에서 내용을 꺼내서 읽을 때 사용하는 통로 입입니다. **읽기 전용** 파일 디스크립터 = 파이프 출구

- fd[1] = 프로세스가 파이프에 내용을 쓸 때 사용하는 통로 입니다. **쓰기 전용** 파이프 파일 디스크립터 = 파이프 입구
- 수도관을 생각하면 이해하기 쉽습니다. 물을 주입할 때 물이 새지 않도록 끝을 반드시 막아야하고 , 물을 뺄 때는 더 들어오지 않도록 시작 부분을 막아야합니다.
- 단방향으로 데이터의 입출력이 되므로 **파이프의 내용을 읽으려면 close(fd[1])로 닫아야 하고 파이프에 내용을 쓰려면 close(fd[0])으로 닫아야 합니다.**

<부모의 데이터를 자식으로 보내기>

- ① 파이프 생성
- ② 부모의 데이터를 파이프에 적어야 하므로 fd[0]를 닫습니다.
- ③ fd[1]을 통해 파이프에 데이터를 씁니다.
- ④ 부모가 파이프에 더 이상 쓸 데이터가 없다면 fd[1]을 닫아서 파이프에 더 이상 쓰지 못하도록 합니다.
- ⑤ 자식이 fd[0]를 통해 파이프에서 데이터를 읽어 옵니다.

<https://reakwon.tistory.com/80>

6) dup2(int old_fd , int new_fd)

ex) dup2(fd[1] , 1)

- = 이제부터 모든 표준 출력은 fd[1]을 통해 파이프 안으로 들어가게 됩니다. 그래서 원래는 표준 출력(프로세스에서 출력하는 printf or execvp)이 셸을 통해 사용자에게 보여야 하는데 파이프로 들어갔기 때문에 사용자에게 보이지 않습니다.
- dup2(fd[1] , stdout) 은 dup2(fd[1] , 1)과 같습니다.

7) system()

- = 터미널에서 실행 시킬 수 있는 명령어를 실행시켜주는 메소드입니다.
- fork() -> wait() -> exec()를 합친 것과 같습니다.
- Vim에서 셸의 명령인 ls를 실행합니다.
- stdlib.h안에 존재하므로 반드시 헤더를 선언해야 합니다.


```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     system("ls");
7     exit(0);
8 }
```

```
User@DESKTOP-BAGIJS2 ~
$ ./a.exe
a.exe      desktop.ini  ssu_shell.zip  test.c
'Cygwin Source 폴더의 역할 .txt'  ssu_shell.c  t1.c
```

8) 과제에서 가장 어려웠던 dup2와 pipe를 구현한 내 코드

```

for(int i=0; i<token_cnt; i++)
{
    //using pipe
    for(int j=0; j<pipe_cnt; j++)
    {
        //first pipe_index = -1 -> should include the start command
        if((i==0&& j ==0) || pipe_index[j] == i)
        {
            if(pipe(fd) == -1)
            {
                printf("pipe error\n");
                exit(1);
            }
            pid = fork();

            //son
            if(pid == 0)
            {
                dup2(tmpfd,0); //now all stdin is from pipe
                if(j!= pipe_cnt-1)
                {
                    dup2(fd[1],1); //now all stdout enter to pipe
                    tokens[pipe_index[j+1]] = NULL;
                }
                close(fd[0]); //close pipe's output door
                if(execvp(tokens[pipe_index[j]+1] , tokens+pipe_index[j]+1) == -1)
                {
                    printf("SSUShell : Incorrect command\n");
                    exit(1);
                }
                exit(0);
            }
        }
    }
}

```

```
//parent
else if(pid > 0)
{
    wait(&status);
    close(fd[1]); //close pipe's input door
    tmpfd = fd[0];
}
else
{
    exit(0);
}
break;
}
```

- tmpfd , fd는 루프 외부에 선언하였습니다.

3. 리눅스 환경에서 bin 폴더

= binary의 약자입니다. 일반적으로 사용하는 리눅스 기본 명령어 프로그램들이 모두 존재합니다.

```
pjw960316@pjw960316-VirtualBox: /bin$ ls
bash          ed            lsblk         openvt        systemd-machine-id-setup
brltty        efibootdump  lsmod         pidof         systemd-notify
bunzip2       efibootmgr   mkdir         ping          systemd-sysusers
busybox       egrep        mknod         ping4         systemd-tmpfiles
bzip2         false       mktemp        ping6         systemd-tty-ask-password-agent
bzcat         fgconsole   more          plymouth      tar
bzdiff        fgrep        mount         ps            tempfile
bzegrep       findmnt      mountpoint    pwd           touch
bzexe         fuser       mt            rbash         true
bzfgrep       fusermount  mt-gnu        readlink      udevadm
bzgrep        getfacl     mv            red           ulockmgr_server
bzip2         grep        nano          rm            umount
bzip2recover  gunzip      nc            rmdir         uname
bzless        gzexe       nc.openbsd   rnano         uncompress
bzmore        gzip        netcat       run-parts     unicode_start
cat           hciconfig   networkctl   sed           vdir
chacl         hostname    nisdomainname setfacl        wdcctl
chgrp         ip          ntfs-3g       setfont       which
chmod         journalctl  ntfs-3g.probe setupcon       whiptail
chown         kbd_mode    ntfsctl       sh            ypdomainname
chvt          kill        ntfscluster   sh.distrib    zcat
cp            kmod        ntfsncmp      sleep         zcmp
cpio          less        ntfsfallocate ss             zdiff
dash          lessecho    ntfsfix       static-sh     zegrep
date          lessfile    ntfsinfo      stty          zfgrep
dd            lesskey     ntfsls        su            zforce
df            lesspipe    ntfsmove      sync          zgrep
dir           ln          ntfsrecover   systemctl     zless
dmesg         loadkeys    ntfssecaudit systemd        zmore
dnsdomainname login        ntfstruncate  systemd-ask-password znew
domainname    loginctl    ntfsusermap   systemd-escape
dumpkeys      lowntfs-3g ntfswipe      systemd-hwdb
echo          ls          open          systemd-inhibit
```

- Tokenize 함수는 2차원 문자열 배열을 사용합니다. 2차원 배열인데 행에 문자열이 저장되고 그 들은 모두 제각기 다른 크기를 갖고 있습니다.

ex) 1행 문자열은 size가 30인데 2행 문자열은 size가 5입니다.

<6강_Limited Direct Execution> 9/16 ~ 9/27

1. Kernel Mode & User Mode

= 커널 메모리를 건드려서 지워 버리는 코드를 적은 프로그램을 쉽게 수행시켜 버리면 컴퓨터가 망가지므로 주요 운영체제 부분을 함부로 건드릴 수 없게 모드를 나눕니다. (Dual-Mode)

- 메모리상에서 이해를 해야 합니다. 4GB의 메모리를 기준으로 1GB는 커널 메모리(=커널 영역), 나머지 3GB는 유저 메모리(=유저 영역)입니다.

- Kernel Mode는 커널 메모리 + 유저 메모리 접근 가능(사용), User Mode는 유저 메모리만을 접근 가능(사용)합니다.

1) Kernel Mode (커널 모드)

= OS의 코어인 커널에 진입하여, 모든 리소스에 접근할 수 있는 권한을 얻어, 모든 명령을 수행 하고 레지스터와 메모리를 완전히 제어할 수 있는 모드 입니다.

= 보호 모드, 특권 모드, 시스템 모드, 제어 모드(Control Mode)라고도 불립니다.

- 유저 모드에서 System call을 호출 하면 소프트웨어 인터럽트(OS의 entry.s가 사용자의 접근을 검사함)가 발생합니다. 소프트웨어 인터럽트가 걸리면 OS가 하드웨어의 도움을 받아서 보호모드로 바꿉니다.

- 사용자가 OS 서비스를 호출 할 때에도 Kernel Mode로 변경됩니다.

- mode bit 가 0 입니다.

2) User Mode

= 일부 명령어만 처리할 수 있는 제한된 모드이므로 커널영역에 접근할 수 없습니다.

- 커널 모드에서 모든 명령어를 수행하고 종료하면 Return From System Call로 인해 User Mode로 돌아옵니다.

- mode bit가 1입니다.

2. Switching

1) Context

= task_struct에 저장되어 있는 멤버들을 의미합니다.

- H/W(Register) 문맥 : thread_struct
- Memory 문맥 : mm_struct
- System 문맥 : task_struct 전체를 의미합니다.

<Example : xv6의 proc.h에서 볼 수 있는 Context>

```
✓ //PAGEBREAK: 17
  // Saved registers for kernel context switches.
  // Don't need to save all the segment registers (%cs, etc),
  // because they are constant across kernel contexts.
  // Don't need to save %eax, %ecx, %edx, because the
  // x86 convention is that the caller has saved them.
  // Contexts are stored at the bottom of the stack they
  // describe; the stack pointer is the address of the context.
  // The layout of the context matches the layout of the stack in swtch.S
  // at the "Switch stacks" comment. Switch doesn't save eip explicitly,
  // but it is on the stack and allocproc() manipulates it.
✓ struct context {
    uint edi;
    uint esi;
    uint ebx;
    uint ebp;
    uint eip;
};
```

2) Context Switching(문맥 교환)

= 현재 실행 중인 프로세스의 문맥(위의 그림의 레지스터 값)을 Kernel Memory의 스택 영역에 저장하고 다음에 실행될 프로세스의 Kernel Memory의 스택 영역에 해당 레지스터 값을 복원하는 작업.

- 프로세스 마다 존재하는 task_struct 구조체는 리눅스 커널에서 매우 중요한 자료구조이며 , 해당 프로세스의 커널의 스택영역에 존재합니다.
- 문맥교환을 할 때 swtch() 함수가 호출 됩니다.

```
swtch(&p->context, mycpu()->scheduler);
```

- 모드의 전환보다 프로세스의 교환이 훨씬 많은 작업을 요구합니다.
- 프로세스의 상태가 바뀝니다.
- exit() -> schedule() -> 문맥교환 -> Zombie
- I/O() -> schedule() -> 문맥교환 -> Block
- Time Quantum Over -> schedule() -> 문맥교환 -> Ready(Preemption)
- 현대의 CPU는 하나의 문맥 교환에 1 micro sec가 소요됩니다. *** micro sec = 1/100만 sec ***

3) Mode Switching(모드 전환)

= '커널 모드 -> 사용자 모드' OR '사용자 모드 -> 커널 모드'로 전환하는 것을 모드 전환 이라고 합니다.

- Interrupt OR System-Call 발생 시에 모드 전환이 발생합니다.
- 모드의 전환은 프로세스의 상태를 바꾸지 않습니다.

3. Interrupt

= H/W Interrupt 와 S/W Interrupt로 나눕니다.

- Interrupt 와 Trap을 과거에는 혼용해서 사용했지만 지금은 명시적으로 구분됩니다.

1) H/W Interrupt

= Interrupt라 불리는 것이 거의 H/W Interrupt 입니다.

- 하드웨어 에서 오는 **비동기적**인(언제 불릴지 모르는) 외부 사건에 의해 발생 합니다.

ex) Clock Interrupt , I/O Interrupt

2) S/W Interrupt

① Trap

= 커널 내부에서 문제가 생겼을 때(오류 , 예외상황) 발생합니다.

= 불법적인 파일 접근 시도처럼 , 현재 수행되고 있는 프로세스에서 생성되는 오류나 예외 조건으로 발생합니다.

- **비동기적**으로 호출됩니다.

ex) Page-Fault , Divide and Zero

② System Call(=Supervisor Call , =Sys-Call)

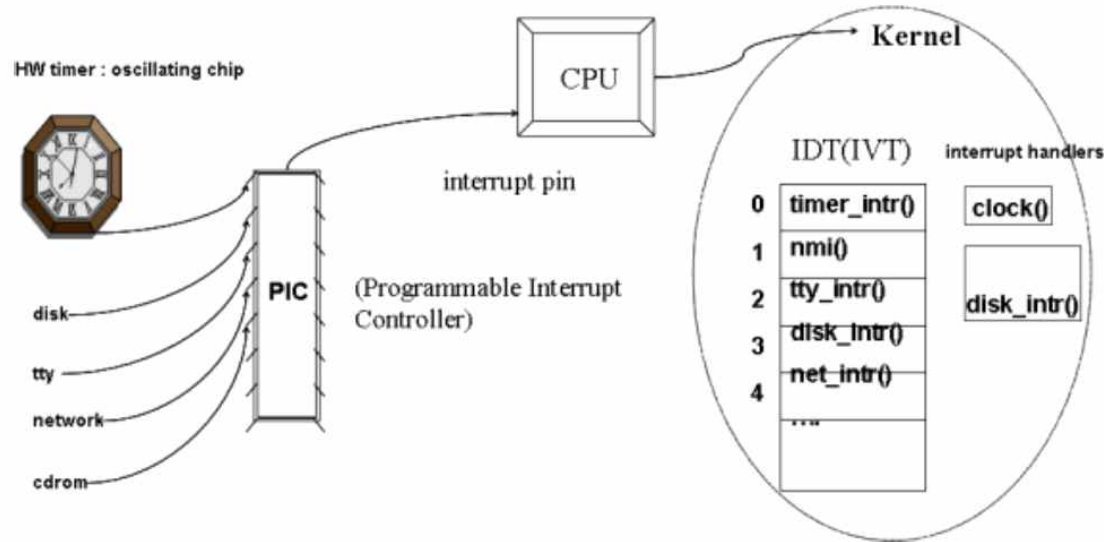
= 사용자가 프로그램 내부에서 System Call을 호출 시켜서 발생합니다.

- **동기적**(언제 불릴지 압니다)으로 호출됩니다.

ex) open(), fork(), read(), write(), close(), exec(), exit(), dup()

3. Interrupt 처리 과정

1) H/W Interrupt와 S/W Interrupt 중에 Trap을 처리하는 과정



- 그림에서 CPU + CPU 오른쪽 부분이 OS 관련 내용 입니다.

- IDT(=Interrupt Descriptor Table) = IVT(Interrupt Vector Table)

① PIC(H/W의 종류)에서 다른 H/W에서 발생한 인터럽트(0 과 1의 반복)를 쉽게 변환(Index 값)해서 CPU에게 전달합니다.

② CPU의 도움을 받아서 Kernel내부에 존재하는 IDT를 참고합니다.

③ IDT에는 H/W Interrupt에 관련된 함수들의 주소가 배열형식으로 저장되어 있습니다.

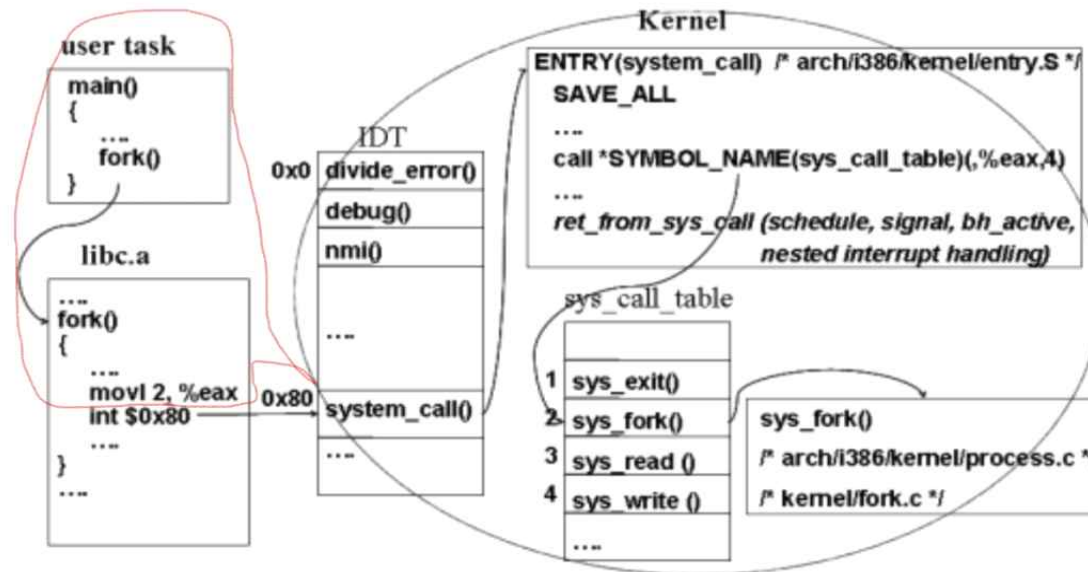
④ CPU에서 받은 Index값에 맞는 함수의 주소를 찾아가기 위해 함수가 저장되어 있는 Interrupt Handler로 점프(분기) 하여 해당 함수를 실행하고 돌아옵니다.

<Trap의 정도에 따른 OS의 처리 반응>

- Trap이 치명적이면 현재 수행되고 있는 프로세스는 종료 상태로 만들고 , 프로세스를 교환합니다.
- Trap이 치명적이지 않으면 오류의 특성에 따라 OS는 어떤 복구 함수를 수행 하거나 단순히 사용자에게 통보하거나 , 프로세스를 교환합니다.
- OS는 관련 오류 또는 예외 조건이 치명적인지 아닌지를 판단합니다.

2) S/W Interrupt 중 System Call을 처리하는 과정

시스템 호출 처리 과정 II



- 기본적으로 H/W Interrupt와 S/W Interrupt의 Trap을 처리하는 과정과 똑같지만 한 가지의 추가 작업이 존재합니다.

- movl 2 , %eax = Register_eax에 값 2를 넣으라는 어셈블리 명령어 입니다.

① 사용자 프로그램의 코드에서 fork()를 실행합니다.

② libc.a(라이브러리)와 링크가 되고 어셈블리어로 **컴파일**이 됩니다.

③ fork()에 관한 함수가 자동으로 만들어 집니다.

④ fork()의 body 부분에서 movl 2 , %eax를 통해 ax 레지스터에 값 2를 넣어 줍니다.

⑤ int %0x80를 통해 시스템 콜 인터럽트를 발생시킵니다. 이 순간에 **OS는 사용자 모드에서 커널 모드로 전환**됩니다. 시스템 콜이 아닌 다른 인터럽는 Interrupt Handler로 점프합니다. 그러나 시스템 콜은 특수한 Interrupt Handler인 entry.S로 점프합니다.

- ⑥ entry.S에서 ax 레지스터의 값 2를 참조하고 sys_call_table에 인자로 2를 전해줍니다.
- ⑦ sys_call_table에서 2번 index에 해당하는 함수의 주소를 참조합니다.
- ⑧ 참조한 함수가 저장되어 있는 System Call Handler로 점프하여 실행합니다.

5. 병행성 문제

- 인터럽트가 발생하고 있는 상황에 또 인터럽트가 발생하면 어떻게 처리할 지는 중요한 문제입니다.
 - 리눅스는 중첩 인터럽트가 불능이라 중첩되는 인터럽트를 무시하는 OS 입니다.
 - 하지만 RTOS(Real Time OS)(실시간 OS)는 중첩 인터럽트를 해결 합니다
- ex) 자동차 , 비행기에서 사용하는 OS는 RTOS 입니다

<2가지 RTOS>

- Hard RTOS : 완벽한 실시간 처리가 없으면 죽을 수 있는 상황일 때 사용하는 RTOS
- Soft RTOS : 완벽한 실시간 처리가 없어도 절대 죽지 않을 때 사용하는 RTOS = ex) Youtube 버퍼링

<7강_Scheduling> 9/23 ~ 10/2

1. 용어 및 사전 지식

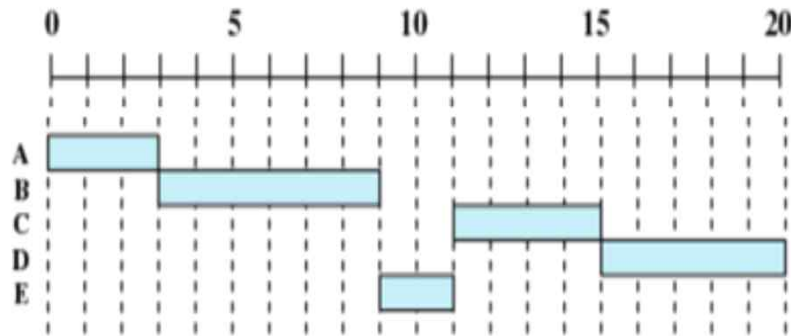


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

*** 필기에서 작업(Task)은 프로세스와 동일하게 생각해도 무방합니다. ***

1) Arrival Time

= 프로세스의 도착시간 입니다. 다시 말해 , **프로세스가 메모리에 올라온 시간** 입니다.

- 바로 실행되어서 Start Time과 Arrival Time이 같아질 수 있지만 , 대부분 Ready 상태 입니다.

ex) B : 2초

2) Start Time

= **프로세스가 실행되는 시간** 입니다.

= 그림에서는 파란색 막대기의 시작 부분입니다.

ex) B : 3초

3) End Time

= 프로세스가 종료되는 시간 입니다.

= 그림에서는 파란색 막대기의 끝 부분입니다.

ex) B : 9초

4) Service Time

= 프로세스가 실행 된 시간 입니다.

= End Time - Start Time

ex) B : 6초

5) Turnaround time(TA)

= 작업 반환 시간

= End Time - Arrival Time

= ex) b : 7초

6) Response Time(R)

= 작업 응답 시간

= Start Time - Arrival Time

= ex) b : 1초

7) Time Quantum(TQ)

= Cpu를 사용할 수 있는 시간.

8) Overhead

= 어떤 일을 처리하기 위해 들어가는 간접적인 처리 시간 입니다. Process_A를 단순하게 처리하는데 10초가 걸리지만 안정성을 위해 부가적인 작업을 추가하여 실제로 15초가 걸렸다면 오버헤드는 5초가 됩니다.

2. Non_Preemption Scheduling

= 중간에 CPU를 뺏기지 않으므로 다음 프로세스는 반드시 이전 프로세스가 모든 작업을 완료하고 종료되어야 실행 할 수 있습니다.

1) FIFO Scheduler

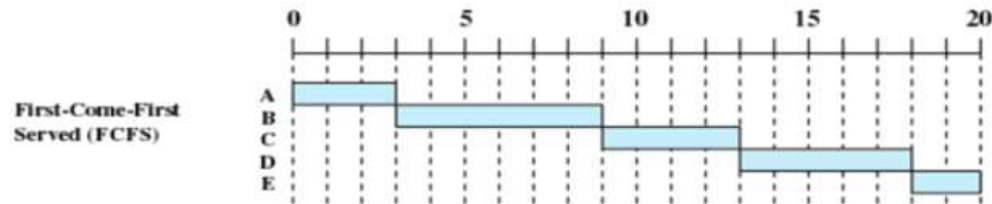


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

= First In First Out = Arrival Time이 빠른(먼저 메모리에 올라온) 프로세스를 우선적으로 실행합니다.

- 단순하고 구현하기가 쉽습니다.

- Convoy Effect(호위병 효과)라는 단점이 존재합니다.

ex) 이마트에서 계산할 때 내 앞에 100만원 어치를 계산하는 아줌마가 있고 , 나는 껌 하나를 계산하려 합니다. 나는 매우 짧은 시간이면 되지만 늦게 왔기 때문에 아줌마를 기다려야 하므로 매우 슬프습니다.

<Convoy Effect>

= 왕(짧은 작업)은 호위병(큰 작업)보다 앞으로 나가지 않습니다.

① Case 1

= Process_A가 0초에 들어오고 CPU를 1000초나 사용합니다. Process_B는 0.000001초에 들어오고 CPU를 1초만 사용합니다. 하지만 Process_A가 미세한 차이지만 먼저 들어왔기 때문에 Process_B는 1000초를 기다려야 합니다.

- 평균 TA는 $(1000+1001)/2 = 1000.5$, 평균 R은 $(0 + 1000)/2 = 500$ 입니다.

② Case 2

= Process_A가 0초에 들어오고 CPU를 1초만 사용합니다. Process_B는 0.000001초에 들어오고 CPU를 1000초나 사용합니다.

- 평균 TA는 $(1+1001)/2 = 501$, R은 $(0 + 1)/2 = 0.5$

- Case 1과 Case 2의 차이는 누가 0.000001초 차이로 먼저 들어왔는지에 대한 결과입니다. 그렇지만 결과는 엄청난 차이를 보여줍니다. 이렇듯 Case 2가 Case 1보다 훨씬 좋은 성능을 가짐에도 불구하고 어떠한 경우에도 먼저 들어온 놈을 먼저 처리하는 단점을 Convoy Effect라고 합니다.

2) Shortest Process Next(SPN) = Shortest Job First(SJF)

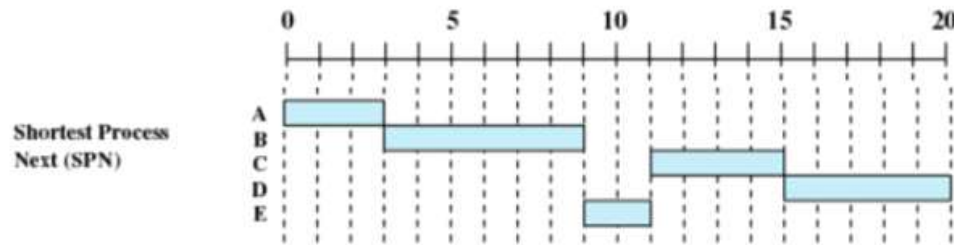


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

= **작업 시간이 짧은 프로세스**부터 우선적으로 처리 합니다.

- FIFO의 Convoy Effect를 해결하기 위해 만들어 졌습니다. 그럼에도 불구하고 실제 성능은 크게 좋아지지 않았습니다.

- 작업 시간이 짧은 프로세스를 아는 것은 현실적으로 불가능 합니다.

- 암기 팁 : SPN은 NP(Non - Preemption)

- **Starvation(기아)** 상태가 발생하는 단점이 존재합니다.

ex) 이마트에서 계산할 때 내 앞에 100만원 어치를 계산하는 아줌마가 있고 , 아줌마 보다 늦게 온 초등학교 1000명이 껌 하나 썩을 계산하려 합니다. 착한 아줌마는 초등학교 1000명 모두를 먼저 계산하도록 해줍니다. 하지만 아줌마는 슬픕니다.

3) Highest Response Ratio Next(HRRN)

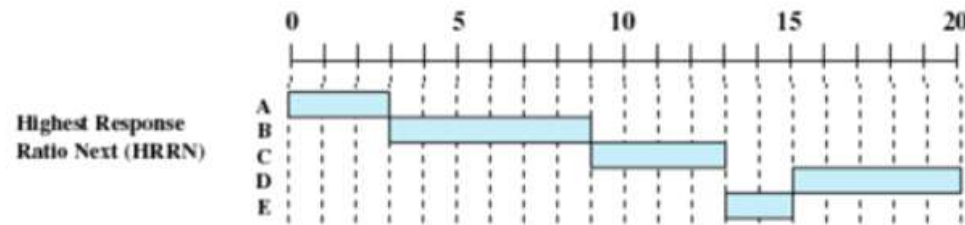


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

<Waiting time>

= 현재 시간 - Arrival time

= 스케줄링을 위해 스케줄 함수를 호출할 때 $(\text{Waiting time} + \text{Service Time}) / \text{Service Time}$ 의 값을 보고 가장 큰 프로세스를 실행시킵니다. 즉, 대기시간을 고려한 스케줄링 기법입니다.

- SPN의 단점인 기아현상을 해결하기 위한 기술입니다. (SPN의 보완된 기술)

- SPN에서는 9초에 E(껌 산 초등학생)가 들어가는데, HRRN은 그래도 오래 기다리신 C(아줌마)가 E보다 먼저 실행됩니다.

- 9초에 스케줄 함수를 호출하면, $C(5\text{초 기다림} + 4\text{초 서비스}) / 4\text{초 서비스}$, $D(3 + 5) / 5$, $E(1 + 2) / 2$ 이므로 값이 가장 큰 C를 실행 시킵니다.

3. Preemption Scheduling

= 중간에 CPU를 뺏을 수 있으므로 다음 프로세스는 이전 프로세스가 작업을 완료하지 않았어도 강제로 종료하고 자신이 실행 될 수 있습니다.

1) Shortest Time-to-Completion First(STCF) = Shortest Remaining Time(SRT)

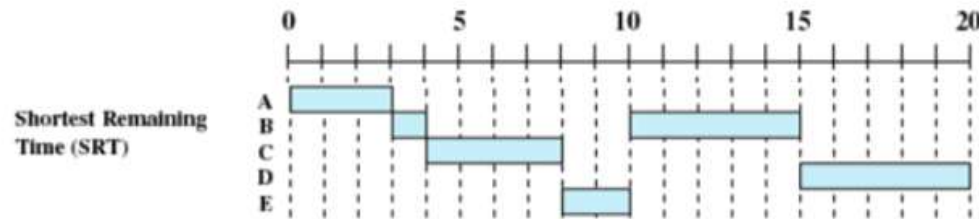


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

= 새로운 프로세스가 도착하면 남아 있는 시간을 우선으로 하는 스케줄링 기법입니다.

- SJF에 Preemption 기능을 추가한 SJF의 보완된 기술이므로 PSJF라고도 합니다.

- 중간에 무작정 뺏기는 경우와 Time Quantum이 지나서 뺏기는 경우로 나눌 수 있습니다.(이 경우는 어느 정도 시간을 보장해줍니다.)

- 2초에 B가 들어오지만 , A는 Service Time중에 2초를 이미 수행했으므로 1초의 Service Time이 남았고 B는 아직 실행된 적이 없으므로 Service Time이 6초가 남았습니다. 그러므로 남아 있는 시간이 짧은 A가 수행 됩니다.

- 4초에 C가 들어오면 B는 5초가 남았고 , C는 4초가 남았으므로 B를 중지시키고 C를 수행 시킵니다.

- 6초에 D가 들어오면 B는 5초가 남았고 , C는 2초가 남았고 D는 5초가 남았으므로 C를 계속 수행시킵니다.

- 8초에 E가 들어오면 B는 5초가 남았고 , C는 모두 수행해서 종료되었고 , D는 5초가 남았고 E는 2초가 남았으므로 E를 수행 시킵니다.

- 10초에 B는 5초가 남았고 , D도 5초가 남았고 , E는 모두 수행해서 종료 되었습니다. B와 D가 남은 시간이 같으므로 먼저 들어온 B를 수행 시킵니다.

- 15초에 남은 D를 수행시킵니다.

2) Round Robin(RR)

Round-Robin
(RR), $q = 1$

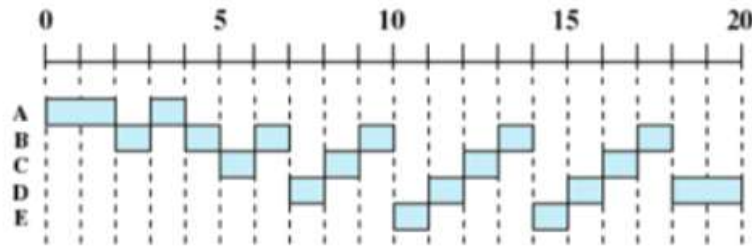


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

= 모든 프로세스에게 TQ를 부여하여 골고루 돌아가며 CPU를 사용하는 스케줄링 기법입니다.

- TQ를 무한대로 길게 하면 non preemption이 되어 버립니다.

- TQ를 너무 짧게 하면 스케줄링이 많아져서 trivial(사소한) time인 Scheduling Time이 더 이상 trivial 하지 않고 오버헤드가 커지게 됩니다.

- 작업량이 많은 프로세스는 스케줄링이 많아져서 trivial(사소한) time인 Scheduling Time이 더 이상 trivial 하지 않고 오버헤드가 커지게 됩니다.

- TQ가 끝날 때 마다 스케줄 함수를 호출하여 큐의 가장 앞쪽에 저장된 프로세스를 실행 시킵니다.

- TQ가 1초라면 1초일 때 A는 CPU를 사용 할 권한을 잃고 종료되어야 합니다. 하지만 1초일 때 메모리상에 대기 중인 다른 프로세스가 없으므로 다시 A가 TQ 1초를 부여 받습니다. 2초일 때는 메모리상에 대기 중인 다른 프로세스 B가 있으므로 B를 실행 시킵니다. 큐의 측면으로 생각해보면 대기 중인 다른 프로세스가 없다는 의미는 큐가 비어있다는 뜻이므로 자기 자신이 큐에 올라가자마자 나오게 됩니다.

- 위의 자료들과 관련 없는 예시 입니다. 컴퓨터에 프로세스는 A , B , C만 있다고 가정 합시다. A가 실행되었고 실행 도중에 B가 메모리에 올라 왔습니다. 그러나 TQ가 끝나지 않았으므로 A가 계속 실행되고 , B는 큐 자료구조에 저장되어 A의 TQ가 끝나고 자신이 실행되기를 기다립니다. A의 TQ가 끝나고 B가 실행되려 할 때 C가 메모리에 올라오면 일단 B는 실행되는데 문제가 없지만 큐 자료구조에 A를 먼저 저장할 지 C를 먼저 저장할 지 정해야합니다. 결론부터 말하면 **스케줄링 기법을 작성할 때 일관성**만 있으면 되지 A가 먼저 인지 C가 먼저 인지 중요하지 않습니다. 그러나 언제는 A를 먼저 넣고 , 언제는 C를 먼저 넣는 스케줄링은 불가능합니다. 여담으로 ,교수님께서서는 새로 메모리에 올라온 C를 A보다 먼저 큐에 넣습니다.
- 현대 os의 기반인 스케줄러 이므로 가장 실제 os의 스케줄링 기법과 유사합니다.

3) 기본 MLFQ

① 정의 및 특성

= Multi-level Feedback Queue

= Round Robin은 프로세스의 작업 크기와 관련 없이 모든 프로세스가 똑같은 TQ를 갖는 단점이 있습니다. 이 단점을 보완하기 위해 큐를 계층화하여 여러 개를 사용하는 스케줄러입니다.

- 하나의 큐에 있는 프로세스들의 우선순위는 동일합니다.
- 프로세스가 메모리에 올라오면 무조건 가장 높은 계층의 큐에 배치합니다.
- 높은 계층의 큐는 TQ가 짧고 CPU를 사용할 확률이 높습니다.(=우선순위가 높습니다.)
- 낮은 계층의 큐는 TQ가 길고 CPU를 사용할 확률이 낮습니다.(=우선순위가 낮습니다.) 낮은 계층의 큐의 프로세스가 실행이 되려면 그 위의 계층의 모든 큐가 비어있어야 합니다.
- I/O 작업을 많이 하는 프로세스는 높은 계층의 큐에 있을 것 이고 , CPU 작업을 많이 하는 프로세스는 낮은 계층의 큐에 있을 것입니다.
- 같은 계층의 큐에서는 먼저 들어온 프로세스부터 실행합니다.
- 주어진 TQ를 모두 사용하면 우선순위가 낮아져서 다음 큐로 내려갑니다.
- 주어진 TQ전에 I/O작업을 하여 CPU를 양도하면 , 우선순위가 변화하지 않게 되어 같은 큐로 다시 들어갑니다.
- 변형 MLFQ도 이 특성들은 모두 만족합니다.

<Example 1>

3개의 큐가 존재한다고 가정합니다(Queue_A , Queue_B , Queue_C).

- Top 계층에는 A가 있고 , Middle 계층에는 B가 있고 , Bottom 계층에는 C가 있습니다. C는 상위 계층인 A 와 B가 비어있어야 C안에 들어 있는 프로세스를 실행 시킬 수 있으므로 CPU를 사용하기가 어려울 것 이지만 , 주어진 TQ는 매우 깁니다.
- A의 TQ는 1초 , B의 TQ는 2초 , C의 TQ는 4초.
- 작업 우선순위는 $A > B > C$

<Example 2>

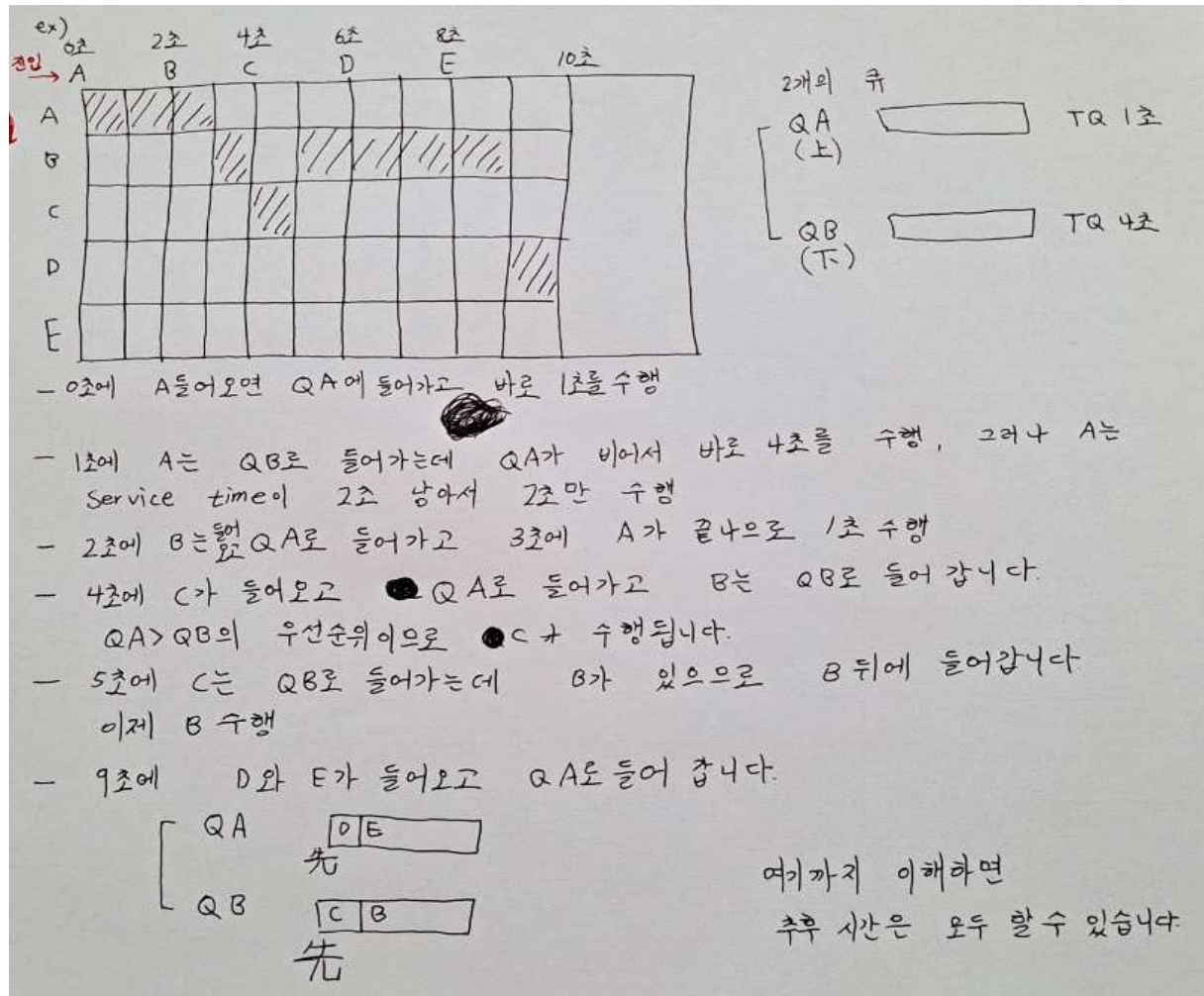


Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

② 기본 MLFQ의 문제점

- **Starvation** 발생 : SPN 방식처럼 , 높은 계층의 큐에 계속 프로세스가 들어오면 낮은 계층의 큐에 있는 프로세스는 실행되지 못합니다.
- **Gaming** 발생 : CPU를 많이 쓰는 프로세스가 TQ의 99%는 CPU를 쓰고 끝날 때 1%만 I/O를 하면 계속 동일 큐에 머무를 수 있습니다.
- **프로세스의 특성 변화에 대처 불가** : 초반에는 CPU를 많이 써서 낮은 계층의 큐로 내려갔는데 나중에는 I/O만 하는 작업이면 부당함이 생깁니다.

4) 변형 MLFQ

- = 기본 MLFQ의 문제점을 해결하기 위해 고안된 새로운 MLFQ입니다.
- 현대 OS는 변형 MLFQ를 기반으로 만들어 졌습니다.

① 새로 추가한 Rule_1 : Boosting

- = **일정 시간이 지나면 시스템 내에 존재하는 모든 프로세스들을 최상위 큐로 이동시킵니다.**
- Boosting을 너무 자주 하면 I/O 작업과 대화형 작업이 불리하고 , 너무 가끔 하면 Starvation이 생깁니다.
- Boosting의 Frequency를 정하는 문제는 아직도 난제입니다.
- Starvation과 프로세스의 특성 변화에 대처 불가능한 문제를 해결합니다.

② 새로 추가한 Rule_2 : Gaming Tolerance(게임 내성)

- = **CPU 사용시간의 합을 계산하고 , 그 값이 프로세스가 존재하는 큐의 TQ를 넘어가면 낮은 단계의 큐로 내려 보냅니다.**

ex) TQ의 99%일 때 I/O를 하는 프로세스는 계속 동일 큐에 머무릅니다. 하지만 CPU 사용 시간의 합을 이용하면 , CPU를 99% 차지하고 I/O를 하므로 한번은 운이 좋게 동일 큐로 가지만 , 다음에 수행될 때는 CPU를 TQ의 1%를 차지한 순간 합이 100%가 되므로 낮은 단계의 큐로 내려갑니다.

- 모든 프로세스 마다 CPU 사용시간을 측정해야 하므로 오버헤드가 발생합니다.
- Gaming을 해결합니다.

4. Scheduling Performance

성능 비교

	선택함수	결정모드	응답시간	문맥교환비용	프로세스에게 미치는 영향	기아
FCFS	$\max[w]$	Non preemption	길 수 있음	최소	짧은 프로세스에게 불리	X
Round Robin	상수	Preemption	짧은 프로세스 유리	최소	모든 프로세스에게 공정	X
SPN	$\min[s]$	Non preemption	짧은 프로세스 유리	커질 수 있음	긴 프로세스에게 불리	O
SRT	$\min[s-e]$	Preemption	좋음	커질 수 있음	긴 프로세스에게 불리	O
HRRN	$\text{Max}(w+s/s)$	Non preemption	좋음	커질 수 있음	어느 정도 공정한 편	X

w = 지금까지 실행 또는 대기하면서 시스템 내에 기다린 시간
 e = 지금까지 실행하는 데에 소요한 시간
 s = e 를 포함하여 프로세스가 요구한 총 서비스 시간

5. Incorporating I/O

= 모든 프로세스는 I/O가 있으므로 이제는 I/O를 포함해서 생각합시다.

= 프로세스가 CPU를 사용하고 있는 순간에는 다른 프로세스는 CPU를 사용하지 못하는 단점이 있습니다. 반면 , 프로세스가 I/O 작업을 하는 동안에는 CPU를 사용하지 않으므로 이 때 다른 프로세스는 CPU를 사용 할 수 있습니다.

- 결국 CPU를 사용하는 시간이 짧은 프로세스(껌 산 초등학생)는 I/O 작업이 많은 프로세스 입니다.

- I/O 하는 순간에는 프로세스를 Block 상태로 변경하고 , 다른 프로세스가 CPU를 사용하도록 합니다. -> CPU Utilization 향상

- 현대 OS는 가능하면 CPU를 많이 쓰는 프로세스보다 I/O 작업을 많이 하는 프로세스를 더 우선해서 스케줄링 합니다. 이를 위해 task_struct에 counter(CPU를 사용하는 시간) 변수를 사용합니다.

6. 초기 유닉스 스케줄링

- 지금까지는 이론을 배웠고 , 이번 소단원은 실제로 초기 유닉스에서 사용한 스케줄링을 배웁니다.

1) 용어

- Counter 와 Nice 모두 task_struct에 존재하는 변수입니다.

① Counter

= 프로세스가 CPU를 사용하는 시간을 정확하게 클럭으로 나타냅니다.

ex) clock_time이 60hz라면 1초에 60번 진동하는 것 이므로 , 1초가 지나면 Counter의 값은 60이 됩니다.

② Nice

= 프로세스의 스케줄 우선순위입니다.

= Nice = 최근의 CPU 사용량/2 + 60(=기본 수준 사용자 우선순위 값)

- Nice의 값이 작으면 우선순위가 큰 것 이며 CPU를 사용할 확률이 높아집니다.

- Nice의 값이 크면 우선순위가 작은 것 이며 CPU를 사용할 확률이 낮아집니다.

③ Decay

= CPU를 사용 한 시간이 오래 됨을 의미하는 함수입니다. CPU를 사용 한 시간을 감소시킵니다. 보통 반감 시킵니다.

= Counter를 Input으로 넣고 Counter/2를 Output으로 합니다.

ex) Counter = decay(Counter) = Counter/2

2) 예시

Time	Process A		Process B		Process C	
	Priority	CPU Count	Priority	CPU Count	Priority	CPU Count
0	60	0	60	0	60	0
		1				
		2				
		-				
		60				
1	75	30	60	0	60	0
				1		
				2		
				-		
				60		
2	67	15	75	30	60	0
						1
						2
						-
						60
3	63	7	67	15	75	30
		8				
		9				
		-				
		67				
4	76	33	63	7	67	15
				8		
				9		
				-		
				67		
5	68	16	76	33	63	7

- 파란색 배경일 때 프로세스가 CPU를 사용합니다.

- CPU Count = Counter , Priority = Nice

- 60Hz CPU 이므로 , Counter는 1초에 60씩 증가합니다.

- Process_A는 0초일 때 Counter는 0이고 , Nice는 $0/2 + 60 = 60$

- Process_A는 1초일 때 Counter는 60이 된 후 decay 함수로 인해 30으로 반감하고 , Nice는 $30/2 + 60 = 75$ 가 됩니다.

- Process_A는 2초일 때 Counter는 decay 함수로 인해 30에서 15로 반감하고, Nice는 $15/2 + 60 = 67$ 이 됩니다.

- 반드시 공식에 기반 하여 **최근 Counter를 구한 후 Nice를 계산 합니다.**

- 프로세스는 CPU를 사용하지 않아도 Counter와 Nice값을 갱신합니다.

<결과>

- CPU를 사용 않으면 Nice값은 계속 낮아져 0에 수렴하므로 , 우선순위가 커지며 , CPU를 사용할 가능성이 점점 증가합니다.

- I/O는 CPU를 사용하지 않으므로 I/O를 많이 하는 프로세스는 CPU를 사용할 확률이 높아집니다. 반면에 CPU를 많이 쓰는 프로세스는 계속 Nice 값이 증가하므로 CPU를 사용할 확률이 낮아집니다. 스케줄링 설계 방식으로 인해 이는 매우 자명합니다.

3) 특징

- Round Robin과 유사하지만 별도의 Queue를 만들지 않고 연산만으로 스케줄링이 가능합니다.
- 리눅스 스케줄링의 기반이 됩니다.

<과제 3번 공부> 9/30 ~ 10/14

= xv6 운영체제에서 스케줄링 해보기

1. 참고 자료

1) 설치

```
$ sudo apt-get update && sudo apt-get install git nasm build-essential qemu gdb
```

<https://oslab.kaist.ac.kr/xv6-tools/>

= 해당 명령어를 리눅스 셸에서 실행시키고 , 사이트에 있는 명령어를 실행 시켰습니다.

2) 사용자 지정 System Call 등록하기

<https://medium.com/@viduniwickramarachchi/add-a-new-system-call-in-xv6-5486c2437573>

<https://www.youtube.com/watch?v=21SVYiKhcwM>

- 2번째 링크인 유튜브의 내용을 따라하면 되지만 개념은 1번째 링크에 적혀있습니다.

3) Schedule

<https://www.youtube.com/watch?v=DZ0-GMtOtEc>

2. 사용자 지정 System Call

- 나의 펠기 6장의 '2) S/W Interrupt 중 System Call을 처리하는 과정'의 그림을 자세하게 구현한 방법입니다. 그림은 이미 만들어진 시스템 콜을 호출하는 방식이고, 과제는 사용자 지정 시스템 콜을 추가하는 것이니 순서는 다릅니다. 뇌피셜 : 하나는 정방향, 다른 하나는 역방향으로 생 각합시다.
- 확실하지 않지만 커널의 시스템 콜(sys_cps())과 유저의 시스템 콜(cps())과 유저의 시스템 콜을 부르는 명령어(ps())를 잘 구분해야 합니다. 왜 3 단계를 거치는지 모르겠고, 2단계로 할 수 있을 것 같음.
- int main() { '내용' }에서 int main() 부분을 함수의 이름이라 하고 '내용' 부분을 함수의 바디라고 하겠습니다.

<커널 시스템 콜 만들기>

= 커널 시스템 콜은 앞에 sys_가 붙습니다. 이들의 정확한 동작 방식은 모르지만 유저가 만든 유저 시스템 콜을 참조하여 커널에서 실행 시키는 것 같습니다.

1) syscall.h

```
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_sleep 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_cps 22
24 #define SYS_hello 23
```

- syscall.h에 커널 시스템 콜의 번호를 #define으로 부여합니다.
- 최초의 커널 시스템 콜은 21개가 존재합니다.

2) syscall.c

```
109 static int (*syscalls[])(void) = {
110     [SYS_fork]    sys_fork,
111     [SYS_exit]    sys_exit,
112     [SYS_wait]    sys_wait,
113     [SYS_pipe]    sys_pipe,
114     [SYS_read]    sys_read,
115     [SYS_kill]    sys_kill,
116     [SYS_exec]    sys_exec,
117     [SYS_fstat]   sys_fstat,
118     [SYS_chdir]   sys_chdir,
119     [SYS_dup]     sys_dup,
120     [SYS_getpid]  sys_getpid,
121     [SYS_sbrk]    sys_sbrk,
122     [SYS_sleep]   sys_sleep,
123     [SYS_uptime]  sys_uptime,
124     [SYS_open]    sys_open,
125     [SYS_write]   sys_write,
126     [SYS_mknod]   sys_mknod,
127     [SYS_unlink]  sys_unlink,
128     [SYS_link]    sys_link,
129     [SYS_mkdir]   sys_mkdir,
130     [SYS_close]   sys_close,
131     [SYS_cps]     sys_cps,
132 };
133
```

```
85 extern int sys_chdir(void);
86 extern int sys_close(void);
87 extern int sys_dup(void);
88 extern int sys_exec(void);
89 extern int sys_exit(void);
90 extern int sys_fork(void);
91 extern int sys_fstat(void);
92 extern int sys_getpid(void);
93 extern int sys_kill(void);
94 extern int sys_link(void);
95 extern int sys_mkdir(void);
96 extern int sys_mknod(void);
97 extern int sys_open(void);
98 extern int sys_pipe(void);
99 extern int sys_read(void);
100 extern int sys_sbrk(void);
101 extern int sys_sleep(void);
102 extern int sys_unlink(void);
103 extern int sys_wait(void);
104 extern int sys_write(void);
105 extern int sys_uptime(void);
106 extern int sys_cps(void);
107 extern int sys_hello(void);
108
```

- syscall.c에 syscall.h에 선언한 상수를 연결 시켜줍니다.

그로 인해 다른 파일에 존재하는 상수 값을 syscall.c에서 참조할 수 있게 됩니다.

- 이제 sys_call_table에 22번으로 커널 시스템 콜이 추가 됩니다.

- syscall.c에 함수의 이름과 바디를 모두 정의하고 싶지만 이름만을 갖고 있는 원형만 선언해줍니다.

3) sysproc.c

```
93 int
94 sys_cps(void)
95 {
96     return cps();
97 }
98
99 int
100 sys_hello(void)
101 {
102     return hello();
103 }
sysproc.c [+]
```

- 커널 시스템 콜을 만들기는 했지만 아직 이름만 존재하는 껍데기입니다.

- 비어 있는 커널 시스템 콜을 채우기 위해 함수 바디를 sysproc.c에 선언해줍니다.

- 기존에 만들어진 커널 시스템 콜도 유저 시스템 콜을 그냥 return하는 것이 많습니다. ex) sys_fork() , sys_exec() ...

- Body에 내용을 넣은 것 또한 잘 나옵니다.

<유저 시스템 콜 만들고 커널 시스템 콜과 연결>

1) usys.S

```
1 #include "syscall.h"
2 #include "traps.h"
3
4 #define SYSCALL(name) \
5     .globl name; \
6     name: \
7     movl $SYS_ ## name, %eax; \
8     int $T_SYSCALL; \
9     ret
10
11 SYSCALL(fork)
12 SYSCALL(exit)
13 SYSCALL(wait)
14 SYSCALL(pipe)
15 SYSCALL(read)
16 SYSCALL(write)
17 SYSCALL(close)
18 SYSCALL(kill)
19 SYSCALL(exec)
20 SYSCALL(open)
21 SYSCALL(mknod)
22 SYSCALL(unlink)
23 SYSCALL(fstat)
24 SYSCALL(link)
25 SYSCALL(mkdir)
26 SYSCALL(chdir)
27 SYSCALL(dup)
28 SYSCALL(getpid)
29 SYSCALL(sbrk)
30 SYSCALL(sleep)
31 SYSCALL(uptime)
32 SYSCALL(cps)
33 SYSCALL(hello)
```

- 유저 시스템 콜 테이블에 제가 만든 유저 시스템 콜을 정의해줍니다.

- 유저 시스템 콜에서 번호를 만드는 어셈블리 코드가 존재합니다. 이 번호로 커널 시스템 콜의 syscall.h와 연결 하는 것 같습니다.(뇌피셜)

2) user.h

```
4 // system calls
5 int fork(void);
6 int exit(void) __attribute__((noreturn));
7 int wait(void);
8 int pipe(int*);
9 int write(int, const void*, int);
10 int read(int, void*, int);
11 int close(int);
12 int kill(int);
13 int exec(char*, char**);
14 int open(const char*, int);
15 int mknod(const char*, short, short);
16 int unlink(const char*);
17 int fstat(int fd, struct stat*);
18 int link(const char*, const char*);
19 int mkdir(const char*);
20 int chdir(const char*);
21 int dup(int);
22 int getpid(void);
23 char* sbrk(int);
24 int sleep(int);
25 int uptime(void);
26 int cps(void);
27 int hello(void);
28
```

- This would be the function which the user program calls. This function will be mapped to the system call with the number 22 which is defined as SYS_getyear preprocessor directive. (원문)

= 유저 시스템 콜의 이름을 정의하고 , 유저 시스템 콜과 사용자 시스템 콜을 번호를 가지고 Mapping 하여 연결 합니다.

3. defs.h

```
104 //PAGEBREAK: 16
105 // proc.c
106 int      cpuid(void);
107 void      exit(void);
108 int      fork(void);
109 int      growproc(int);
110 int      kill(int);
111 struct cpu* mycpu(void);
112 struct proc* myproc();
113 void      pinit(void);
114 void      procdump(void);
115 void      scheduler(void) __attribute__((noreturn));
116 void      sched(void);
117 void      setproc(struct proc*);
118 void      sleep(void*, struct spinlock*);
119 void      userinit(void);
120 int      wait(void);
121 void      wakeup(void*);
122 void      yield(void);
123 int      cps(void);
124 int      hello(void);
```

- 유저 시스템 콜의 이름 부분을 선언합니다.

4. proc.c

```
536 int cps()
537 {
538     cprintf("name");
539     return 22;
540 }
541
542 int hello()
543 {
544     cprintf("helloxv6");
545     ~
546 }
```

- 유저 시스템 콜의 바디 부분을 선언합니다.

- 이 부분의 코드가 시스템 콜이 콘솔에 보여주는 역할입니다.

- return값이 반드시 시스템 콜이어야 할까?

<유저 시스템 콜을 호출하는 프로그램 만들기>

- 이제 시스템 콜은 모두 만들었습니다. 하지만 어떠한 이유인지는 모르겠으나 유저 시스템 콜을 바로 사용하지 않고 유저 시스템 콜을 실행하기 위한 .c파일을 하나 만들어 줍니다.

```
1 #include "types.h"
2 #include "stat.h"
3 #include "user.h"
4 #include "fcntl.h"
5
6 int main()
7 {
8     hello();
9     exit();
10 }
11
12
13
14
15
16
17
18 UPROGS=\
19     _cat\
20     _echo\
21     _forktest\
22     _grep\
23     _init\
24     _kill\
25     _ln\
26     _ls\
27     _mkdir\
28     _rm\
29     _sh\
30     _stressfs\
31     _usertests\
32     _wc\
33     _ps\
34     _hello_test\
35     _zombie\
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52 EXTRA=\
53     mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
54     ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c ps.c hello_test.c zombie.c\
55     printf.c umalloc.c\
56     README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
57     .gdbinit.tmpl gdbutil\
58
```

- 그림 1은 hello_test.c 프로그램이며 유저 시스템 콜을 호출하는 역할을 합니다.
- Makefile에 해당 프로그램을 등록하여 유저 시스템 콜을 호출할 때 해당 프로그램의 이름을 사용하도록 합니다.
- hello()를 예로 들면 hello()는 .c파일에서 돌아가는 함수입니다. 그러므로 이 함수는 셸 환경에서는 적어봐야 에러만 납니다. 그러므로 셸 환경에서 hello()를 돌리기 위해 hello_test.c 프로그램을 만들고 해당 프로그램을 Makefile에 등록합니다.
- Makefile에 등록하면 xv6 셸 내에서 해당 명령어를 테스트를 해볼 수 있습니다.

4. 기타 공부 내용

- Ctrl + Alt를 누르면 qemu의 마우스 고정이 풀립니다.
- xv6는 MIT에서 만든 교육용 운영체제입니다.
- xv6는 멀티 코어 CPU를 지원합니다.
- 구조체 포인터를 이용하여 proc.h에 있는 cpu와 proc에 접근하여 그들의 객체를 만듭니다.
- ptable = 64개(NPROC)의 프로세스를 저장한 테이블
- proc.c = 프로세스 관련 함수를 모두 모았습니다.

xv6 kernel은 모든 프로세스를 proc 구조체를 통해 관리한다. proc 구조체는 ptable이라는 proc구조체의 배열로 관리되며 xv6에서 최대 생성될 수 있는 프로세스는 64개이다. userinit() 함수는 제일 먼저 allocproc()함수를 호출하여 할당할 수 있는 proc 구조체가 있는지 확인하고 있으면 할당받는다. proc 구조체를 할당받았으면 pid와 kernel stack을 할당받는다. 이후 할당받은 kernel stack을 아래 그림과 같이 구성한다.

<우선순위>

proc.h에 priority 변수 설정

proc.c에 acquire에 priority 값을 설정

<allocproc>

ptable에서 UNUSED 상태의 proc을 찾아서 개를 EMBRYO(태아) 상태로 만들어 주고 해당 프로세스의 pid를 최근의 pid + 1로 바꿉니다.

<유저 시스템 콜 hello()에서 다른 유저 시스템 콜인 set_prio(int n)을 호출>

1) user.h

```
32 int set_prio(int);
```

2) defs.h

```
129 int set_prio(int);
```

3) proc.c

```
720 //set priority of process
721 int set_prio(int n)
722 {
723     cprintf("set_prio!!!\n");
724     struct proc *p;
725
726     //choose pid -> then that pid's process priority can change using this function.
727     for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
728     {
729         //success
730         if(p->pid == 1)
731         {
732             p->priority = n;
733             return 28;
734         }
735     }
736
737     //fail
738     return -1;
739 }
```

4) sysproc.c

```
131 int
132 sys_set_prio(void)
133 {
134     int pid;
135
136     if(argint(0, &pid) < 0)
137         return -1;
138     return set_prio(pid);
139 }
```

- 4단계를 제외 하고는 기존과 동일하게 시스템 콜을 만듭니다.

<테스트 프로그램을 만들어서 셸에서 Parameter가 있는 시스템 콜을 호출하기>

```
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+1FF8DDDD+1FECDDDD C980

Booting from Hard Disk...
cpu1: starting 1
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ helloname_test xv6
hello xv6
$ _
```

= xv6 셸에서 Parameter가 있는 시스템 콜에 인자(xv6)를 넣어서 호출합니다.

1) user.h

```
28 int helloname(char*);
```

2) defs.h

```
125 int helloname(char*);
```

3) proc.c

```
int helloname(char* str)
{
    cprintf("hello %s\n", str);
    return 24;
}
```

4) sysproc.c

```

105 int
106 sys_helloname(void)
107 {
108     char* str;
109     if(argstr(0, &str) < 0)
110     {
111         return -1;
112     }
113     return helloname(str);
114 }
115

```

5) helloname_test.c(유저 시스템 콜을 테스트 하는 프로그램)

```

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)
1 #include "types.h"
2 #include "stat.h"
3 #include "user.h"
4 #include "fcntl.h"
5
6 int main(int argc , char *argv[])
7 {
8     helloname(argv[1]);
9     exit();
10 }
~

```

- 5단계를 제외 하고는 기존과 동일하게 시스템 콜을 만듭니다.

<10/10>

1. 교수님이 주신 테스트를 RR 짜지 않고 돌렸을 때 exit 순서가 이상하게 나옴.
2. 원래는 유튜브에서 만들어진 fork()로 테스트 해보려했는데 4번을 할 때 주신 테스트 프로그램이 fork()였습니다.
3. 2개의 함수의 인자는 현재 부모프로세스(seqinc_prio)에서 만들 자식 프로세스의 개수
4. 지금 까지의 테스트에서는 테스트 코드를 잘못짜서 lock을 걸지 않아도 priority가 변경되었음. 하지만 lock을 걸어야 set_prio가 정상적으로 작동함.

```

PID PPID SIZE      Number of Context Switch
1    0   4096      22      1001
2    1  12288     25      1001
3    2  49152     14      1001
4    2  16384     32      1001
5    4  12288     1400     1001
6    4  12288     1434      11
7    4  53256     1431      21
8    4  53256     1419      31
9    4  53256     1428      41
10   2  49152     15      1001
$ _

```

4번이 1000이고 5 ~9가 11 , 21 , 31 , 41 , 51 이어야 하는데 출력 프로그램(getprocinfo_test)이 틀

린것같다.

**** 출력 프로그램이 틀렸다. 출력 프로그램이 틀린거므로 기존 프로그램은 오류가 없다.**

```

$ ps
name  pid  priority  state
init  1    1001    SLEEPING
sh    2    1001    SLEEPING
ps    3    1001    RUNNING
$ seqinc_prio 3
iget 1000
Priority of parent priority 1000 /// pid  4
process = 1000
priority 1000 /// pid  4
priority 1000 /// pid  4
priority 1000 /// pid  4
priority 1000 /// pid  4
priority 1000 /// pid  4
priority 1 /// pid  5
priority 1 /// pid  5
priority 1 /// pid  5
priority 1000 /// pid  4
priority 1 /// pid  5
priority 1000 /// pid  4
priority 11 /// pid  6
priority 11 /// pid  6
priority 1 /// pid  5
priority 11 /// pid  6
priority 21 /// pid  7
priority 1 /// pid  5
priority 21 /// pid  7
priority 11 /// pid  6
priority 21 /// pid  7
priority 11 /// pid  6
priority 21 /// pid  7
priority 1 /// pid  5
priority 21 /// pid  7
priority 1 /// pid  5
priority 11 /// pid  6

```

4번이 부모데 1000으로 잘 바뀌고 , 5번이 1 , 6번이 11 , 7번이 21로 잘 바뀐다.

<10/12>

- 처음에 alloc에서 할당해준 priority를 짧은 순간이지만 읽는다. 그리고 그걸 schedule 해버려서 priority에 적용함.
- struct proc *p; 애는 프로세스 테이블 안에 있는 프로세스를 가리키는 포인터 변수인데 , 뭘 가리키는지 반드시 선언해 줘야함.
- for(p = ptable.proc; p< &ptable.proc[NPROC]; p++)에서 p가 가리키는 것을 정해주므로 p->priority는 이상이 없지만 이런 거 없이 p->priority 하는 것은 아무것도 가리키지 않는 포인터 변수에게 포인터 변수가 가리키고 있는 프로세스의 priority 값을 원하는 것이다.
- 스케줄 함수는 ptable의 0 ~ 64까지의 process를 순차적으로 탐색하는데 이걸 무한번 한다.
- ex) ptable의 6번 proc을 검사하고 , 그 다음에 7번 proc을 검사하고 ... 64번을 검사하고 다시 0번부터 검사함.
- 그러므로 이전에 스케줄링은 0 -> 1 -> 2를 하고 3~64만 반복하도록 만들었으므로 당연히 에러 남.
- Pid_1과 Pid_2는 INIT과 Sh이므로 반드시 부팅과 쉘 입력에 필요한 프로세스이므로 애네 실행 안시키면 qemu 안켜지는 거야.

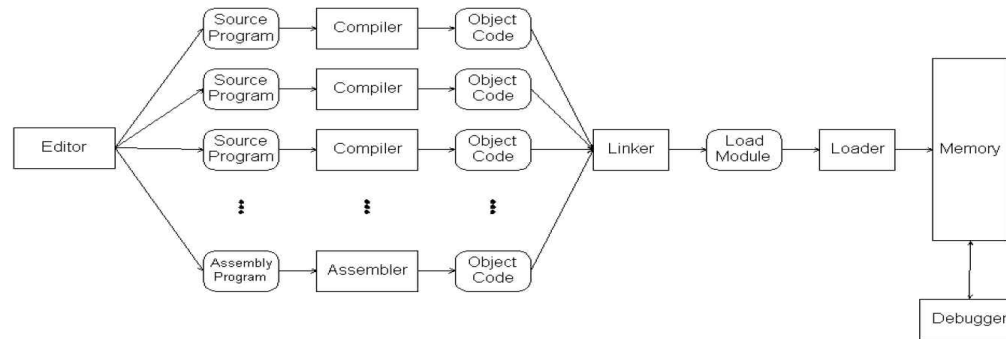
<10/13>

일반적으로 대부분의 OS가 기본 우선 순위와 관련된 변수가 PCB내 멤버로 존재하고..이를 외부/내부에서 설정하게 되고 그 우선 순위는 실행되면서 변하게 됨. XV6에서는 우선순위 설정함수가 allocproc()이고

<13강_Address Spaces & Memory Management> 10/10 ~ 10/13

1. 배경지식과 용어

1) Source Program & Object Program & EXE Program



Object Code of Fig 2.5 (Fig 2.6 - I)

5	0000	COPY	START	0
10	0000	FIRST	STL	RETADR
12	0003		LDB	#LENGTH
13			BASE	LENGTH
15	0006	CLOOP	+JSUB	RDREC
20	000A		LDA	LENGTH
25	000D		COMP	#0
30	0010		JEQ	ENDFIL
35	0013		+JSUB	WRREC
40	0017		J	CLOOP
45	001A	ENDFIL	LDA	EOF
50	001D		STA	BUFFER
55	0020		LDA	#3
60	0023		STA	LENGTH
65	0026		+JSUB	WRREC
70	002A		J	@RETADR
80	002D	EOF	BYTE	C'EOF'

Object Program (Fig 2.8)

```

HCOPY 000000001077
T0000001D17202D69202D4B1010360320262900003320074B10105D3F2FEC032010
T00001D130F20160100030F200D4B10105D3E2003454F46
T0010361DB410B400B44075101000E32019332FFADB2013A00433200857C003B850
T0010531D3B2FEA1340004F0000F1B410774000E32011332FFA53C003DF2008B850
T001070073B2FEF4F000005
M00000705
0F2016
M00001405
0F200D
M00002705
E000000

```

① gcc -c 'Source Program'

= Source Program(.c)을 컴파일+어셈블 해서 Object Code(ex : 17202D)들로 이루어진 Object Program(.o)을 생성합니다.

- 결과로 기계어로 구성된 오브젝트 프로그램인 test.o가 생성 됩니다.
- .c파일과 .o 모두 디스크에 저장 됩니다.

```
User@DESKTOP-BAGIJS2 ~  
$ gcc -c test.c  
  
User@DESKTOP-BAGIJS2 ~  
$ ls  
'Cygwin Source 플 더 의 역 할 .txt'  ssu_shell.c  test.c  
desktop.ini  ssu_shell.zip  test.o
```

② gcc -o 'Execute Program' 'Source Program'

= Source Program을 원하는 이름의 실행 파일로 만듭니다. gcc 명령으로 실행 파일을 만드는 것이고 , -o는 원하는 이름으로 만드는 것 뿐.

- 결과로 실행 프로그램인 execute_test.exe가 생성 됩니다.
- 이름을 지정 하지 않으면 Default로 a.exe가 생성 됩니다.
- 윈도우 환경에서는 .exe가 실행파일 이고 리눅스 환경에서는 .out이 실행파일입니다.
- .exe 파일은 실행되기 전에 Passive 상태로 디스크에 저장되어 있습니다. 이 파일이 실행(/execute_test OR 마우스 더블클릭)되면 Active 상태로 메모리에 올라갑니다.

```
User@DESKTOP-BAGIJS2 ~  
$ gcc -o execute_test test.c  
  
User@DESKTOP-BAGIJS2 ~  
$ ls  
'Cygwin Source 플 더 의 역 할 .txt'  execute_test.exe  ssu_shell.zip  test.o  
desktop.ini  ssu_shell.c  test.c
```

<Example : 내가 만든 'seeprocess.exe'를 실행시켜서 메모리에 올리기>

```
User@DESKTOP-BAGIJS2 ~/BOJ/source
$ g++ -o seeprocess boj_14890.cpp

User@DESKTOP-BAGIJS2 ~/BOJ/source
$ ls
boj_1051.cpp  boj_13458.cpp  boj_14500.cpp  boj_14502.cpp  boj_14888.cpp  boj_14890.cpp  boj_14891.cpp  boj_15651.cpp  boj_15683.cpp  boj_17413.cpp  boj_18111.cpp  boj_3048.cpp  boj_5212.cpp  seeprocess.exe
boj_13305.cpp  boj_14499.cpp  boj_14501.cpp  boj_14503.cpp  boj_14889.cpp  boj_14890_before.cpp  boj_15650.cpp  boj_15652.cpp  boj_1676.cpp  boj_1748.cpp  boj_3024.cpp  boj_3190.cpp  boj_8911.cpp

User@DESKTOP-BAGIJS2 ~/BOJ/source
$ ./seeprocess.exe
```

작업 관리자

파일(F) 옵션(O) 보기(V)

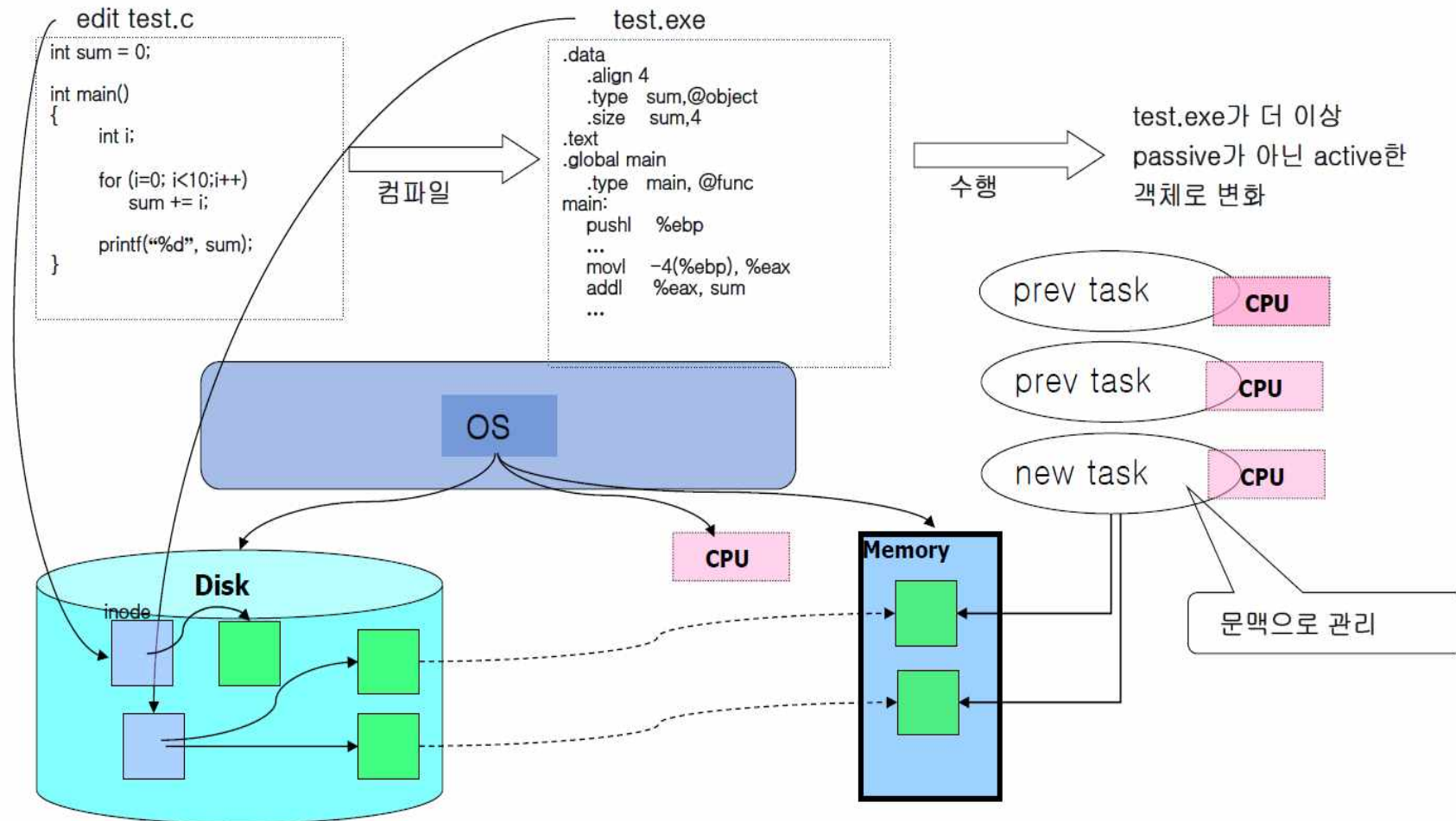
프로세스 성능 앱 기록 시작프로그램 사용자 세부 정보 서비스

이름	상태	PID	6% CPU	29% 메모리	0% 디스크	네트
> Runtime Broker			0%	2.1MB	0MB/s	
Runtime Broker		9028	0%	2.0MB	0MB/s	
seeprocess		9208	0%	0.8MB	0MB/s	
Sink to receive asynchronous c...		2080	0%	0.8MB	0MB/s	
> Spooler SubSystem App		3300	0%	5.1MB	0MB/s	
> System Guard 런타임 모니터 ...		12204	0%	2.4MB	0MB/s	
User OOB Broker		7188	0%	0.8MB	0MB/s	
Usermode Font Driver Host		1004	0%	9.8MB	0MB/s	
Windows Defender SmartScreen		1540	0%	5.4MB	0MB/s	
> Windows Security Health Service		12864	0%	1.9MB	0MB/s	
> Windows 셸 환경 호스트			0%	0MB	0MB/s	
Windows 오디오 장치 그래프 ...		1644	0.7%	6.0MB	0MB/s	
Windows 작업을 위한 호스트 ...		3872	0%	2.9MB	0MB/s	
Windows 호스트 프로세스(Run...		6052	0%	0.6MB	0MB/s	

간단히(D) 작업 끝내기(E)

- ls명령어로 확인해보면 seeprocess.exe를 실행시키기 전에는 디스크에만 저장되어 있고 메모리에는 올라가지 않았습니다.
- ./seeprocess.exe로 프로그램을 실행해야 비로소 메모리에 올라가(로딩) PID_9208번의 프로세스가 되어 실행됩니다.

2) 디스크에서 메모리로.



- Disk 그림에서 초록색 사각형은 Disk_Block을 의미합니다. 리눅스에서 1개의 블록은 4KB를 Default_Size로 합니다.
- Memory 그림에서 초록색 사각형은 Page_Frame을 의미합니다. Disk_Block이 4KB라면 Page_Frame도 4KB입니다.
- Memory 그림에는 사실 Page_Table 도 존재합니다.
- test.c는 매우 작은 크기이므로 1개의 블록으로 충분합니다. (최대 4KB)
- test.exe는 test.c보다 크기가 크므로 2개의 블록을 사용합니다. (최대 8KB)
- 두 프로그램은 모두 디스크에 존재하고 test.exe는 실행하면 메모리로 올라갑니다.
- 디스크에 존재하는 모든 파일(test.c , test.exe , 음악.mp3)은 inode_struct를 갖고 있습니다. inode_struct는 커널에서의 개념이고 유저 개념에서는 stat입니다.
- OS는 초기에 수행될 코드와 참조될 데이터가 포함되도록 단지 하나 혹은 몇 개의 블록만 반입하는데 프로세스의 코드나 데이터들 중에서 임의 시점에 메모리에 load 시키는 부분을 Resident Set이라고 합니다.

2. Address Space Overview

1) 정의 및 특성

= 메모리의 주소 공간입니다.

- 과거에는 메모리가 작았기 때문에 실제 주소 공간을 사용했으나 , 현대에는 메모리의 크기가 커져서 가상 주소 공간을 사용하게 되었습니다.
- 주소 공간에는 연속적으로 데이터를 저장하지 않아도 됩니다.

2) 목표

① Transparency(투명성)

= 메모리의 내부구조를 알지 못하여도 사용자가 메모리를 사용하는데 큰 어려움이 없게 합니다.

② Efficiency(효율성)

= 느리지 않아야 하고 , 공간도 효율적으로 사용해야 합니다.

③ Protection(보호)

= 메모리의 커널 영역을 보호해야 합니다.

3) 관리

① Placement(배치)

= 프로세스를 메모리의 어느 주소공간에 저장할 지를 관리합니다.

② Protection(보호)

= 프로세스 사이의 경계를 어떻게 할지 관리합니다.

- 가상화를 하지 않았던 시절은 프로세스와 프로세스 간의 경계에 해당 하는 값들을 레지스터에 저장했습니다.

③ Replacement(재배치)

= 메모리가 많은 프로세스를 올려놓아서 더 이상 다른 프로세스를 넣을 공간이 없을 때 , 기존의 프로세스를 빼고 새로운 프로세스를 넣습니다.

④ On-Demand Loading(요구 시 반입)

= User가 프로세스를 실행하면 메모리에 올립니다.

- Loading 대신 Fetch를 사용하여 Demand Fetch라고도 합니다.

<Pre-Fetch>

= User가 자주 사용하는 프로그램을 OS가 인지하여 User가 실행하지 않아도 메모리에 스스로 올리는 방법입니다.

- 불가능한 방법이지만 현대 OS는 기능의 극히 일부분은 사용하고 있습니다.

ex) 아침 8시 마다 유저가 잠을 실행한다면 아침 8시에 OS가 스스로 잠을 실행시킵니다.

3. Virtual Memory Address & Physical Memory Address

1) 정의

① Virtual Memory Address

= 프로세스가 참조(사용)하고, 사용자가 보는 주소

= Virtual Address

= 가상 메모리 주소, 가상 주소, 논리 주소라고 부릅니다. 저는 앞으로 **가상 주소**라고 필기하겠습니다.

- 가상 주소가 같더라도 실제 주소는 반드시 다릅니다.

- 가상 주소를 따라가서 실제 주소를 찾아가 보면 Swap영역에 있을 수도 있습니다.

- Code(=Text) + Data + Stack + (Heap)으로 이루어져 있습니다.

- 가상 주소를 고정된 크기로 나누는 단위는 **Page**입니다.

- 레지스터에는 주소만 저장되지 않고 필드 값도 들어갑니다.

- 가상 주소는 레지스터의 크기에 비례하여 주소 공간이 생성됩니다.

- 가상 메모리의 크기는 실제 메모리의 크기보다 큽니다. 과장해서 말하면 가상 메모리는 무제한의 크기를 갖는 환상을 보여줍니다. 하지만 실제 메모리 보다 큰 가상 메모리를 사용하게 되면 실제 메모리와 디스크 사이에서 끊임없이 페이지를 교체하는 Thrashing이 발생하여 매우 느려집니다.

ex) 32비트 레지스터 -> 2^{32} 개의 주소 공간.

② Physical Memory Address

= 실제 메모리(RAM)의 주소

= Physical Address

= 실제 메모리 주소, 실제 주소, 물리 주소, Main Memory라고 부릅니다. 저는 앞으로 **실제 주소**라고 필기하겠습니다.

- 실제 주소를 고정된 크기로 나누는 단위는 **Page Frame**입니다.

The differences between page and frame:

A **page** in a paging system is a **virtual memory block with a fixed length**.

But a **frame** in a paging system is a **main memory block with fixed length**.

A **page** has **virtual address** and it is transferred as a **unit between main memory and secondary memory**.

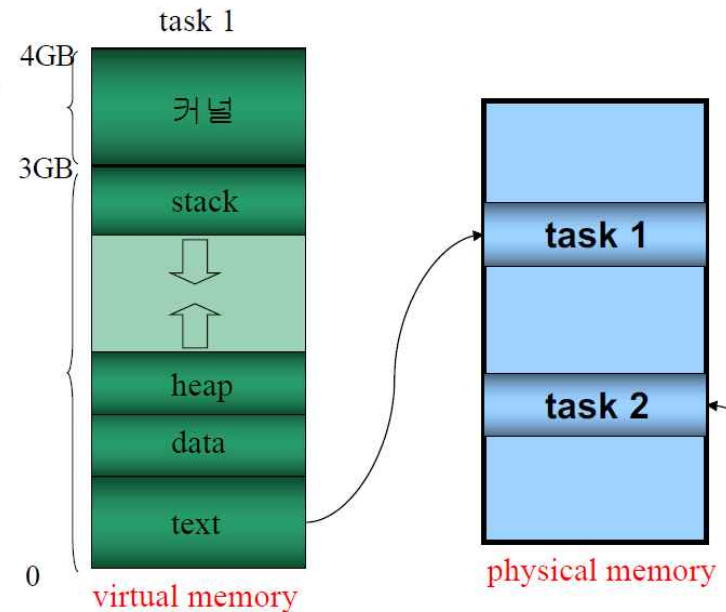
But a **frame** can only **hold one page of virtual memory**.

Programs and data are **stored on disk which is divided into fixed sized blocks**. They are called **pages**. These **blocks** are used as **virtual memory**.

Main memory on the other hand is divided into **fixed sized blocks** such that the size is equal to the size of a page.

A **page address** is a **virtual address** that refers to the **address on secondary memory**.

But a **frame address** is an **address of physical location on main memory**.



- Virtual Memory(가상 메모리)와 Virtual Memory Address(가상 주소)는 관련이 매우 깊지만 조금 다른 개념 입니다.
- Physical Memory(실제 메모리)와 Physical Memory Address(실제 주소)는 관련이 매우 깊지만 조금 다른 개념입니다.
- 메모리 공간을 Byte 단위로 쪼개서 각각의 공간마다 주소를 붙인 것이 주소입니다. 두 개념을 크게 구분하지 않아도 괜찮습니다.
- 가상 메모리는 Page들로 이루어져 있고 , 실제 메모리는 Page-Frame들로 이루어져 있습니다.

4. Physical Memory 분할 기법

= 가상 메모리와 가상 주소를 사용하지 않고 오직 실제 메모리와 실제 주소만을 사용하는 구시대의 기법.

1) Single Programming System

= 오직 1개의 프로세스만이 메모리에 올라가 있을 수 있습니다.

ex) MS-DOS

<주어진 메모리(100kb)보다 더 큰 프로세스(120KB)가 들어온다면?>

① 더 큰 메모리를 구매합니다.

② 중첩 Overlay를 사용합니다.

= 중요한 데이터 영역(여기서는 20KB)있다면 50+20 / 50+20으로 쪼개서 메모리에 올립니다.

2) Multiprogramming and Time Sharing

= 다수의 프로세스를 메모리에 올립니다. 그 과정에서 기존 프로세스 상태를 디스크에 저장하고 , 디스크에서 필요하면 다시 메모리로 올리는 구시대의 시분할 기법을 사용합니다.

- 디스크를 거치므로 매우 느립니다.

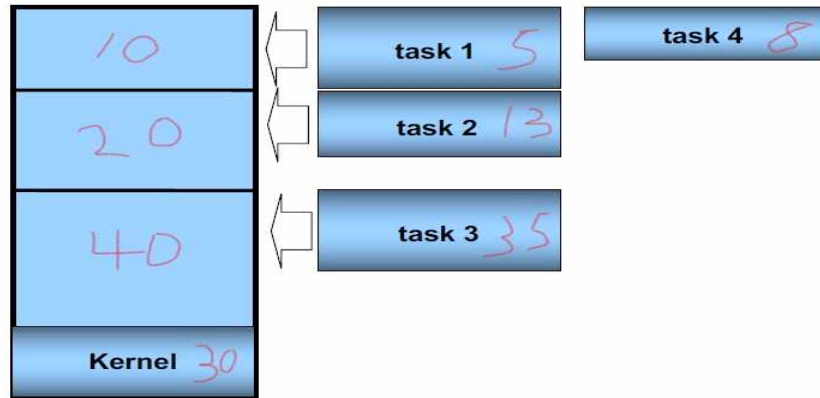
- 현대에는 디스크를 사용하지 않고 메모리의 proc.c를 이용합니다.

*** 아래 그림에서 10KB , 20KB , 40KB으로 메모리를 분할한 공간을 '공간'이라고 하고 , 10KB에 task1을 넣어서 남는 공간 5KB를 'Hole'이라고 합니다. 공간과 Hole을 잘 구분해야 합니다.

① 고정 분할

<절대변역>

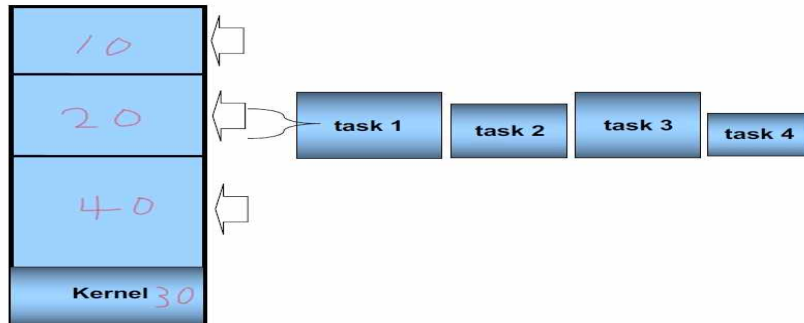
= 메모리를 고정 크기로 분할합니다. 아래의 그림에서 100KB를 10 , 20 , 40 , 30(Kernel)로 분할했습니다. 모든 공간마다 Queue를 둡니다.



- 1번 공간에는 10KB보다 작거나 같은 프로세스를 올립니다.
- 2번 공간에는 10KB보다 크고 20KB보다 작거나 같은 프로세스를 올립니다.
- 3번 공간에는 20KB보다 크고 40KB보다 작거나 같은 프로세스를 올립니다.
- 모든 task가 10KB 보다 작다면 2번 큐와 3번 큐가 채워지지 않아 2번 공간과 3번 공간이 사용되지 않고 낭비가 되므로 이를 해결하기 위해 재배치 기법을 도입하게 됩니다.
- 1번 공간에 task1을 올리면 5KB만 남기 때문에 5KB는 사용할 수 없는 공간(hole)이 되므로 낭비가 생깁니다.
- 이런 현상을 **Internal Fragmentation(내부 단편화)**라고 부릅니다.

<재배치>

= 메모리를 고정 크기로 분할합니다. 아래의 그림에서 100KB를 10 , 20 , 40 , 30(Kernel)로 분할했습니다. Queue를 1개만 둡니다.



- 절대 번역은 메모리의 분할된 공간마다 알맞은 크기의 프로세스를 올렸다면 , 재배치는 비어 있는 공간에 크기와 상관없이 올립니다.
- task1 ~ task4는 자신이 들어 갈 수 있는 공간이 존재한다면 들어가서 메모리에 올라갑니다.
- 재배치 또한 내부 단편화가 생깁니다.

② 가변분할

= 고정 분할 방법에서 발생한 Internal Fragmentation을 해결하기 위해 가변 분할이 도입되었습니다.

- task1 = 8KB , task2 = 15KB라고 가정할 때 , task1을 메모리에 올리기 위해 메모리에 해당 task와 딱 맞는 공간을 만들어 줍니다. task1을 위해서는 8KB의 공간을 만들고 task2를 위해서는 15KB의 공간을 만들어줍니다.
- 하지만 가변분할도 **External Fragmentation(외부 단편화)**이라는 문제가 발생합니다.

<가변분할 Example>

- First Fit = 메모리의 제일 위에 있는 공간에 프로세스를 넣습니다.
- Best Fit = 메모리에서 가장 알맞은 공간에 프로세스를 넣습니다.
- Worst Fit = 메모리에서 가장 큰 공간에 프로세스를 넣습니다.

메모리에서 정의에 맞는 공간을 포인터를 이용하여 찾아야하므로 결국 성능은 3개가 비슷합니다.

- Next fit = 메모리의 어떤 공간을 포인터가 가리키고 있으면 다음에 있는 공간에 들어갈 수 있으면 프로세스를 넣습니다.
- Next fit은 포인터 이동이 없는 Best Fit입니다.

<Internal Fragmentation>

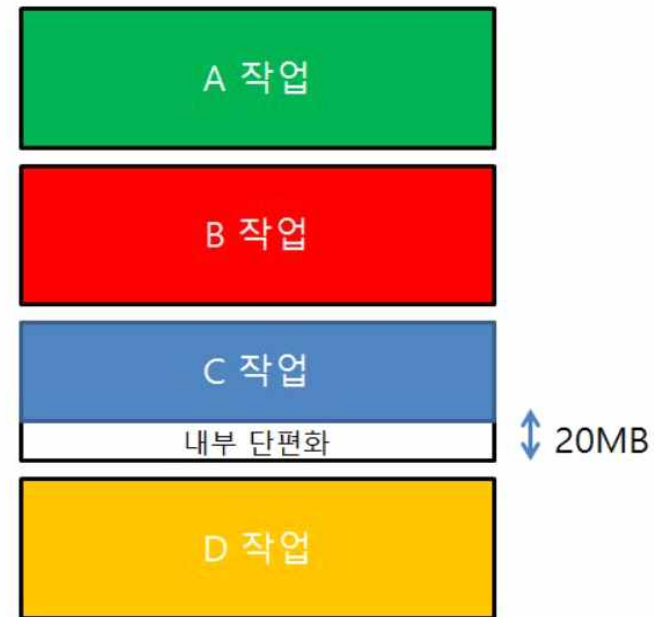
= 내부 단편화

= 빈 공간(50MB)에 C 작업(30MB)을 넣으면 20MB의 Hole이 생깁니다.



내부 단편화

여기 메모리를 보시게 되면 빈공간 50mb의 공간이 있습니다. 그렇게 되면 C작업이 이 메모리에 올라갈 수 있는 것이죠. 하지만 이 메모리에 올라가면 되면 아래 사진처럼 되게 되는데요



내부 단편화

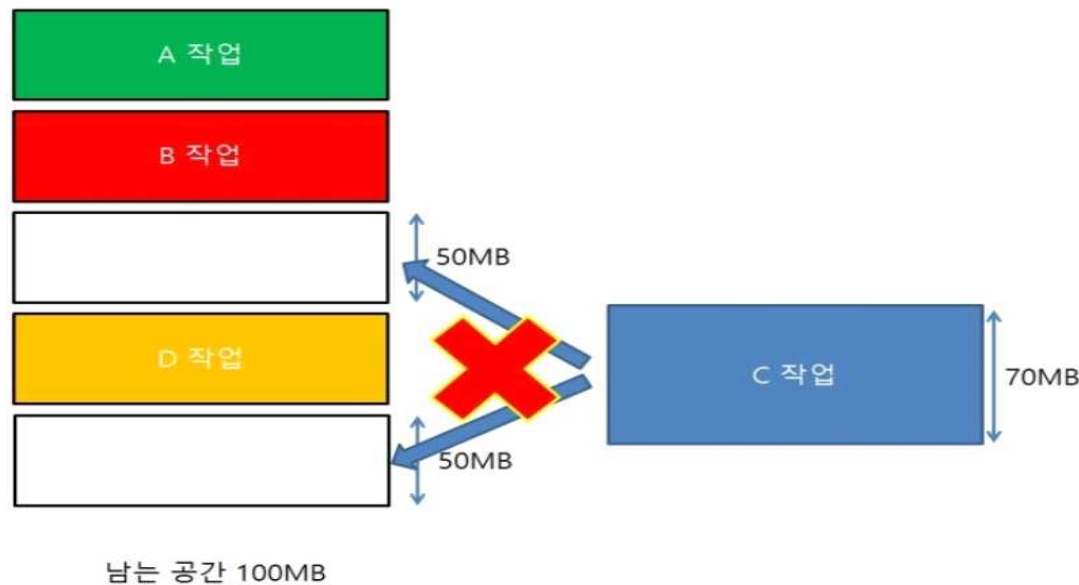
보시는 것처럼 공간이 50MB 인데 안에 작업이 30MB이므로 20MB라는 비어 있는 공간은 실제로 작업되지는 않으며 유용하게 사용될 수 없는 공간이 되는 것이지요. 이러한 낭비가 생기는 것을 내부 단편화라고 합니다.

<External Fragmentation>

= 외부단편화

= 각각의 작업에 맞는 공간들을 할당하다가 공간들에 있던 작업이 해제되면 빈 공간들이 많이 생기게 됩니다. 해당 공간은 정해진 크기가 있으므로 아래의 그림과 같은 문제점을 발생시킵니다.

ex) 100MB의 메모리 공간에 10MB 작업 10개를 올리기 위해 공간(10MB)을 10개 만들어 줍니다. 10개 모두 해제 되는 순간 10MB의 빈 공간 10개가 생기게 되고, 남은 공간이 100MB임에도 불구하고 작업이 11MB만 되어도 메모리에 올릴 수 없게 됩니다.



외부 단편화는 메모리에 남는 공간은 100MB인데 C라는 작업이 70MB일때 일어나는 경우입니다. 메모리에는 100MB라는 용량이 남아 있지만 C작업은 메모리에 들어가지 못하게 됩니다. 즉 작업보다 많은 공간이 있더라도 실제로 그 작업을 받아 들이지 못하는 경우를 외부 단편화라고 하는 것이지요.

- Compaction 또는 coalescing으로 해결합니다.

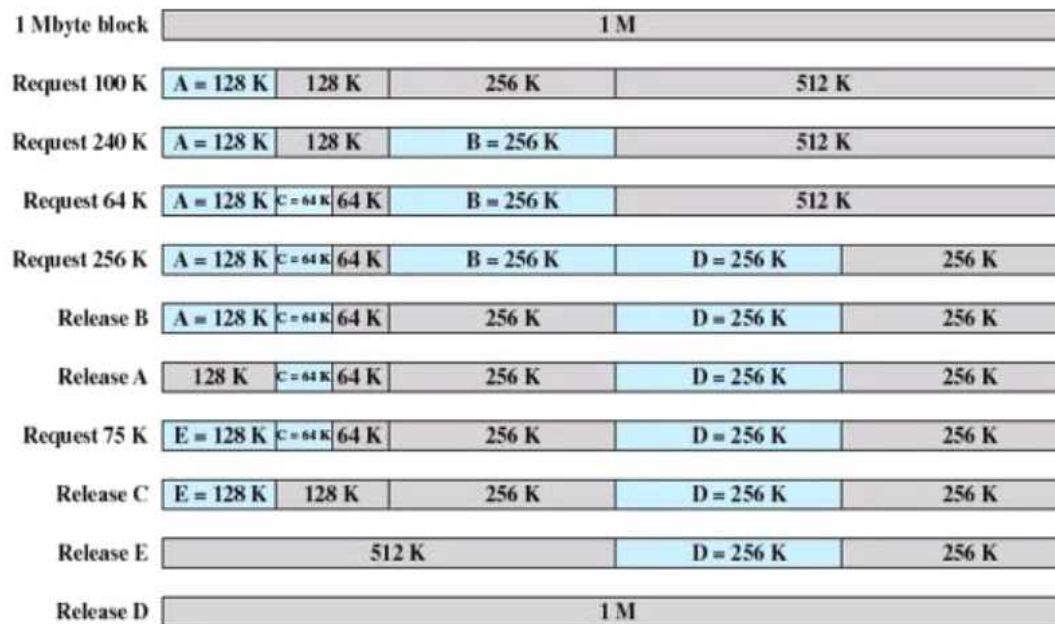
<compaction>

= 메모리 중간 중간에 비어있어서 낭비되는 공간을 없애기 위해 hole들을 전부 붙여버립니다.

<coalescing>

= 인접한 두 Hole만 붙여버립니다.

③ Buddy Algorithm



- 메모리를 분할 할 때 2^n 으로 분할합니다. 그림에서는 1MB를 128KB + 128KB + 256KB + 512KB로 분할했습니다.
- 100KB 프로세스는 64KB보다 크고 ,128KB보다 작으므로 128KB에 들어가고 28KB의 hole을 생성합니다.
- 240KB 프로세스는 128KB보다 크고 , 256KB보다 작으므로 256KB에 들어가고 16KB의 hole을 생성합니다.
- 64KB 프로세스는 32KB보다 크고 , 64보다 작거나 같으므로 64KB에 들어가고 hole을 생성하지 않습니다.
- 256KB 프로세스는 128KB보다 크고 , 256KB보다 작거나 같으므로 256KB에 들어가고 hole을 생성하지 않습니다.
- 프로세스가 Release(해제)되면 자신과 같은 2^n 의 연속된 빈 공간이 있다면 자동으로 합쳐집니다. 그림에서는 64KB의 데이터인 Process_C가 해제 된 순간 , 바로 옆의 64KB 크기의 빈 공간과 합쳐져서(자동 Compact) 128KB의 공간을 만들어 냅니다.

- 128KB 공간에 100KB 프로세스를 넣으면 28KB의 hole이 발생하므로 내부 단편화는 존재하지만 , 외부 단편화는 해결 했습니다.

- 재배치의 경우 Compaction과 Coalescing을 자주 해야 하지만 Buddy는 인근의 같은 2^n 공간과 자동으로 Compaction이 되므로 외부 단편화를 해결 합니다.

5. Virtual Memory 분할 기법_1 : Paging

1) Paging Overview

분할 객체	분할 단위	분할 단위의 크기(Default)
디스크	Block	4KB
실제 메모리	Page Frame	4KB
가상 메모리	Page	4KB

= 메모리의 가상 주소와 실제 주소를 모두 일정하게 Default Size(4KB)로 잘게 분할해서 사용하는 분할 기법입니다.

- 외부단편화는 존재하지 않고 , 프로세스의 마지막 부분에서만 내부단편화가 존재합니다.

- 분할 단위의 크기가 Default로 4KB이므로 고정 분할 기법과 유사하며 내부 단편화가 발생해 봤자 4KB 보다 작습니다.

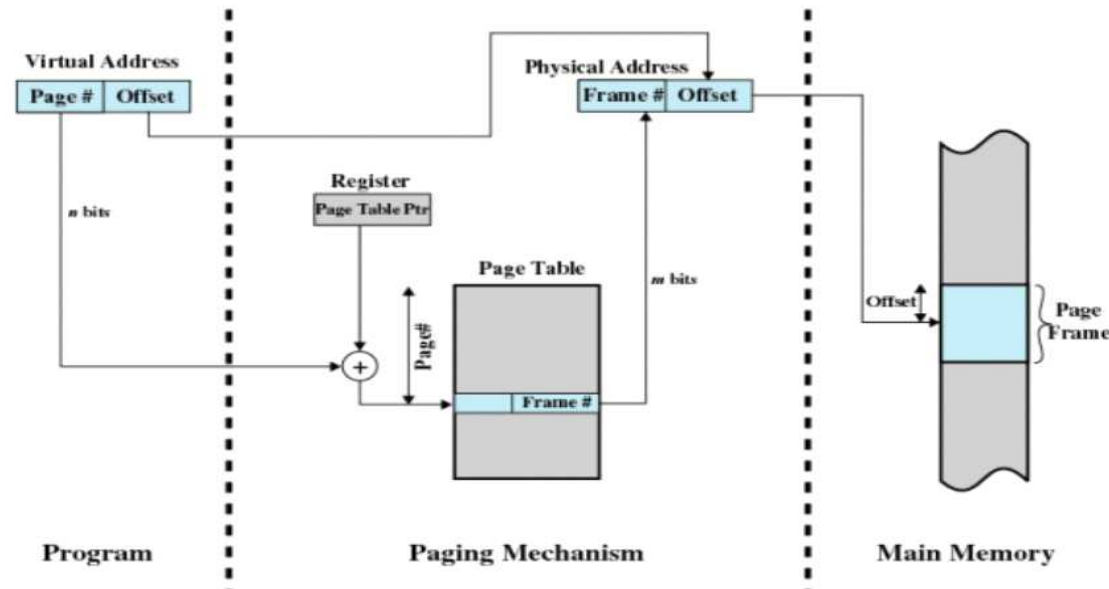
- 고정 분할 기법은 21KB 크기의 프로그램을 메모리에 올리려면 반드시 20KB보다 크거나 같은 공간에 넣어야 합니다. 만약 20KB 보다 크거나 같은 공간이 50KB만 존재한다면 30KB의 Internal Fragmentation이 발생함에도 불구하고 50KB에 넣어야 합니다. 하지만 **페이징 기법은 21KB 프로그램을 메모리에 올리기 전에 6개의 page(4KB)로 분할합니다. 가상 메모리의 6개의 page가 실제 메모리의 6개의 page frame과 page table을 통해 mapping이 되고 , 흩어져서 page frame에 저장됩니다.** 그로 인해 내부 단편화가 엄청나게 줄어 듭니다.

- Page table 대신에 포인터를 사용할 수 있지만 , 4byte씩의 포인터 관련 메모리 낭비가 생깁니다. 따라서 MMU의 도움을 받는 page table이 포인터를 사용하는 것보다 성능이 좋습니다.

- 여담으로 ios는 분할 단위의 크기가 16KB이고 Android는 4KB입니다. 16KB씩 읽으면 메모리의 낭비와 디스크의 낭비가 심하지만 빠릅니다.

ex) 2GB 크기의 영화를 4KB씩 읽고 쓰는 것 보다 16KB씩 읽고 쓰는 것이 당연히 빠릅니다.

2) 구체적인 Paging 과정 (32bit 주소 체계)



① 가상 주소

= 32비트의 가상 주소에서 상위 n 비트를 **Page # (페이지 번호)**라 하고, 하위 $32-n$ 비트를 **offset**이라고 나눌 수 있습니다.

<Example : 도서관에서 257번에 꽂혀 있는 책을 찾는 예시로 위의 개념을 이해해봅시다.>

- 도서관은 1000권의 책이 있고 100번 라인, 200번 라인, 300번 라인, etc, 900번 라인으로 분류를 해놓았습니다. 257번 책은 200번 라인에 가서 57번째에 있는 책입니다. 여기서 200번 라인이 **Page #**이고, 시작부터 57번째가 **offset**입니다.

<Page #>

- **Page #**을 **Page Table**에 input으로 주어주면 **Page Table**에서 탐색 과정을 거쳐 **Frame #**을 output으로 출력합니다.

- **Page #**은 0부터 시스템에서 페이지 테이블에 저장할 수 있는 항목(=Record)의 수(=PTE의 개수)사이의 주소를 나타냅니다.

ex) **Page #**이 6비트라면 페이지 테이블에 저장할 수 있는 항목의 수는 2^6 인 64개가 됩니다. 그러므로 **Page #**은 0~64 사이의 값을 주소로 갖게 됩니다.

<Offset>

- Offset은 가상 주소와 실제 주소가 동일합니다. 확실하지는 않지만 가상 주소 도서관의 200번 라인의 57번째 책은 실제 주소 도서관의 900번 라인의 57번째 책인 것 입니다.

- Offset은 Page의 크기와 Page Frame의 크기를 나타냅니다.

ex) 가상 주소가 16비트 일 때 offset이 10비트라면 Page Frame의 주소 번지가 0 ~ 2^{10} 이 됩니다. Offset은 어떤 Page Frame의 시작 주소에서 offset 만큼 떨어진 곳에 저장함을 나타내기 위해 사용하고 , 메모리는 주소 번지 1개 당 1 Byte를 저장합니다. 따라서 offset이 10비트면 Page Frame의 크기가 1KB($=2^{10}$ Bytes = 1024 Bytes)가 됩니다.(시스템 프로그래밍 교재에 필기했었습니다.) 현대 OS는 Page 와 Page Frame의 크기를 일치 시키므로 Page의 크기도 1KB가 됩니다.

② 실제 주소

- 원리는 가상 주소와 완전히 동일하지만 명칭의 차이만 조금 존재합니다,

- Page Frame #(페이지 프레임 번호), offset으로 이루어져 있습니다.

③ Page Table

= 가상 주소와 실제 주소 사이의 Mapping을 위해 만들어진 Table(단순 배열)입니다.

- P_Bit , D_Bit , A_Bit , 그 외 Bit와 Page Frame #으로 이루어져 있습니다.

플래그 비트

- 접근 비트(Accessed bit) : 페이지에 대한 접근이 있었는지를 나타낸다.
- 변경 비트(Dirty bit) : 페이지 내용의 변경이 있었는지를 나타낸다.
- 존재 비트(Present bit) : 현재 페이지에 할당된 프레임이 있는지를 나타낸다.
- 읽기/쓰기 비트(Read/Write bit) : 읽기/쓰기에 대한 권한을 표시한다.

페이지 테이블은 프로세스마다 존재한다는 사실을 숙지해야 한다(우리가 논의하는 대부분의 페이지 테이블 구조는 프로세스 마다 존재하는 구조이다. 역 페이지 테이블(inverted page table)이라는 예외적인 기법도 있다). 위의 예에서 다른 프로세스를 실행해야 한다면 운영체제는 이 프로세스를 위한 다른 페이지 테이블이 필요하다. 새 프로세스의 가상 페이지는 다른 물리 페이지에 존재하기 때문이다(공유 중인 페이지가 없다면).

- 프로세스 마다 자신의 Page Table을 1개씩 갖고 있고 , 각 프로세스의 페이지 테이블은 실제 메모리에 존재합니다.

- P 값이 0 이면 프로세스가 Swap 영역에 존재하고 , 1 이상이면 Memory에 존재합니다.

- MMU(CPU에 존재하는 H/W)가 Page Table을 참조합니다.

<Page Table Entry(=PTE)>

= Page Table은 하나의 큰 자료구조이며 , 그 자료구조는 n개의 비트로 이루어진 항목(=Record)로 구성되어 있습니다. 이 Record가 PTE 입니다.

- PTE의 크기는 Page Table의 하나의 레코드의 크기이고 , 다시 말해 특수 Bit + Page Frame #의 비트 수 입니다.

ex) PTE가 4Bytes 라면 페이지 테이블을 구성하는 여러 개의 PTE가 모두 4Bytes의 크기이며 , 특수 Bit + Page Frame #의 비트수가 4Bytes라는 의미 입니다.

④ Page Faults

= 사용자가 요구하는 가상 주소를 MMU로 보내어 MMU가 Page Table을 참조해 해당 가상 주소와 match 되는 실제 주소를 확인하지만 실제 주소가 존재하지 않아서 O/S에게 디스크를 참조하도록 인터럽트를 거는 현상.

- Page table에 존재하는 valid bit가 1이면 요구된 페이지의 가상주소를 보고 페이지가 실제 메모리에 존재하는 실제 주소로 이동하여 Page Fault가 발생하지 않습니다. 하지만 valid bit가 0이면 가상주소를 보고 실제 주소로 사상시켜주지 못하므로 Page Fault가 발생합니다.

- Page Fault가 발생하면 OS는 디스크에 접근하게 되므로 추가적으로 수백만 사이클의 시간을 Page Fault처리에 사용합니다.

- Page Table Entry(=PTE = Page Table)에는 index가 존재하지 않고 P_bit , M_bit , 그 외_bit , Page Frame #(페이지 프레임 번호)로 이루어져 있습니다.

- P_bit , M_bit , 그 외_bit를 통해서 실제 메모리의 페이지 프레임에 대한 정보를 얻고 페이지 프레임 번호로 이동합니다.

- [Reference](https://www.youtube.com/watch?v=Nif2TZ5Cohw) : <https://www.youtube.com/watch?v=Nif2TZ5Cohw>

3) 큰 페이지 테이블 해결 방법

① n단계 구조의 페이지 테이블

= Page #의 비트가 크다면 Page Table에 저장할 수 있는 항목수가 많게 되어 Page Table의 크기가 커집니다. 그 결과 Page Frame을 매우 많이 사용하게 되고 , 이는 메모리의 낭비가 됩니다. 그러므로 n단계로 Page Table을 쪼개서 Page Table의 크기를 줄입니다.

- 32 bit 가상주소가 존재하고 11 bit를 Offset으로 한다고 가정합니다. 기존의 방식(1단계 페이지 테이블)으로 페이지징을 하면 , 21bit를 하나의 Page #으로 하기 때문에 21bit에 해당하는 Page Table을 온전히 Page Frame에 저장해야 하고 , 이는 매우 큰 메모리 낭비를 발생 시킵니다. 심지어 2^{21} bit에 해당하는 Page Frame을 절대 모두 사용하지 못합니다. 그러므로 새로운 방식(3단계 페이지 테이블)으로 페이지징을 하면 , 21bit를

7, 7, 7 bit로 나눠서 3개의 페이지 테이블을 만들어 줍니다. 각각의 Table은 7bit에 해당하는 Page Table을 만들어서 Page Frame에 저장하기 때문에 하나의 PTE가 1Byte 라고 하면 2^{21} bytes의 Page Table을 Page Frame에 저장하던 방식을 -> 2^7 bytes * 3의 Page Table을 Page Frame에 저장합니다.

- 이 방법이 시간은 좀 더 걸리지만 메모리의 낭비는 급격하게 줄일 수 있습니다.

ex) 전교생이 10000명인 학교에서 학생을 학생 번호 6452번으로 저장하다가 -> 3학년 14반 47번 학생으로 저장합니다.

② Inverted Page Table(역 페이지 테이블)

= Page Table에서 사용하지 않는 부분이 낭비 되므로 실제로 쓰는 부분만 역으로 테이블을 만드는 기법입니다.

- Hash Function을 사용합니다.

= Hash Function의 결과인 Hash Value는 해시 테이블의 Index로 사용됩니다.

- Hash는 하나 이상의 값을 연결할 수 있기 때문에 체인 기법을 사용해서 혼선을 없앱니다.(파일처리)

③ TLB(Translation Lookaside Buffer)

= H/W에 고속의 캐시 버퍼를 사용해서 속도를 증가시키는 기법입니다.

= Page Table의 캐시라고 생각하면 됩니다.

- 최근에 참조한 프로세스는 또 참조할 가능성이 크기 때문에 그 프로세스를 TLB에 저장합니다.

- MMU와 Page Table로 mapping 하는 방식도 빠르지만 결국에는 Loop를 돌아서 mapping되는 실제 주소를 찾아야합니다. 하지만 TLB는 특수 고속 캐시를 사용하므로 Loop를 돌지 않고 한 번에 가상 주소와 mapping 되는 실제 주소를 찾아냅니다. 그러므로 TLB 방식은 MMU-Mapping에 비하여 매우 빠릅니다.

- TLB를 먼저 검사하고 원하는 프로세스가 존재 하지 않는다면 기존의 MMU-Mapping 방식으로 mapping 합니다.

<Associative Mapping (연관 사상)>

= TLB가 특수 고속 캐시를 사용해서 한 번에 mapping 하는 방식.

4) Paging의 주요 고민

- 페이지 크기가 작으면 내부 단편화의 양이 적지만 페이지 테이블에 들어가는 레코드(=항목)의 개수가 많아지므로 결국 테이블이 커집니다. 그 결과 페이지 폴트가 자주 발생합니다.
- 현대 OS에서 사용하는 메모리가 커지지만 , 프로세스의 크기도 커지고 있습니다. 그 결과 점점 Localization이 점점 감소됩니다.

5) 문제 풀이

<Tip>

- 1KB = 2^{10} Bytes
- 1MB = 2^{20} Bytes
- 1GB = 2^{30} Bytes
- 가상 주소와 실제 주소가 다른 비트 수로 구성 된 경우가 많습니다.
- 페이지 크기 = Page # , 페이지 프레임 크기 = Page Frame #
- offset 부분이 1010111 이고 페이지 #이 십진수로 2라면 10/1010111로 입니다.

<문제 1 : 16비트 가상 주소에서 6비트를 페이지번호로 하면 페이지의 크기는 얼마입니까?>

= offset이 10비트 이므로 페이지의 크기는 2^{10} bytes(=1KB)입니다.

<문제 2 : 32비트 가상 주소에서 16KB의 페이지 크기를 갖고 36비트 실제주소에서 페이지 테이블의 항목 수는 얼마입니까?>

= 32비트 중 16KB는 2^{14} 바이트 이므로 offset은 14비트이고 page #은 18비트입니다. 36비트도 14비트가 offset이므로 22비트.

<문제 3 : 24비트 가상 주소에서 256바이트 페이지 크기를 갖는 가상 주소가 0x123456이면 offset은 얼마입니까?>

= 페이지 크기가 256 바이트(2^8)이므로 offset은 8비트입니다. 그러므로 16진수로 나타낸 0x123456의 끝에 있는 두 개의 수 0x56이 offset입니다.

<문제 4>

페이지 테이블은 16비트 가상 주소와 4K 페이지를 사용한다고 할 때, 각 프로세스의 페이지 테이블을 보고 아래 표의 빈 칸을 채우시오.

Process 1		Process 2	
0	3	0	1
1	0	1	6
2	7	2	4
3	5	3	2

Process	Address	Page #	Offset	물리 Address
1	12,000	(1)	(3)	(5)
2	8,000	(2)	(4)	(6)

= 페이지의 크기가 4KB(2^{12} 바이트)이므로 4비트의 Page #과 12비트의 Offset을 갖습니다.

<Process_1>

- 가상 주소가 10진수로 12000이므로 2진수로 나타내면 0x 0010 1110 1110 0000 이고 16진수로 나타내면 0x2EE0입니다.
- 0x2EE0 중에서 Page #이 4비트이므로 0x2이고 , offset은 12비트이므로 0xEE0입니다.

(1) : 0x2

(3) : 0xEE0

(5) : 0x7EE0

<Process_2>

- 0001 1111 0100 0000

(2) : 0x1

(4) : 0xF40

(6) : 0x6F40

<문제 5>

운영체제의 주소 공간을 위해 단일 단계 페이지 테이블을 사용한다고 할 때 다음 표는 가상페이지 번호와 이에 대응하는 페이지 프레임 번호를 보여준다. 모든 엔트리는 valid하다고 하고, 페이지 크기는 4096(byte), 운영체제 페이지 테이블의 시작주소(물리 주소)가 0x3420이고 하나의 PTE 크기는 8 byte 일 때 가상 페이지 3번에 대응하는 PTE의 가상주소(운영체제 주소공간)는?

가상 페이지 번호	물리 페이지 프레임 번호
0	0
1	12
2	2
3	7
4	9
5	8
6	20
7	4
8	3
9	1

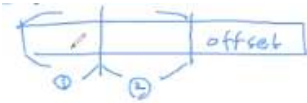
- 모든 엔트리는 valid = p_bit가 1이어서 프로세스가 메모리에 있음
 - 페이지 크기 = 4096바이트 = 2^{12} 바이트 -> offset은 2^{12}
 - 이 페이지 테이블이 실제 메모리에 저장된 주소가 0x3420입니다. -> 메모리의 3번 페이지 프레임에 420 offset 부터 해당 테이블 존재.
 - 근데 페이지 테이블을 참조 하면 3번 프레임은 가상 주소의 페이지 번호 8번과 mapping 됩니다.
 - 그러므로 해당 페이지 테이블이 실제 메모리에 저장 된 곳의 시작 주소를 찾기 위한 가상 주소의 시작 주소는 0x8420입니다.
 - 가상 주소의 page #이 3번이면 0x18을 더합니다. 그러므로 0x8438 입니다.
 - 가상 페이지 3번에 대응하는 PTE의 주소와 가상 페이지 3번에 대응하는 녀석의 주소는 다릅니다. 이 문제는 int arr[10]의 주소가 100일 때 arr[3]의 주소를 묻는 문제지 , arr[3]에 어떤 주소를 저장했을 때 그 주소를 구하는 게 아닙니다.
 - 결국 페이지 , 페이지 프레임 번호는 메모리를 쪼개 순서 입니다.
- ex) 16byte 메모리를 1byte로 쪼개면 0번이 0~1byte고 , 1번이 1~2byte구역 입니다.

<문제 6>

가상 메모리 시스템을 설계할 때 페이지 테이블 항목의 크기는 4-byte이고 각 페이지 테이블은 단일 물리적 프레임에 대응되며 가상 주소 공간을 (256GB) 지원하고, 2단계 페이징 테이블 기법을 사용한다고 할 때, 시스템의 최소 페이지 크기는 얼마인가?

- 페이지 테이블 항목의 크기 = PTE의 크기
- 각 페이지 테이블은 단일 물리적 프레임에 대응되며 = 한 프로세스의 페이지 테이블은 한 페이지 프레임에 들어갑니다. 해당 조건문을 문제에서 적어 주지 않아도 이 조건은 Default로 적용된다고 생각하고 문제를 푼시다.

① 2단계 페이징 기법이므로 그림을 그리고 시작 합시다.

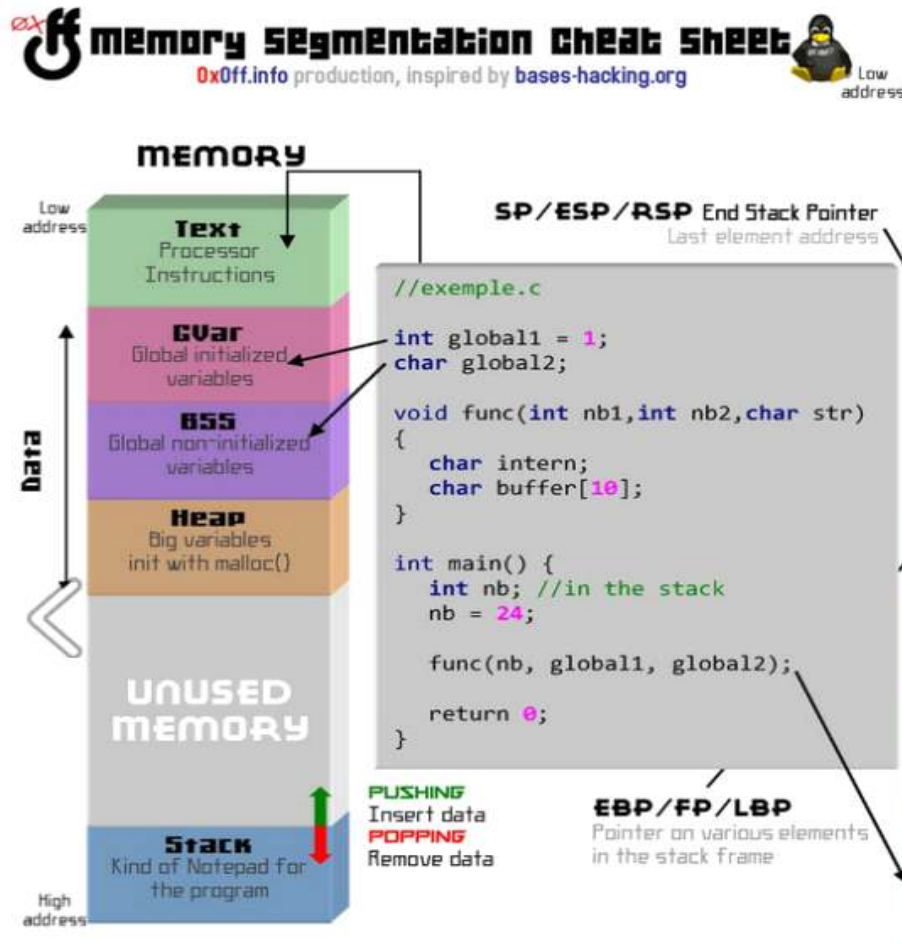


- ② 가상 주소가 256GB $\rightarrow 2^{38}$ bytes $\rightarrow$ 38비트로 가상 주소가 구성되어 있습니다.
- ③ PTE가 4bytes 이므로 페이지 테이블의 항목1개의 크기가 4bytes 입니다.
- ④ 페이지 프레임의 크기를 구하는 문제고 , 이는 offset과 같습니다.
- ⑤ 1단계라면 페이지 프레임 안에 엄청 큰 페이지 테이블이 전부 들어가야 하지만 , 2단계이므로 반으로 쪼개서 그 크기만 들어가는 페이지 프레임의 크기를 구하면 됩니다.
- ⑥ 페이지 프레임의 크기를 2^p bytes라고 하면 offset은 p개의 비트로 구성됩니다.
- ⑦ PTE가 4bytes이므로 페이지 테이블의 최대 항목 수는 $2^{(p-2)}$ 입니다. 왜냐하면 2^p bytes의 페이지 프레임에 크기가 4bytes인 항목을 $2^{(p-2)}$ 개 넣을 수 있기 때문입니다. 페이지 테이블의 최대 항목 수는 Page #입니다.
- ⑦ 그러므로 38 비트 중에서 p-2 비트 2개와 offset인 p 비트를 더해서 38이 나와야 하므로 p=14가 됩니다. 그러므로 p가 페이지 프레임의 크기기 때문에 2^{14} Bytes = 16KB가 됩니다.

- 3단계로 나누면? $4p - 6 = 38$ 이므로 정답은 2^{11} Bytes가 됩니다. (위의 단계를 이해하면 매우 간단합니다.)

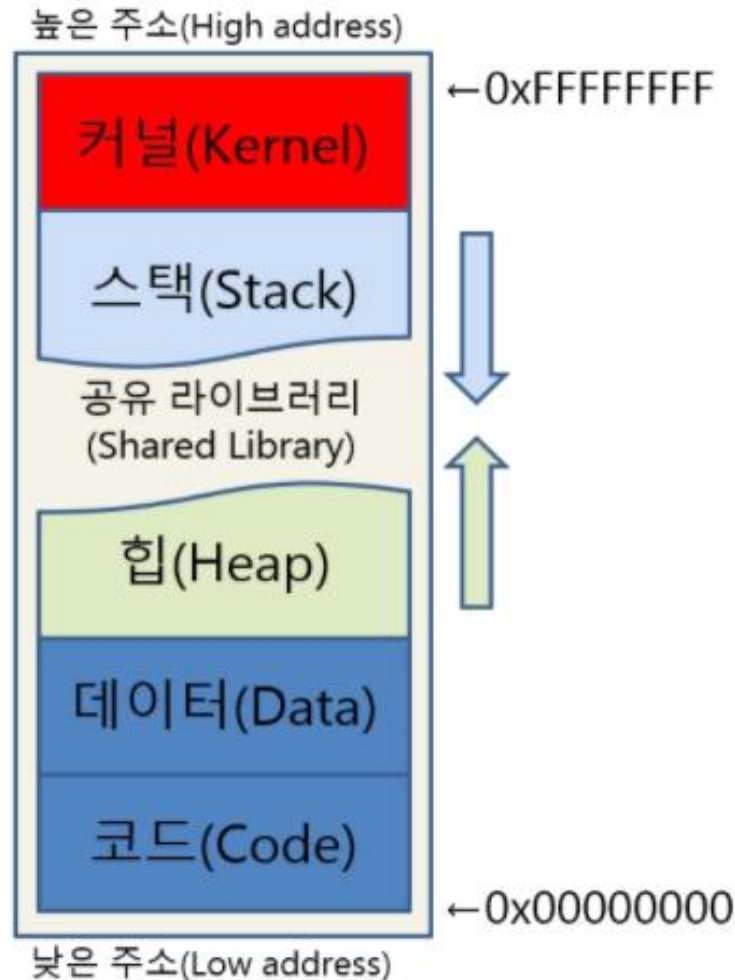
6. Virtual Memory 분할 기법_2 : Segmentation

1) Segmentation 정의



- 실제 메모리를 4가지 Segment(Code, Data, Stack, Heap)로 구분할 수 있습니다. 이를 Segmentation이라고 합니다.
- Segmentation을 사용하면 OS는 각 Segment를 실제 메모리의 각기 다른 위치에 배치할 수 있고, 사용되지 않는 가상 주소 공간이 실제 메모리를 차지하는 것을 방지할 수 있습니다.
- Paging은 물리적인 크기의 블록(4KB)마다 메모리를 자르는 방식이지만 Segmentation은 논리적인 크기의 Segment마다 메모리를 자르는 방식이기 때문에 각각의 Segment(Code, Data, Stack, Heap)의 크기가 일반적으로 같지 않습니다.
- Segment의 크기는 고정되어 있지 않고 가변적이기 때문에 크기를 변경시킬 수 있습니다.
- OS는 Paging과 Segmentation을 함께 사용할 수 있기 때문에 Segment들을 Page 단위로 쪼갭니다.
- 가상 주소를 Segment 기법으로 나눠서 Segment #, Offset으로 할 수 있지만 가변길이 이므로 계산하기가 어렵습니다.
- Paging과 다르게 내부단편화는 발생하지 않으나, 적은 양의 외부 단편화는 발생합니다.

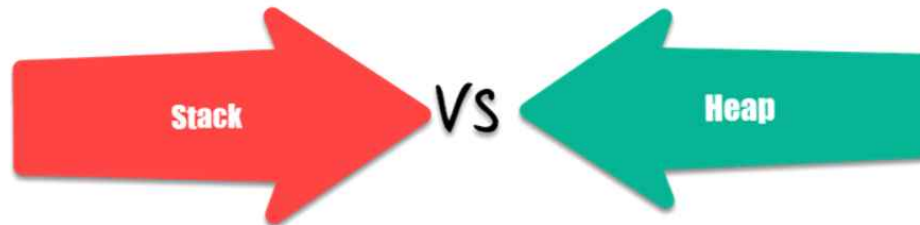
2) Process Image(프로세스 구조) : 프로세스는 분리되어 메모리의 각각의 Segmentation(stack , heap , data , code)에 저장됩니다.



1. Code(=Text)
= Vs Code에서 작성한 소스 파일이 실행파일 상태(기계어)로 저장 되어있는 메모리 영역
2. Data
= **전역 변수와 Static 변수**가 저장되는 메모리 영역
= Data영역을 초기화 된 Data 영역과 초기화 되지 않은 Data 영역으로 나눌 수 있는데 초기화 되지 않은 Data영역을 BSS(Block Started By Symbol)라고 합니다.
3. Heap
= 낮은 주소에서 높은 주소로 확장되며 , **사용자가 직접 할당하고 해제**해야 하는 메모리 영역
4. Stack
= 높은 주소에서 낮은 주소로 확장되어서 커널 영역을 절대 침범하지 않습니다. 스스로 할당되고 해제되고 , 지역변수와 매개변수가 저장되는 메모리 영역
5. Kernel
= OS 내부에 존재하는 핵심적인 소프트웨어 들이 저장되는 메모리 영역이므로 사용자가 접근할 수 없습니다.
= **메모리에 상주**합니다.

- VS Code에서 주소를 출력해보면 Data영역이 Stack영역보다 주소 값이 큰데 , 이는 가상 주소이기 때문에 그런 것 같습니다.

3) Stack vs Heap



Parameter	Stack	Heap
Type of data structures	A stack is a linear data structure.	Heap is a hierarchical data structure.
Access speed	High-speed access	Slower compared to stack
Space management	Space managed efficiently by OS so memory will never become fragmented.	Heap Space not used as efficiently. Memory can become fragmented as blocks of memory first allocated and then freed.
Access	Local variables only	It allows you to access variables globally.
Limit of space size	Limit on stack size dependent on OS.	Does not have a specific limit on memory size.
Resize	Variables cannot be resized	Variables can be resized.
Memory Allocation	Memory is allocated in a contiguous block.	Memory is allocated in any random order.

- 스택에는 지역변수 , 함수의 매개변수 , 포인터 변수가 저장됩니다.

- 힙(heap) 영역은 사용자가 직접 관리할 수 있는 '그리고 해야만 하는' 메모리 영역입니다.

- 힙 영역은 사용자에게 의해 메모리 공간이 동적으로 할당되고 해제됩니다. 힙 영역의 크기는 RAM에 의존하므로 결국 한정되어 있기 때문에 반드시 Free()를 통해 사용하지 않는 것은 해제해야 합니다.

Allocation and Deallocation	Automatically done by compiler instructions.	It is manually done by the programmer.
Deallocation	Does not require to de-allocate variables.	Explicit de-allocation is needed.
Cost	Less	More
Implementation	A stack can be implemented in 3 ways simple array based, using dynamic memory, and Linked list based.	Heap can be implemented using array and trees.
Main Issue	Shortage of memory	Memory fragmentation
Locality of reference	Automatic compile time instructions.	Adequate
Flexibility	Fixed size	Resizing is possible
Access time	Faster	Slower

<힙에 저장 되는 것>

- If a value type is part of a class (as in your example), it will end up on the heap.
- If it's boxed, it will end up on the heap.
- If it's in an array, it will end up on the heap.
- If it's a static variable, it will end up on the heap.
- If it's captured by a closure, it will end up on the heap.
- If it's used in an iterator or async block, it will end up on the heap.
- If it's created by unsafe or unmanaged code, it could be allocated in any type of data structure (not necessarily a stack or a heap).

```
int main(){  
    Person person; // on stack  
    Person* person2 = new Person(); // on heap  
}
```

Whether or not the variables are on the heap or stack depends upon how the object is allocated. So all members of the class will be allocated how the object is allocated.

<Heap Size에 대한 설명과 , Heap을 사용하는 순간>

The heap is the segment of memory that is not set to a constant size before compilation and can be controlled dynamically by the programmer. Think of the heap as a “free pool” of memory you can use when running your application. The size of the heap for an application is determined by the physical constraints of your RAM (Random access memory) and is generally much larger in size than the stack.

We use memory from the heap when we don't know how much space a data structure will take up in our program, when we need to allocate more memory than what's available on the stack, or when we need to create variables that last the duration of our application. We can do that in the C programming language by using malloc, realloc, calloc and/or free. Check out the example below.

4) Segmentation Fault (= Segment Fault)

Segmentation fault - a **memory access violation** caused by a program attempting to access an illegal memory location

= 불법적인 주소에 접근할 시에 발생합니다.

- 만약 가상 주소에서 힙의 영역이 a번지 ~ b번지 일 때 실제 주소에서 힙의 영역이 a' 번지 ~ b' 번지가 됩니다. 사용자가 b번지를 넘어서 b+100 번지에 접근하고 힙에 선언하려 하면 OS는 Segment Fault를 발생시키고 강제 종료시킵니다. (확실하지 않음)

① vector에 할당되지 않은 메모리 지역의 값 요구

```
vector<int> v;
int main()
{
    for(int i=0; i<1; i++)
    {
        cout << v[i];
    }
    return 0;
}
```

```
pjw96@ParkJiWon ~/BOJ/source
$ ./a.exe
Segmentation fault (core dumped)
```

- vector를 선언만 하고 메모리를 할당하지 않은 상황에서 vector에 포함된 원소의 값을 요구하기 때문에 Segmentation Fault 발생.
- reserve()를 사용하면 이유는 모르지만 segmentation fault가 발생하지 않습니다,

② 공부하세요!!!

5) Segmentation의 장점

① 보호나 공유에 좋습니다.

- 메모리를 절약하기 위해 때로는 주소 공간들 간에 특정 메모리 Segment를 공유합니다. 특히 Code 영역을 공유합니다.

② Locality 문제를 해결 합니다.

<Thrashing>

= 메모리가 가득 차서 어떤 페이지를 메모리에서 swap으로 보냅니다. 이 과정에서 I/O가 발생합니다. 그런데 방금 보낸 프로세스가 메모리에 필요해서 다시 메모리로 올립니다. 이 과정에서 또 I/O가 발생합니다. 이런 과정이 자주 일어나면 의미 없는 I/O(블록 교체)가 많이 발생하므로 Performance가 좋지 않게 됩니다.

- Thrashing 때문에 메모리가 가득 찼을 때 어떤 페이지를 메모리에서 swap으로 보내야 의미 없는 I/O를 줄일 수 있을지 OS는 고안해야합니다. 현대 OS는 Principle of locality(지역성의 원리)로 Thrashing을 해결합니다.

<Locality> (조금 다시 내일 공부해서 하자...)

= 메모리의 특정 페이지를 참조하면 그 다음에는 주변에 있는 페이지가 참조되는 경우가 많습니다. 그러므로 어떤 페이지 근처의 페이지들을 group화하여 이 뭉텅이를 Resident Set이다. -> 그 뭉텅이들에 있는 page는 빼지 말자.

<Locality(지역성) 문제>

- Spatial Locality(공간적 지역성), Temporal Locality(시간적 지역성) 으로 나눌 수 있습니다.

= <Locality(구역성) 문제>

= RSS(Resident Set) - 메모리에 올라갈 때 우리는 On-Demand를 쓰지만 조금은 예측해서 프로세스를 메모리에 올리고 싶어.

두가지 지역성 -> Spatial locality(공간적) , temporal locality(시간적) 들로 뭉텅이 이름

<15강_Address Translation> 10/15

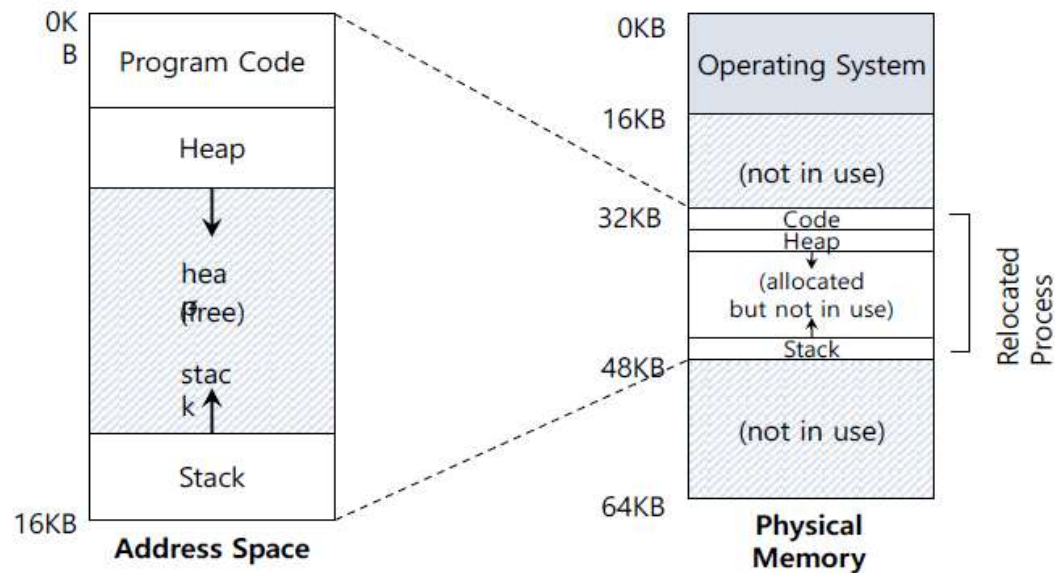
- = 주소 변환은 가상 주소를 실제 주소로 변환 시키는 방법을 의미 합니다.
- = 이번 강에서는 현대 OS가 사용하는 가상 주소 방식과는 다른 학습을 위한 버전으로 설명 합니다.
- = 아래 그림으로 모든 내용을 설명 하겠습니다.

```
void func () {  
    int x = 3000;  
    x = x + 3;  
    ...  
}
```



Presumption: ebx has address of x which is located 15KB

```
128 movl 0x0 (%ebx), %eax ; load 0+ebx into eax  
132 addl $0x03, %eax      ; add 3 to eax register  
135 movl %eax, 0x0 (%ebx) ; store eax back to mem
```



1. 가정

- Address Space는 사용자가 생각하는 프로세스의 주소 공간 이고 , Physical Memory는 프로세스가 실제로 저장 되는 실제 메모리 입니다.
- 주소 공간은 실제 메모리 내에 존재하므로 실제 메모리 보다 크기가 작습니다. 그림을 참고하면 , 주소 공간은 16KB이고 실제 메모리는 64KB 입니다.
- 모든 프로세스는 16KB로 통일 합니다.
- 이 가정들은 모두 말도 안 되는 이야기입니다.

2. 과정

- Program Code에는 사용자가 적은 코드가 어셈블리어로 변환되어 저장됩니다.
- `int x = 3000;` -> x가 스택에 올라가고 값 3000을 갖게 됩니다.
- 사용자 입장에서 주소는 0KB번지부터 시작 되지만 실제 메모리에서는 32KB번지부터 시작됩니다. 따라서 주소 공간을 실제 메모리의 주소로 변환시켜 주어야 합니다.
- Base Register(32KB)를 두고 이 값에 offset을 더해서 mapping을 시켜줍니다.
- Limit Register(=Bound Register)를 48KB로 설정해서 Base Register + offset이 48KB를 넘으면 할당된 실제 메모리의 주소를 오버하는 것이므로 ERROR가 나도록 합니다.
- 주소 공간을 0KB번지로 했으니까 실제 메모리의 0KB번지에 올리면 될 것 같지만 , 0KB번지는 Kernel 영역 입니다.
- 주소 공간을 32KB번지로 하고 실제 메모리의 32KB번지에 올린 다면 Mapping이 필요 없게 되므로 좋은 방법일 것 같지만 , 사용자가 32KB부터 시작을 하면 실제로는 32000Byte 부터 주소를 사용하는 것이므로 주소가 매우 길어지고 복잡해집니다.

3. 단점

- 1) 주소 공간을 실제 메모리에 올리는 것은 메모리의 낭비 입니다. 그로 인해 실제 메모리에 주소 공간을 올리지 않는 가상 주소를 사용하게 됩니다.
- 2) 이 방식은 결국 , 고정 분할 방식과 유사하여 사용자가 실제 메모리에 프로세스를 올릴 때 정해진 장소에만 올릴 수 있습니다. 그로 인해 실제 메모리에 원하는 장소에 올릴 수 있는 가상 주소를 사용하게 됩니다.

<21강_Page Replacement> 12/17

<Page Replacement>

= 메모리에 stream에서 요청한 페이지가 존재하지 않고 , 디스크에 해당 Page가 존재하면 Page Fault가 발생합니다. 이를 해결하기 위해 메모리에 존재하는 어떤 페이지와 디스크에 존재하는 요청한 Page를 교환해야 합니다. 이 과정에서 **메모리에 어떤 페이지와 교환할 지를 정하는 6가지의 알고리즘**을 배웁니다.

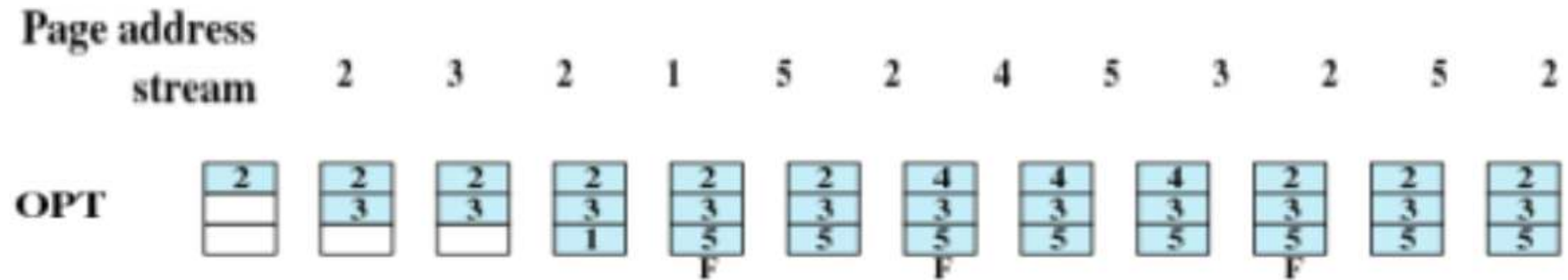
- 페이지는 4KB(default) 단위로 메모리를 구성하며 , 여러 개의 불연속 페이지가 1개의 프로세스를 구성합니다.

<6개의 페이지 교체 기법 설명에서 사용할 전제 조건>

Page address												
stream	2	3	2	1	5	2	4	5	3	2	5	2

- 1 ~ 5는 페이지의 이름입니다.
- 2 -> 3 -> 2 -> etc -> 5 -> 2 순서로 메모리에 해당 페이지가 있는지 확인합니다.
- 그림에서 나오는 모든 배열모양은 메모리이며 , 메모리 내부에 존재하지 않는다면 Page Fault입니다.
- 초기에는 메모리가 비어있는 상태이므로 무조건 Page Fault가 일어납니다.

1. Optimal



= 미래를 예측해서 페이지를 교체합니다.

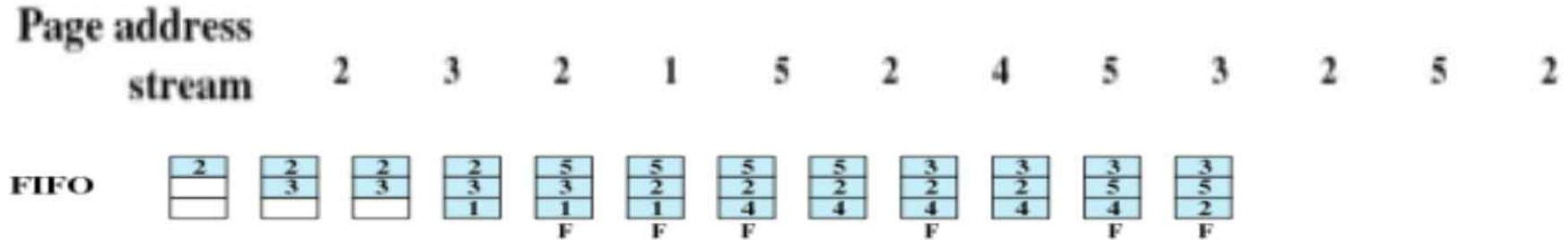
- 완벽한 optimal을 구현하지 않고, 페이지 폴트의 횟수를 고려하여 몇 %가 완벽한 optimal과 비슷한지 확인합니다.

- 1번 배열을 해석할 때 , 2번 페이지는 비어있는 상태니까 Page Fault가 발생하고 메모리에 2번을 넣습니다. 그림에서는 Page Fault(F)를 표기하지 않았지만 사실은 F로 표기해야 합니다. 2번 배열 , 4번 배열도 마찬가지입니다.

- 4번 배열 상태에서 Page address stream에서 5번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 2는 6번째 stream으로 오고, 3은 9번째 stream으로 오지만 1번은 stream으로 오지 않기 때문에 1과 교체하고 5를 메모리에 올립니다.

- 이처럼 미래를 예측해야 하므로 구현이 불가능 합니다.

2. FIFO (First In First Out)



= 현재 저장되어 있는 페이지 중에서 가장 오래있었던 페이지와 교체합니다.

- 4번 배열 상태에서 Page address stream에서 5번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야합니다. 현재 메모리에서 가장 오래 있었던 2와 5를 교체합니다.
- 5번 배열 상태에서 Page address stream에서 6번째 페이지인 페이지_2가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야합니다. 현재 메모리에서 가장 오래 있었던 들어온 3과 2를 교체합니다.
- 6번 배열 상태에서 Page address stream에서 7번째 페이지인 페이지_4가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야합니다. 현재 메모리에서 가장 오래 있었던 들어온 1과 4를 교체합니다.
- 지금은 메모리에 3개의 페이지를 저장하지만 4개의 페이지를 저장할 수 있게 해도 Page-Fault가 적게 일어나지 않아 성능이 좋아지지 않습니다. 이걸 FIFO 이상 현상 이라고 합니다.

3. LIFO (Last In Last Out)

= 현재 저장되어 있는 페이지 중에서 가장 적게 있었던 페이지와 교체합니다.

- 4번 배열 상태에서 Page address stream에서 5번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 현재 메모리에서 가장 적게 있었던 1과 5를 교체합니다.
- 6번 배열 상태에서 Page address stream에서 7번째 페이지인 페이지_4가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 현재 메모리에서 가장 적게 있었던 5와 4를 교체합니다.
- FIFO와 비슷한 성능이 나옵니다.

4. LRU(Least Recently Used)

Page address
stream

2 3 2 1 5 2 4 5 3 2 5 2

LRU

2	2	2	2	2	2	2	2	3	3	3	3
	3	3	3	5	5	5	5	5	5	5	5
			1	1	1	4	4	4	2	2	2
				F		F		F	F		

= 최근에 가장 참조가 되지 않은 페이지와 교체하기

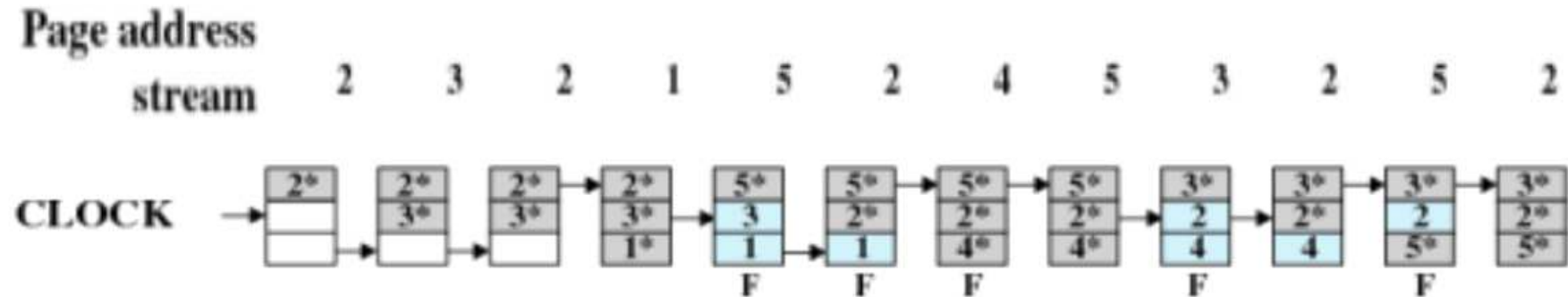
- 오버헤드(시간을 모두 저장함)가 매우 커지기 때문에 오히려 성능이 떨어집니다.
- 4번 배열 상태에서 Page address stream에서 5번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 페이지_2는 4번 배열 상태 기준으로 Page address stream을 참고하면 1번째, 3번째에 참조했기 때문에 1(4번째 - 3번째)번 전에 참조했습니다. 페이지_3은 4번 배열 상태 기준으로 Page address stream을 참고하면 2번째에 참조했기 때문에 2(4번째 - 2번째)번 전에 참조했습니다. 페이지_1은 4번 배열 상태 기준으로 Page address stream을 참고하면 지금 참조했기 때문에 0(4번째 - 4번째)번 전에 참조했습니다. 그러므로 5와 3을 교체합니다.
- 5번 배열 상태부터 동일한 방식으로 계속 진행합니다.

5. LFU (Least Frequently Used)

= 최근에 자주 참조가 되지 않는 페이지와 교체하기

- 오버헤드가 매우 커지기 때문에 오히려 성능이 떨어집니다.
- 모든 페이지 마다 몇 번의 참조가 발생했는지 기록합니다.
- 4번 배열 상태에서 Page address stream에서 5번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 페이지_1은 1번 , 페이지_2는 2번 , 페이지_3은 1번 참조된 상태입니다. 이렇게 같은 참조횟수가 존재할 때는 FIFO를 사용하여 먼저 들어온 페이지를 뺍니다. 3번이 먼저 들어왔으므로 3과 5를 교환합니다.
- Page Fault가 일어나지 않은 경우에서도 해당 페이지에 대한 참조 횟수는 증가시킵니다.
- 5번 배열 상태에서 동일한 방식으로 계속 진행합니다.

6. Clock (Not Used Recently)



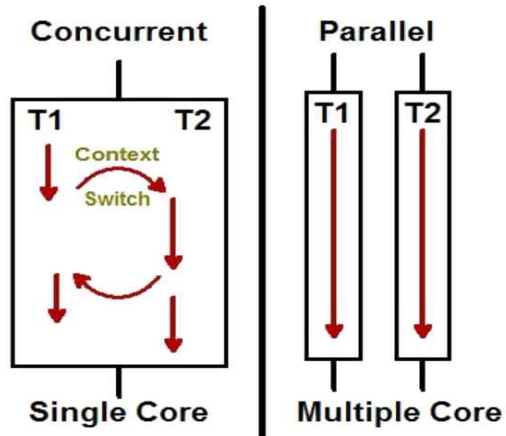
- 별도의 오버헤드 없이 커널이 제공하는 변수, 자료를 그대로 이용하여 LRU와 성능이 비슷하게 구현한 Page Replacement 방법
- Page Table에서 사용하는 write-bit(w=m=d) 와 read-bit(r=u)을 사용하는 2 handed clock이 존재하지만 강의에서는 write-bit와 read-bit를 합쳐서 *로 나타내는 1 handed clock만을 공부합니다.
- 페이지에 그림과 같이 *을 붙이면 참조를 했다는 의미로 교체할 수 없습니다.
- 페이지에 *을 붙이지 않으면 교체를 할 수 있습니다.
- 화살표는 포인터이며, 포인터가 가리키는 페이지를 참조했는데 *가 있다면, *을 제거하고 다음 페이지를 포인터로 가리킵니다.
- 3번 배열 상태에서 Page address stream에서 2가 들어왔는데 2도 아닌 2*이 있다면 포인터도 움직이지 않고 그냥 넘어갑니다.
- 4번 배열 상태에서 Page address stream에서 5번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 포인터가 가리키는 2*은 교체할 수 없기 때문에 *을 떼고 포인터를 이동시킵니다. 포인터가 가리키는 3*은 교체할 수 없기 때문에 *을 떼고 포인터를 이동시킵니다. 포인터가 가리키는 1*은 교체할 수 없기 때문에 *을 떼고 포인터를 이동시킵니다. 포인터가 가리키는 2는 *이 없기 때문에 교체할 수 있습니다. 그러므로 2와 5를 교환하고 5에 *을 붙이고 포인터를 한 칸 이동 시킵니다.
- 5번 배열 상태에서 Page address stream에서 6번째 페이지인 페이지_2가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야 합니다. 포인터가 가리키는 3은 교체할 수 있기 때문에 2와 교체할 수 있습니다. 그러므로 3과 2를 교체하고 2에 *을 붙이고 포인터를 한 칸 이동 시킵니다.
- 9번 배열 상태에서 Page address stream에서 10번째 페이지인 페이지_2가 메모리에 존재하지만 *이 붙어있지 않습니다. 그러므로 2에 *을 붙

여주고 반드시 포인터는 이동시키지 않습니다.

- 10번 배열 상태에서 Page address stream에서 11번째 페이지인 페이지_5가 메모리에 존재하지 않기 때문에 메모리에 존재하는 페이지를 1개 교체해야합니다. 2*에서 *을 제거하고 포인터를 한 칸 이동하면 4를 만나고 4는 *이 없기 때문에 교체할 수 있습니다. 그러므로 4와 5를 교체하고 5에 *을 붙이고 포인터를 한 칸 이동시킵니다.

<26강_Concurrency(동시성)> 10/27 ~ 10/28

1. Concurrency(동시성) vs Parallelism(병렬성)



동시성	병렬성
동시에 실행되는 것 같이 보이는 것	실제로 동시에 여러 작업이 처리되는 것
싱글 코어에서 멀티 쓰레드(Multi thread)를 동작시키는 방식	멀티 코어에서 멀티 쓰레드(Multi thread)를 동작시키는 방식
한번에 많은 것을 처리	한번에 많은 일을 처리
논리적인 개념	물리적인 개념

- 좌측 그림은 Thread를 2개 처리하는 예제입니다.

① Single Core

= Thread 단위로 scheduling하여 빠르고 자주 Context Switching을 하여 CPU 제어 권한을 넘겨주어 처리합니다.

② Multiple Core

= Thread_1은 1번 Core가 , Thread_2는 2번 Core가 처리하여 동시에 독립적으로 처리합니다.

*** OS 강의에서 대단원_2가 Concurrency이므로 , 이 강의에서 배우는 Multi-Thread는 싱글 코어에서 동작하는 방식입니다. ***

<reference>

<https://seamless.tistory.com/42>

https://m.blog.naver.com/PostView.nhn?blogId=and_lamyland&logNo=221182893855&proxyReferer=https:%2F%2Fwww.google.com%2F

2. 자원의 소유 vs 스케줄링 단위

1) 자원의 소유

- 프로세스 마다 자신의 Code , Data , Stack , Heap 영역을 갖고 있습니다. 메모리의 Code , Data , Stack , Heap 영역에 프로세스의 Code , Data , Stack , Heap에 해당하는 내용을 올립니다.(내 생각)
- 메모리 , I/O 장치 , 파일 같은 자원들에 대한 제어와 소유를 담당합니다.
- 프로세스들 간에 자원들에 대한 불필요한 간섭이 일어나는 것을 막기 위해 운영체제는 보호 기능을 수행합니다.

2) 스케줄링 단위

- = 프로세스가 엄청 무거움. -> 메모리 많이 차지
- 프로세스 단위로 switch(A,B)를 하면 너무 힘들다.
- 자원의 소유는 프로세스 단위로 하고 스케줄 단위를 Thread 단위로 하자. 즉 **자원 소유 / 스케줄링을 독립적으로 운영체제가 처리**
- thread는 같은 프로세스 이므로 같은 자원을 쓰지만 , thread는 프로세스를 잘게 쪼갠 것. -> 더 잘게 쪼개지므로 block 되는 단위가 적어짐.

3. Thread

1) Thread 정의

= 한 프로세스 내에 존재하는 실행 흐름.

- 프로세스를 쪼갠 미니 프로세스라고 이해했습니다. 그러므로 가볍습니다.

- Thread는 비동기적으로 처리하기 때문에 다른 Thread와 함께 각각의 작업을 동시에 처리 할 수 있습니다.

- 현대 CPU는 다중 코어로 발전해서 OS는 자원들을 충분히 활용하기 위해 병렬 프로그래밍을 하게 되는데 , 병렬 처리의 핵심도구가 Thread 입니다.

- 코드가 복잡합니다.

- 완벽해보이지만 Thread 간의 자원 경쟁(상호 배제)과 데드락 이라는 문제점이 존재합니다.

3. 스레드 (Thread)

= 하나의 프로세스를 여러 갈래의 작업으로 나누어 동시에 진행한다.

식당에서 조리대 = memory
 요리사 = ~~프로세서~~ CPU (코어)
 요리 = 프로세스
 요리과정 = 스레드

ex) 돈가스 정식 (프로세스) { 돈가스 튀기기 (스레드1)
 장국 끓이기 (스레드2)
 밥 짓기 (스레드3)
 양배추 썰기 (스레드4)

2) Single-Thread vs Multi-Thread

① Single-Thread

= Thread 1개가 프로세스 이므로 Thread 개념이 의미가 없습니다.

- 과제 4번이 리눅스에서 thread를 이용하는 것인데 , 리눅스는 thread를 지원하지 않는다고 합니다. 사실 , 리눅스에서 지원하지 않는 것은 커널 모드에서의 thread이며 , **유저 모드에서의 thread(=pthread = 표준 thread)**는 지원합니다. 과제 4번은 유저 모드의 thread에 관한 내용입니다.
- Windows , Mac은 커널모드에서의 thread도 지원합니다.
- 스케줄링의 단위가 프로세스입니다.

ex) ms-dos , 3.0 이전의 리눅스

② Multi-Thread

= 하나의 프로세스 내에서 여러 개의 thread를 지원하는 기능입니다.

- PCB처럼 thread를 추상화한 구조체인 TCB(Thread Control Block)가 존재합니다.
- 하나의 프로세스를 쪼개서 **독립적인 미니 프로세스**인 Thread로 스케줄링을 운용합니다.
- 스케줄링의 단위가 Thread입니다.
- 1번 thread가 I/o해서 block 된다고 해도 같은 프로세스의 3번 thread는 block 되지 않을 수 있습니다. -> CPU Utilization 극대화
- 문맥 교환의 단위가 Thread이므로 process scheduling보다 경제적입니다.
- 프로세스의 병렬성을 높여 Multi-Processor 활용이 가능합니다.
- 게임 프로그래밍에서 여러 캐릭터들을 동시에 움직이는 기법이 Multi-Thread 방식이며 , 각각의 캐릭터가 하나의 Thread 입니다.

*** Coroutine은 Thread와 항상 비교되는 개념이므로 Unity Study에 필기해 놓았으니 공부합시다. ***

3) Thread의 구성 요소

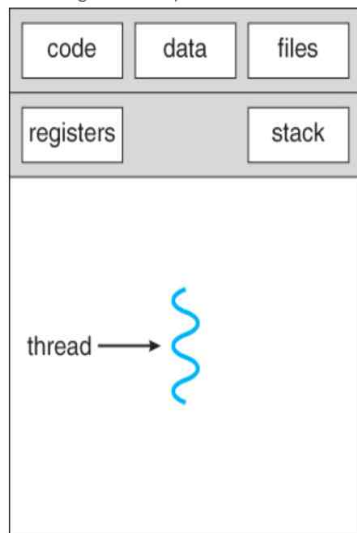
① Thread마다 Thread의 수행상태가 존재해야 합니다.

② Thread마다 PC-Counter 값이 존재해야 합니다.

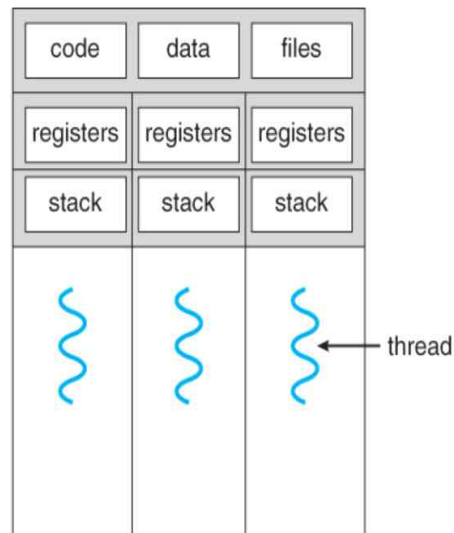
③ Thread마다 Stack이 존재해야 합니다.

- 한 프로세스 내에 존재하는 thread 끼리는 Register와 Stack을 제외한 주소 공간(Code + Data + Heap)은 모두 공유하지만 별도의 register와 stack을 갖습니다. 그 결과, Register와 Stack 영역은 각 Thread에서만 접근이 가능하고, 주소 공간(Code + Data + Heap)과 메모리 및 파일 자원은 모든 Thread에서 접근이 가능합니다.

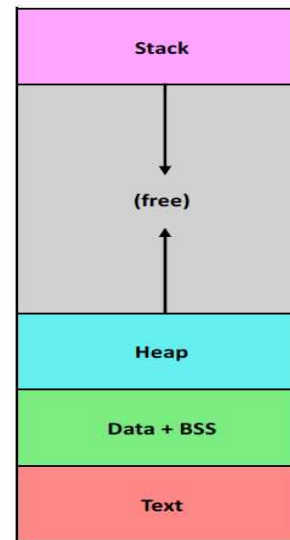
- 메모리(주소 공간) 및 파일 자원은 프로세스 단위로 할당 되므로 한 프로세스 내의 모든 프로세스가 공유합니다.



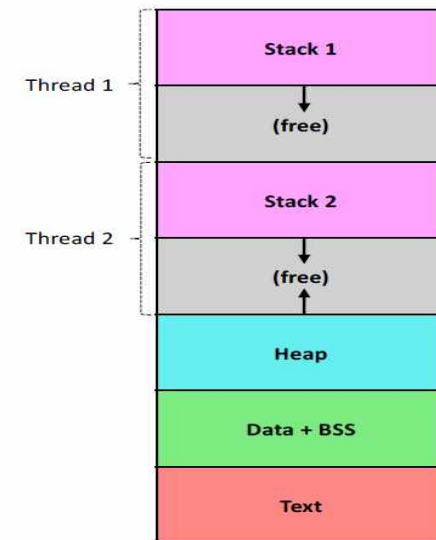
single-threaded process



multithreaded process



Single Thread



Multiple Threads

4) Multi-Thread의 장점

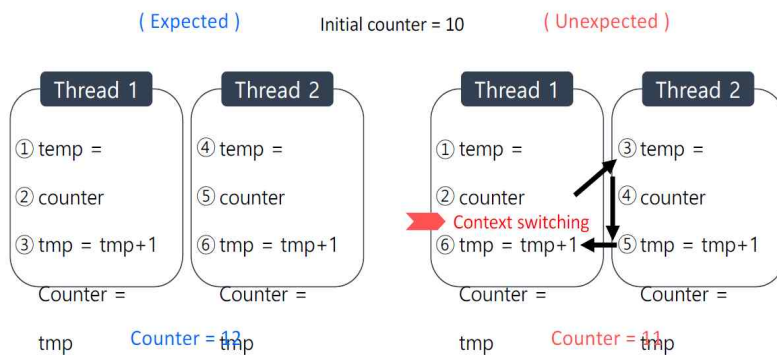
- ① 새로운 프로세스를 생성(fork)하는 시간 보다 , 존재하는 프로세스 내에서 새로운 thread(=pthread)를 생성하는 시간이 10배나 짧습니다.
- ② 프로세스 종료시간 보다 thread 종료시간이 더 짧습니다.
- ③ PCB의 크기보다 TCB의 크기가 훨씬 작기 때문에 같은 프로세스 내에 있는 두 thread 간 교환이 효율적입니다.
- ④ thread는 서로 다른 수행 프로그램 간 통신에서도 효율적입니다.

5) Multi-Thread의 단점

- ① 실행 순서를 파악하기 어렵습니다.

- ② Race Condition(경쟁 조건)

= 다수의 thread가 동시에 자원에 접근하면 결과를 예측할 수 없습니다. 명령어의 실행 순서에 따라 결과가 달라집니다.



- Thread 1이 정상적인 방식으로 수행 되면 Counter의 값은 12가 됩니다. (왼쪽 그림)
- Thread 1을 실행하는 도중에 schedule이 되어서 CPU 사용권을 뺏기면 Thread 2가 실행되고 , Context Switching이 발생하여 counter의 값이 11이 됩니다. (오른쪽 그림)
- counter는 Thread 1과 Thread 2가 공유하는 전역 변수입니다.

6) Lock

= 경쟁 조건으로 인해 결과를 예측하지 못하는 현상을 해결하기 위해 Lock을 사용합니다.

- 병행성은 훌륭한 기술이지만 Race Condition 이라는 문제를 발생시키고 , 그로 인해 실행 결과를 예측하기 어렵습니다. 사용자는 Lock을 걸어서 이를 해결합니다.

① H/W Support

ex) test&set명령어 , exchange명령어 , interrupt disable

② S/W support

<OS에서 지원하는 Concurrency>

- Semaphore
- 메시지 전달 : 두 프로세스가 대화하는 형식입니다. 다시 말해 , 서로 화장실을 가는지 압니다. 잘 사용하지 않습니다.

7) Thread의 상태

= 수행 , 준비 , 블록

- Thread끼리의 상태 변환에서도 문맥 교환이 발생합니다.
- 한 Thread가 블록상태이지만 같은 프로세스 내의 다른 Thread는 준비 상태일 수 있습니다.
- 보류 상태는 프로세스 단위의 개념이므로 보류 상태가 없습니다. 프로세스가 보류상태가 되어버리면 모든 Thread가 보류상태가 되는 것이므로 Thread와 연관시키면 안 됩니다. 다시 말해 한 Thread의 블록이 전체 프로세스를 블록 시키지 않습니다.
- 하나의 Thread가 전체 프로세스에 영향을 미치면 유연성이 사라집니다.

8) Level 별 Thread

① ULT(User-level thread)

= 사용자 수준 Thread

② KLT(Kernel-level thread)

= 커널 수준 Thread

<27강_Thread API + 과제 4번> 10/28 , 11/10

*** 26강 PPT 17쪽 부터의 내용을 합쳤습니다. ***

*** 예시로 과제4에서 직접한 내용을 첨부하였습니다. ***

1. pthread의 정의

= OS가 제공하는 시스템 라이브러리이며 , 시스템 콜이 아닙니다.

- Thread를 제어하기 위해 사용합니다.
- pthread_함수이름()의 형식으로 구성되어 있습니다.

ex) pthread_create()

2. pthread 라이브러리를 사용하기 위한 조건

① #include <pthread.h>를 반드시 선언해줍니다.

② Linux에서 컴파일을 하는 경우 gcc 명령어 맨 끝에 -lpthread를 붙여 주어야 합니다.

파일(F) 편집(E) 보기(V) 검색(S) 터미널(T) 도움말(H)

```
pjw960316@pjw960316-VirtualBox:~$ gcc master-worker master-worker.c -lpthread
pjw960316@pjw960316-VirtualBox:~$ ./master-worker 10000 1000 4 3 > output
pjw960316@pjw960316-VirtualBox:~$ awk -f check.awk MAX=10000 output
OK. All test cases passed!
pjw960316@pjw960316-VirtualBox:~$
```

③ 보통은 main 함수가 주 thread가 됩니다.

3. pthread 함수

1) pthread_create(4)

```
master_thread_id = (int *)malloc(sizeof(int) * num_masters);
master_thread = (pthread_t *)malloc(sizeof(pthread_t) * num_masters);

pthread_create(&master_thread[i], NULL, generate_requests_loop, (void *)&master_thread_id[i]);
```

- Thread를 생성합니다.
- pthread_t 타입의 구조체인 master_thread를 num_masters만큼 생성합니다. master_thread_id는 master_thread의 ID 입니다.

① 1번 매개변수

= 생성할 Thread

② 2번 매개변수

= pthread_attr_t * attr 값이며 , 대부분 NULL을 갖습니다.

③ 3번 매개변수

= 생성한 Thread에서 실행시킬 함수의 주소를 넣습니다.

④ 4번 매개변수

= 3번 매개변수에 해당하는 함수에 넣어줄 인자.

- void* 형 타입으로 선언하여 어떤 타입의 value도 받을 수 있게 합니다.
- Thread의 ID를 인자로 넣어줍니다.

2) pthread_join(2)

```
//wait for master threads to complete
for (i = 0; i < num_masters; i++)
{
    pthread_join(master_thread[i], NULL);
    printf("master %d joined\n", i);
}
```

- 다른 Thread가 종료되는 것을 주 Thread가 기다려줍니다.
- main 함수가 주 thread 이고 , master_thread[i]들이 생성된 thread입니다. master_thread[i]들이 모두 끝나기 전에 주 thread인 main함수가 종료되지 않도록 합니다.
- 시스템 콜의 wait과 동일하다고 생각해도 무방합니다.
- 2번 매개변수의 값이 NULL이면 반드시 detach가 필요합니다.

① 1번 매개변수

- = 어떤 thread를 기다릴 것인지 선언해줍니다.
- main 함수가 master_thread[i]들이 실행되고 , 종료될 때 까지 기다려줍니다.

② 2번 매개변수

- = 반환 값에 대한 포인터

3) pthread_exit()

- = Thread를 종료합니다.

4) pthread_detach(3)

- = 부모 Thread(보통 Main함수)와 분리시킵니다.

5) pthread_mutex(1)

pthread_mutex_t mutex;

```
pthread_mutex_lock(&mutex);
while(1)
{
    if(generate_thread_index != num_masters && generate_cnt >= generate_max)
    {
        break;
    }
    else if(generate_thread_index == num_masters && item_to_produce == limit_value)
    {
        break;
    }

    buffer[curr_buf_size++] = item_to_produce;
    print_produced(item_to_produce, thread_id);
    generate_cnt++;
    item_to_produce++;
}
generate_thread_index++;
pthread_mutex_unlock(&mutex);
```

= Lock과 관련된 pthread입니다.

- pthread_mutex_lock(&mutex)가 정상적으로 작동하려면 , 프로세스 내에서 어떤 thread도 lock을 갖고 있지 않아야 합니다. 쉽게 말하면 , 화장실에 들어가기 위해서는 화장실이 비어있어야 합니다.
- lock ~ unlock 사이의 코드는 임계영역(화장실)으로 잡힙니다.
- 과제4에서 thread를 3개 만들고 thread 모두 위에 코드를 각각의 thread에서 수행하도록 하였습니다.
- 만약 Lock이 없다면 1번 스레드가 while문을 도는 동안 2번 스레드 , 3번 스레드도 while문을 돌게 되고 순서가 엉망이 되어서 올바른 결과가 나오지 못합니다. 다시 말해 , 화장실에 3명이 동시에 들어가는 현상입니다.
- 그러나 Lock/Unlock을 통해 임계영역을 형성하여 , 1번 스레드가 while문(임계영역)을 수행하는 동안은 2번 스레드와 3번 스레드가 while문을 수행하지 못하도록 하여 결과가 올바르게 나오도록 하였습니다.

4. void형 포인터

= 모든 자료형의 주소를 저장할 수 있는 자유로운 포인터 변수입니다.

= 하지만 해당 주소의 값을 참조하거나 출력하는 것은 불가능 합니다. 그러므로 강제형변환을 사용하여 값을 참조하거나 출력하여야 합니다.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6     int *ptr1;
7     long long *ptr2;
8     void *ptr3;
9
10    ptr1 = &a;
11    ptr2 = &a;
12    ptr3 = &a;
13
14    printf("int_hex addr : %p\nlong long addr : %lld\n", ptr1, ptr2); //1. hex 2. long long -> a가 int이므로 long long에서 warning이 나지만 테스트이므로 괜찮습니다.
15    printf("void_hex addr : %p\n", ptr3); //void 타입으로 a의 주소를 hex로 출력합니다. -> 성공!!!
16    printf("int_value : %d\n", *ptr1); //ptr1이 가리키는 a의 virtual memory에 직접 가서 해당 값을 출력.
17
18    //error-code
19    //printf("%d\n", *ptr3); //ptr3이 가리키는 a의 virtual memory에 직접 가서 해당 값을 출력 -> void 포인터 이므로 에러 발생!!!
20
21    //solve-code
22    printf("void_value with 강제형변환 : %d\n", *(int*)ptr3);
23
24    //change value
25    *(int*)ptr3 = 20;
26    printf("void_value with 강제형변환 : %d\n", *(int*)ptr3);
27
28    return 0;
29 }
```

```
User@DESKTOP-BAGIJS2 ~
$ ./a.exe
int_hex addr : 0xffffcc24
long long addr : 4294954020
void_hex addr : 0xffffcc24
int_value : 10
void_value with 강제형변환 : 10
void_value with 강제형변환 : 20
```

<28강_Mutual Exclusion(상호배제)> 10/28 ~ 11/09

1. 상호배제를 이해하기 위한 교수님의 강의

1) 예시를 위한 용어(공공화장실)

*** 여기서 설명하는 화장실을 변기가 있는 칸 1개라고 축약해서 생각하면 더 이해가 쉽습니다. ***



① Thread(Thread) OR Process(프로세스) : 화장실을 이용하는 사람

② 전역변수 : 휴지 , 변기

③ Lock : 화장실 잠금장치입니다. 내가 화장실에 들어갈 때 문을 잠그고(Lock), 일을 마치면 문을 열고 나오는(Unlock) 과정.

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER
2 ...
3 pthread_mutex_lock(&lock);
4 balance = balance + 1;      // Critical Section
5 pthread_mutex_unlock(&lock);
```

- Lock을 pthread_mutex_t 형식의 변수로 만들어서 사용합니다.
- Lock 변수를 pthread_mutex_lock 함수에 넣으면 잠그는 것이고 , pthread_mutex_unlock 함수에 넣으면 잠금을 해제하는 것입니다.
- pthread_mutex_lock 과 pthread_mutex_unlock 사이에 오는 모든 변수들은 다른 프로세스가 사용할 수 없습니다.

④ Critical Section(임계영역) : 화장실

⑤ Mutual Exclusion(상호배제) : 2명이 동시에 화장실에 들어갈 수 없습니다.

⑥ Race Condition(경쟁조건) : 추석의 휴게소를 생각해 봅시다. 화장실에 많은 인원들이 들어가려고 경쟁 합니다.

⑦ Deadlock(교착상태) : 두 명이 화장실까지 달려가는데 둘 다 동시에 문고리를 잡아서 둘 다 들어가지 못합니다.

2) Lock의 범위

① Coarse-grained lock

= 큰 범위(긴 범위의 코드)에 lock을 겁니다.

- 화장실 전체에 Lock을 걸어버리면 손만 씻으러 오는 애들은 화장실을 사용하지 못하는 단점이 생깁니다.

② Fine-grained lock

= 작은 범위(작은 범위의 코드)에 lock을 겁니다.

- 화장실을 양변기 칸 , 세면대 , 소변기로 구분해서 Lock을 걸면 많은 사람들이 화장실을 각각의 용도로 사용할 수 있지만 , 코드에 Lock 관련 코드가 엄청 나게 증가하여 오버헤드가 커집니다.

3) 예시로 설명하신 상호배제를 위한 요구 조건 6가지

① 상호배제가 강제되어야 합니다.

= 화장실에 2명이 동시에 들어가는 것은 있을 수 없습니다.

② 임계영역이 아닌 곳에서 수행이 멈춘 프로세스는 다른 Thread의 수행을 간섭해서는 안 됩니다.

= 화장실 밖에 있는 사람들은 밖에서 빨리 나오라고 문을 발로차면 안됩니다.

③ 임계영역에 접근하고자 하는 프로세스의 수행이 무한히 미루어져서는 안 됩니다.

= 모든 사람들은 화장실에 들어갈 수 있어야 합니다.

= 교착상태와 기아현상이 발생하면 안 됩니다.

④ 임계영역이 비어있을 때 , 임계영역에 진입하려고 하는 프로세스는 즉시 임계영역에 들어갈 수 있어야 합니다.

= 화장실이 비어 있음에도 불구하고 , 사람들이 화장실에 누군가 있다고 착각하여 아무도 들어가지 않는 현상이 발생하면 안 됩니다.

⑤ CPU의 개수나 프로세스의 상대적 수행 속도에 대한 가정은 없어야 합니다.

= 3대 500KG의 형님이 새치기를 하는 경우는 존재하지 않습니다.

⑥ 임계영역에 들어간 프로세스는 유한 시간 내에 임계영역에서 나와야 합니다.

= 화장실에 들어간 사람은 반드시 일정 시간 이내에 화장실에서 볼일을 마치고 나와야 합니다.

4) 해결책과 문제점(S/W)

*** 전제 조건 ***

- 공유데이터를 사용하기 위해 프로세스 n개가 경쟁하고 있습니다.
- 한 프로세스가 임계 영역에서 실행되면 다른 프로세스는 임계 영역에서의 실행이 불가능함을 보장합니다.
- 실제로는 프로세스가 무수히 많겠지만 , 설명을 위해 모든 예제는 프로세스가 2개 존재한다고 가정하겠습니다.

① 1번

P0 while (turn != 0) do {nothing }; <i>critical section</i> ; turn = 1;	P1 while (turn != 1) do {nothing }; <i>critical section</i> ; turn = 0;
--	--

<Solution>

= Turn(순서) 변수를 사용합니다.

- P0은 turn이 0이 아니면 임계영역에 들어가지 않습니다. 0이 되면 임계영역에 들어가서 turn을 1로 바꿉니다.
- P1은 turn이 1이 아니면 임계영역에 들어가지 않습니다. 1이 되면 임계영역에 들어가서 turn을 0으로 바꿉니다.

<Problem>

- P0이 임계영역에 들어가지 않으면 P1은 평생 임계영역으로 들어가지 못합니다.
- P0이 나오고 다시 임계영역으로 들어가고 싶어도 P0은 들어갈 수 없습니다.
- P1은 임계영역에 들어가고 싶지 않아도 들어가야 합니다.

② 2번

P0 while flag[1] do {nothing } ; flag[0] = true; <i>critical section</i> ; flag[0] = false;	P1 while flag[0] do {nothing } ; flag[1] = true; <i>critical section</i> ; flag[1] = false;
---	---

<Solution>

= Flag(프로세스마다의 T/F를 저장하는 배열)을 사용합니다.

- 자신을 제외한 모든 프로세스의 플래그가 F인지를 검사합니다. 그들이 모두 F라면 자신의 플래그 T로 바꾸고 임계영역에 진입합니다. 임계영역에서 나올 때 자신의 플래그를 F로 바꿉니다.
- 쉽게 생각하면 , 화장실에 들어갈 때 자신 말고 나머지 사람들이 화장실에 있는지 검사를 하고 , 들어갑니다. 그리고 들어가면 자신이 화장실에 들어갔다는 것을 알리고 , 나오면 화장실에 나왔음을 알립니다.

<Problem>

- P0에서 while문이 끝났을 때 CPU 사용권한을 뺏겨서 P1에게 주고 P1이 While문을 합니다. P1에서 while문이 끝났을 때 CPU 사용권한을 뺏겨서 다시 P0에게 주고 P0이 자신의 flag를 true로 하고 CPU 사용권한을 뺏깁니다. P1이 자신의 flag를 true로 합니다. 그 결과 두 프로세스가 동시에 임계영역에 접근하려 하는 현상이 발생하여 , 둘 다 들어가지 못하게 됩니다.
- 위의 현상이 계속 반복되어서 아무도 들어가지 못하므로 Busy-Waiting이 발생합니다.
- **Busy Waiting** : 프로세스가 임계 영역에 들어가기 위한 허가를 획득할 때 까지 변수를 테스트하는 명령을 반복 실행

③ 3번

P0 flag[0] = true; while flag[1] do {nothing } ; <i>critical section</i> ; flag[0] = false;	P1 flag[1] = true; while flag[0] do {nothing } ; <i>critical section</i> ; flag[1] = false;
---	---

<Solution>

- 2번 해결책을 조금 변형했습니다.
- 일단 자신의 플래그를 T로 바꾸고 다른 프로세스의 플래그가 F인지 검사합니다. 모두 F라면 임계영역으로 진입하고 나올 때 자신의 플래그를 F로 바꿉니다.

<Problem>

- 임계 영역에 아무도 들어가지 않았는데 프로세스들은 안에 어떤 프로세스가 존재한다고 착각하여 들어가지 못하는 DeadLock이 발생합니다.

④ 4번

P0 flag[0] = true; while (flag[1]) { flag[0] = false; delay flag[0] = true; } <i>critical section</i> ; flag[0] = false;	P1 flag[1] = true; while (flag[0]) { flag[1] = false; delay flag[1] = true; } <i>critical section</i> ; flag[1] = false;
---	---

<Solution>

- 두개 이상의 프로세스의 flag가 T면 서로 들어가지 못하므로 일정 시간마다 프로세스의 Flag를 T<->F로 계속 변경해줍니다.
- 그러다보면 운이 좋게 Flag가 T인 것이 1개인 순간이 발생하게 되고 , 그 때 Flag가 T인 프로세스가 임계영역에 들어갑니다.
- 임계영역에서 나오면 당연히 자신의 flag를 F로 바꿉니다.
- Flag가 계속 변경되므로 DeadLock은 발생하지 않습니다.
- LiveLock이 발생하지만 4개의 해결책 중에 가장 괜찮은 해결책 중입니다.

<Problem>

- P0과 P1의 delay시간이 같은 리듬에 맞춰서 동일하게 되면 둘 다 F일 때 delay가 발생하면 둘 다 T가 됩니다. 그러므로 의미 Flag 변경이 반복되는 상황을 LiveLock이라고 합니다.
- 결론적으로 S/W로만의 해결책은 모든 문제점을 완벽하게 해결할 수 없습니다.

5) 해결책과 문제점(H/W)

① 인터럽트 금지

= 결국 스케줄이 예측할 수 없는 순간에 발생하므로 생기는 문제야.

= 임계 영역에 들어가면 인터럽트 발생시키지 않음

<문제>

= 인터럽트 금지자체가 부하가 큼

= 이론적으로는 가능하지만 현실적으로는 결국 말도 안되는 방법

② 특별한 기계 명령어

= test & set : 남의 플래그를 보고 나의 플래그를 변화시키는 걸 두개의 명령어를 하나의 명령어로 합쳐서 사용하는 거.

= 하나의 명령어 사이에서는 스케줄이 발생하지 않으니까.

2. 상호배제를 이해하기 위한 교재의 내용

만약 다른 스레드가 `lock()` 을 호출한다면(이 예제에서는 `mutex`에 해당) 사용 중인 동안에는 `lock()` 함수가 리턴하지 않는다. 이와 같은 방식으로 락을 보유한 스레드가 임계 영역에 진입한 상태에는 다른 스레드들이 임계 영역 안으로 진입할 수가 없다.

락 소유자가 `unlock()` 을 호출한다면 락은 이제 다시 사용 가능한 상태가 된다. 어떤 스레드도 이 락을 대기하고 있지 않았다면(어떤 스레드도 `lock()` 을 호출하여 멈춰 있던 상태가 아니라면) 락의 상태는 사용 가능으로 유지된다. 만약에 대기 중이던 스레드가 있었다면(`lock()` 으로 인해 멈춘), 락의 상태가 변경되었다는 것을 (결국엔) 인지하고 (또는 정보를 받아서) 락을 획득하여 임계 영역 내로 진입하게 된다.

락은 프로그래머에게 스케줄링에 대한 최소한의 제어권을 제공한다. 일반적으로 스레드는 프로그래머가 생성하고 운영체제가 제어한다. 락은 스레드에 대한 제어권을 일부 이양 받을 수 있도록 해준다. 락으로 코드를 감싸서 프로그래머는 그 코드 내에서는 하나의 스레드만 동작하도록 보장할 수 있다. 락을 통해 프로세스들의 혼란스런 실행 순서에 어느 정도를 질서를 부여할 수 있다.

<31강_Semaphore> 11/10

1. Semaphore Overview

= 정수 값을 갖는 변수

- Lock과 Condition_변수로 사용할 수 있습니다.
- 두 개 이상의 프로세스들은 간단한 형태의 신호를 이용해 협력합니다.
- Semaphore 변수를 다루는 함수 : semSignal(s) & semWait(s)
- Semaphore 변수를 s로 나타냅니다.

2. Semaphore 중요 용어

1) semSignal(s) = signal = V

= S값을 1 증가

2) semWait(s) = wait = P

= S값을 1 감소

3) 범용 Semaphore

= Semaphore가 정수 값

4) 이진 Semaphore

= Semaphore가 0 OR 1만 갖습니다.

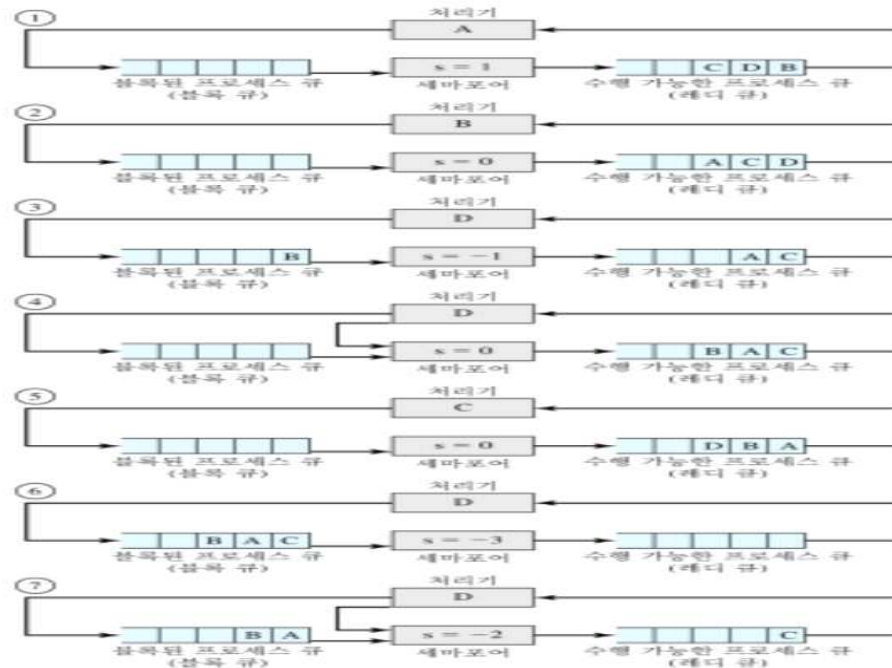
= Mutex 또는 Binary Semaphore로 표기합니다.

- 문제에서 'mutex s' 라고 표기가 되어 있으면 무조건 이진 Semaphore입니다.
- 구현이 간단하지만 예러가 많습니다.

<강성 vs 약성>

① 강성 Semaphore

- = FIFO 방식으로 대기 큐에 가장 오래있던 프로세스부터 Pop합니다.
- 현대에서는 모두 강성 Semaphore를 사용합니다.



② 약성 Semaphore

- = 대기 큐에서 Pop하는 순서를 특별하게 명시하지 않습니다.
- 거의 사용하지 않습니다.

생산자와 소비자 문제

Semaphore 변수 : $s = 0$, 일반 변수 : $n = 0$

생산자가 생산하면 semsignal만 함.

- ABC = 소비자 , D는 생산자

① A가 소비

2 이제 B가 소비하려고 봤는데 $s = 0$ 이라 , 블록됨 $\rightarrow s = -1$

3 D가 들어옴 생산자니까 s 를 0으로 함. B를 레디로 올려줌.

4 B가 레디로 오라옴

5 C를 처리하는데 , B도 오고 A도 오면 다 소비자니까 -3됨

- 다시 보면 이해할 수 있을 듯?

3. 화장실 문제_1 : 범용 Semaphore

1) S 값

- S값은 기본적으로 화장실의 개수 OR 대기 큐에 대기하고 있는 프로세스의 개수
- 문제에서 초기 S값은 언제나 0보다 커야합니다. 왜냐하면 $S > 0$ 이 아니면 화장실의 개수가 0 OR 음수 인데 이런 경우 화장실문제가 아니게 됩니다.

① 양수 : 화장실의 개수

② 문제 진행과정 도중에 0 : 화장실의 개수와 관련이 없고 대기 큐에 대기하고 있는 프로세스가 0개

③ 음수 : 대기 큐에 대기하고 있는 프로세스의 개수

2) Example

- 화장실이 2개. $s = 2$
- p0가 화장실_1에 들어가면서 wait을 외쳐서 $s = 1$ 이 됨.
- P1 , P2 , P3이 화장실_2에 들어가고 싶어 합니다. p1이 화장실_2에 들어가면서 $s = 0$ 되고 p2 , p3 기다리니까 -2가 됩니다.
- '-2'는 2명이 대기하고 있다는 의미이며 , 대기 큐에는 p2 , p3이 대기하고 있습니다.
- p0는 아직 화장실_1에 있지만 p1은 화장실_2에서 일찍 나오면서 ++해서 $s = -1$ 이 됩니다.
- '-1'이면 대기 큐에 있는 놈이 1명이 됩니다.
- wait이랑 signal이 100% 짝을 이루게 됩니다.

4. 화장실 문제_2 : 이진 Semaphore

- Semaphore가 0 OR 1만 갖기 때문에 제한이 발생합니다.
- 구현이 범용 Semaphore 보다 훨씬 쉽습니다.

1) wait

- 만일 값이 0 이면 -를 할 수 없으므로 0을 그대로 유지하고 , 이걸 호출한 프로세스를 대기 큐에 넣습니다.
- 만약 1이면 0으로 하고 , 호출한 녀를 화장실로 넣습니다.
- 대기 큐에 몇 개의 프로세스가 대기하고 있는지 알지 못합니다.
- 대기 큐에 여러 개의 프로세스가 대기 하고 있는데 어떤 프로세스부터 화장실로 넣을까? : 강성 Semaphore OR 약성 Semaphore를 사용합니다.

2) signal

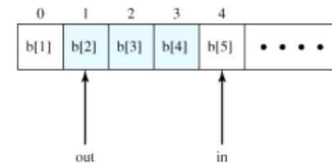
- S값부터 보지 않고 먼저 대기 큐를 봅니다. 대기 큐에 P1 , P2가 있을 때 P1을 화장실에 넣어도 아직 0, P2까지 넣어서 대기 큐가 비어있으면 1로 만듭니다.

5. 생산자/소비자 문제

= 한 쪽은 데이터를 소비해서 버퍼에서 소비하고 , 한 쪽은 데이터를 생산해서 버퍼에 저장

<Example : 파리바게트>

<pre> producer: while (true) { /* 데이터 v를 생산 */; b[in] = v; in++; } </pre>	<pre> consumer: while(true) { while (in <= out) /* 대기 */; w = b[out]; out++; /* 데이터 w 소비*/ } </pre>
---	--



주의: 음영 부분은 버퍼에서 유효한 데이터가 차지한 공간을 나타낸다.

- buffer : 빵 진열대(배열 화장실이라 생각해도 됩니다.)
- 소비자 : 제빵사 이며 , out을 합니다.
- 생산자 : 손님 이며 , in을 합니다.
- 버퍼에 동시에 접근 불가 : 제빵사가 빵을 진열하는 도중에 손님이 빵을 구매할 수 없습니다. 다시 말해 , 진열대에서는 제빵사와 손님이 동시에 존재할 수 없습니다.
- $b[in] = v \ \& \ in++$: 생산자는 in에 해당하는 진열대에 빵을 놓고 , in 값을 증가 시켜 다음 번호의 진열대로 이동합니다.
- $b[out] = out++$: 소비자는 out에 해당하는 진열대에 존재하는 빵을 구매하고 , out 값을 증가 시켜 다음 번호의 진열대로 이동합니다.
- 언제나 $in > out$ 이어야 합니다.
- In과 out이 같으면 생산자와 소비자가 같은 진열대에 동시에 있는 것이므로 절대 있으면 안 되는 상황입니다.
- Out이 in보다 크면 아직 생산자가 빵을 만들지도 않은 진열대 칸으로 이동해서 빵을 구매하는 무의미한 행동입니다. 그러므로 소비자는 대기하게 됩니다. 그러므로 생산한 후에 소비하는 것이 균등하게 이루어져야 합니다. 소비만 한다면 무한대기가 발생합니다.

<알고리즘>

$s = 1$ 이네 -> 화장실 문제인가?

delay = 0이네 -> 뭐지?

wait 이랑 signal이 짝을 이루지 않네 -> 생산자/소비자 문제구나.

append() = 제빵사가 빵 진열

semwaitB(s) = 버퍼를 사용할 수 있나 확인 , 확인했을 때 들어갈 수 있으면 진열대로 가서 할 일 함. 소비자부터 진열대(버퍼)를 접근할 수 있으면

빵도 없는데 헛짓함. 그래서 아래의 delay는 생산자부터 버퍼에 접근할 수 있게 순서를 조정함

semSignalB(delay) , semWaitB(delay) = 생산자부터 들어가도록 순서(중요)를 조정하는 함수 2개 = 제빵사부터 버퍼에 들어갈 수 있게 보장.

semsignalB(s)와 semwaitB(s)는 짝을 이룹니다. -> 화장실

소비자가 먼저 실행은 될 수 있어 , 근데 그래봤자 블록됨.

n++은 빵 개수 증가

take() = 손님이 빵 가져감.

binary semaphore s = 결국 뭐에 대한 공유변수인가? 버퍼에 대한 공유변수다.

//뇌피셜 : 진열대 3번에 제빵사가 빵 놓고 있어도 손님은 3번 ~ 는 당연히 접근할 수 없지만 0~ 2번에도 접근할 수 없다? -> 거의 맞는 듯?

<잘못된 프로그램>

```

/* program producerconsumer */
int n;
binary_semaphore s = 1;
binary_semaphore delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n == 1)
            semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        semSignalB(s);
        consume();
        if (n == 0)
            semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin(producer, consumer);
}

```

	생산자	소비자	s	n	delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	if (n == 1) semSignalB(delay)		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	if (n == 1) semSignalB(delay)		0	1	1
13	semSignalB(s)		1	1	1
14		if (n == 0) semWaitB(delay)	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		if (n == 0) semWaitB(delay)	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	-1	0
21		semSignalB(s)	1	-1	0

- n : in - out , 0보다 큰 정수를 갖는게 정상입니다.
- s 와 delay는 둘 다 이진 semaphore 변수이므로 0 OR 1입니다.
- 초기의 s는 1 , n은 1 , delay는 0
- 생산자가 semWait(s)을 하면 buffer를 사용하고 , buffer에 빵을 진열합니다. in과 out의 차이가 1이되므로 semSignal(delay)를 통해 delay를 1로 만듭니다. semSignal(s)로 s를 1로
- delay 값이 1이되면 소비할 수 있는거 같음

-14번부터 표는 진행하지만 사실 n의 값이 잘못되어서 진행이 되지 않습니다.

1 ~ 5 는 첫 producer() , 6 ~ 9 는 첫 consumer()

1) 1~5번 과정

= 생산하고 , s를 0으로 만들고 , append해서 n을 1로 만들고 , delay 해서 delay는 1되고 semSignal(s)로 s도 1

2) 6~9번 과정

- 문제점 발생 상황 : 원래는 consumer()전체를 수행해야 하는데 consume()까지만 수행하고 거기서 스케줄링이 끝나는 상황이 되면?

- delay가 1이 되어야만 wait을 통해 delay가 0이 됨 , wait으로 s가 1에서 0됨 , n이 1에서 --되서 0됨 , signal해서 s가 1됨 근데 여기서 갑자기 스케줄링 발생 -> 다시 producer()로 돌아감

3) 10~13번 과정

- 생산하고 swait으로 s가 0됨 , n이 다시 1되고 , delay가 1이됨.

4) 14번 ~ 21번 문제발생

- consumer()는 처음부터 하지 않고 아까 하던데서 함 if(n==0)에서 함

- n은1이므로 n은0이 아니므로 다시 producer감 , n은2가됨 , 또 n은0이 아니므로 wait을 진행하지 못함.

5) 왜 이런 오류가 발생하냐?

= 제빵사가 만들고 , 소비자가 구매하고 이게 반복되면 괜찮은데 , 제빵사가 만들고 , 소비자가 구매하고 가려는 순간에 스케줄링이 바뀌면 올바르게 수행이 안됩니다.

<이진 세마포어로 했을 때 발생하는 문제 해결>

1. 지역변수 m을 추가

- m으로 if문을 검사하도록 바꿉니다.

2. 범용 Semaphore로 문제를 해결합니다.

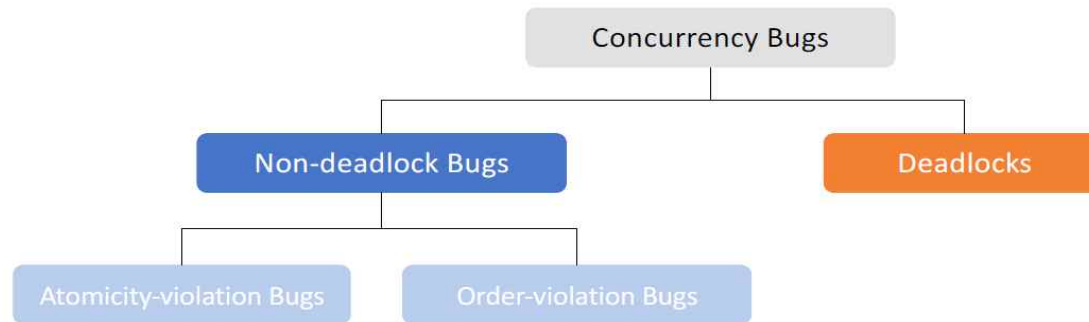
- Delay를 사용하지 않습니다.

- signal(s)와 wait(s)는 화장실을 들어가고 나옴

- signal(n)과 wait(n)은 제빵사와 소비자의 순서

<32강_Common Concurrency Problems : Deadlock> 11/11 ~ 11/23

- 대부분의 OS 수업에서 deadlock을 중요하게 다루지만 실제 OS에서는 구현하지 않습니다. 구현을 하게 되면 OS의 성능이 매우 떨어지게 됩니다.



1. Concurrency Bugs : Non-deadlock Bugs

= 프로그래머의 부주의로 발생하는 버그입니다. 오히려 deadlock bugs보다 많이 발생합니다.

1) Atomicity-violation Bugs

= 순서대로 데이터에 접근해야 하지만 그렇지 못할 때 발생하는 버그

- 상호배제의 확장성 문제와 동일하게 보아도 됩니다. 확장성에 2명 이상 들어가면 atomicity가 깨집니다.
- 프로그래머가 lock/unlock을 제대로 사용하지 못하면 버그가 발생합니다.

2) Order-violation Bugs

= Process_A가 항상 Process_B 이전에 수행되어야 하지만, 그렇지 못할 때 발생하는 버그.

- 상호배제의 생산자/소비자 문제
- Condition Variables를 사용하여 해결합니다.

2. Concurrency Bugs : Deadlocks

1) Deadlock의 발생 조건(3+1)

① 상호배제

= 임계영역에는 하나의 프로세스만이 점유할 수 있으며 다른 프로세스는 간섭할 수 없는 현상.

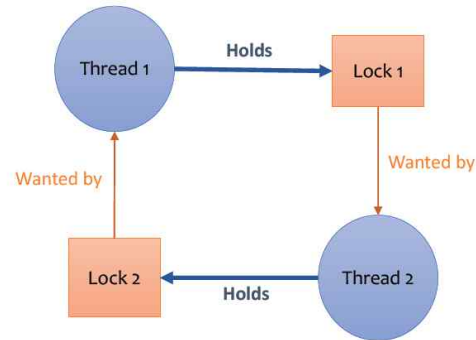
② 점유 대기(Hold and Wait)

= 자원_A를 갖고 있는 프로세스가 자원_B도 얻으려고 기다리고 있는 현상.

③ Non-preemption

= 두 프로세스의 우선순위가 같아서 한 프로세스가 다른 프로세스의 자원을 빼앗지 못하는 현상.

④ 환형 대기



= 자원 할당 구조에서 환형(Loop)이 발생합니다.

- Deadlock의 정의가 바로 해결할 수 없는 환형대기 상태입니다.

2) Deadlock 발생조건의 명제

*** 1)을 참고하면서 공부합시다. ***

<필요조건 , 충분조건>

명제 $p \rightarrow q$ 가 참일 때,

p 는 q 이기 위한 충분조건

q 는 p 이기 위한 필요조건

내가 인간이라면 나는 포유류이다.

인간임은 포유류이기 위한 충분조건 (인간이기만 하면 포유류이기에 충분)

포유류임은 인간이기 위한 필요조건 (인간이려면 우선 포유류인 것이 필요함)

- ① ~ ③ 은 각각 Deadlock의 필요조건이고 , ④는 Deadlock의 필요충분조건입니다.
- ①을 만족하면 Deadlock이 발생했을 수도 있고 발생하지 않았을 수도 있습니다.
- ②를 만족하면 Deadlock이 발생했을 수도 있고 발생하지 않았을 수도 있습니다.
- ③을 만족하면 Deadlock이 발생했을 수도 있고 발생하지 않았을 수도 있습니다.
- ④를 만족하면 Deadlock은 이미 발생한 것을 의미합니다. 다시 말해 , **환형 대기**가 만들어지면 **deadlock**이 발생한 것이고 , **deadlock**이 발생했으면 **환형대기**가 만들어진 것입니다.

<교재 자료>

상호 배제 (Mutual Exclusion): 쓰레드가 자신이 필요로 하는 자원에 대한 독자적인 제어권을 주장한다(예, 쓰레드가 락을 획득함).

점유 및 대기 (Hold-and-wait): 쓰레드가 자신에게 할당된 자원(예: 이미 획득한 락)을 점유한 채로 다른 자원(예: 획득하고자 하는 락)을 대기한다.

비 선점 (No preemption): 자원(락)을 점유하고 있는 쓰레드로부터 자원을 강제적으로 빼앗을 수 없다.

환형 대기 (Circular wait): 각 쓰레드는 다음 쓰레드가 요청한 하나 또는 그 이상의 자원(락)을 갖고 있는 쓰레드들의 순환 고리가 있다.

3) 자원 할당 그래프(RAG)

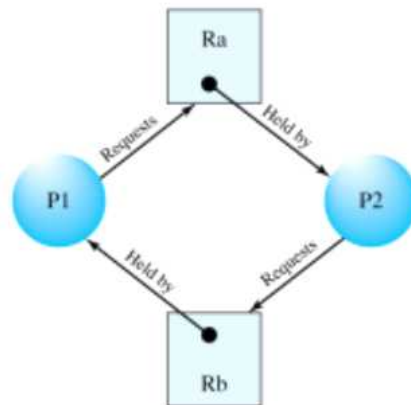
*** 자원 할당 그래프의 정점과 화살표를 그리는 간단한 문제가 기말고사에 100% 나옵니다. ***



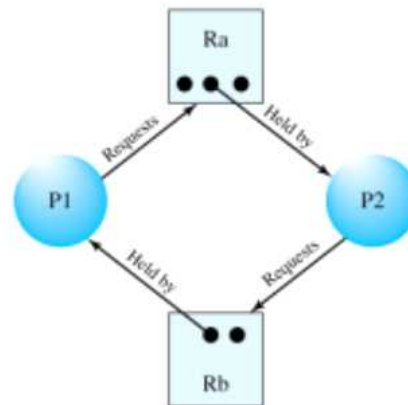
(a) 프로세스가 자원을 요청하고 있다.



(b) 자원이 프로세스에게 할당되었다.



(c) 환형 대기

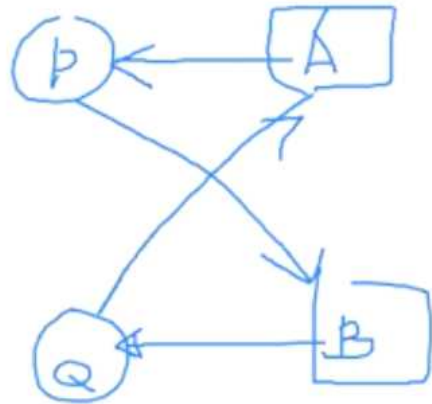


(d) 교착상태가 아닌 경우

- 자원은 보통 메모리를 의미합니다.
- (a) : 프로세스_P1이 자원_Ra를 요구합니다.
- (b) : 자원_Ra가 프로세스_P1에 할당되었습니다.
- (c) : 해결할 수 없는 환형 대기가 만들어져서 Deadlock이 발생하였습니다.
- (d) : 자원_Ra와 자원_Rb가 모두 자원이 풍부하기 때문에 Deadlock이 발생하지 않았습니다.

4) Deadlock의 진행

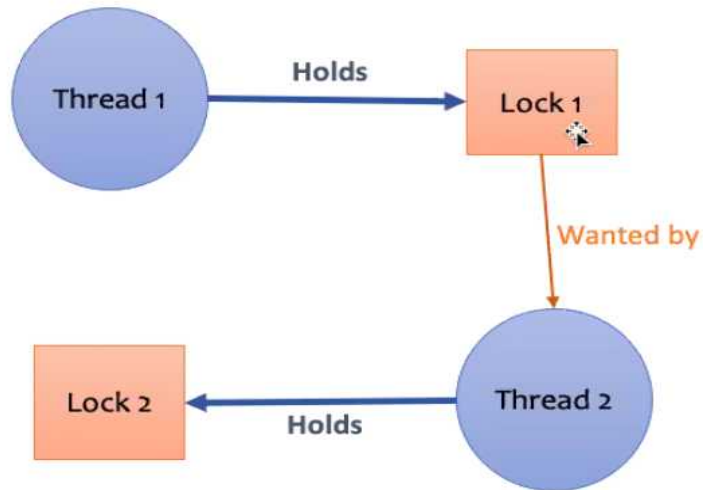
*** 다시 듣기 ***



- 프로세스_P , 프로세스_Q , 자원_A , 자원_B로 설명하겠습니다.
- Get : 프로세스 얻음
- Release : 프로세스 요구.
- P는 A를 가집니다. 그리고 Q는 B를 가집니다. 그리고 P는 B를 요구하고 Q는 A를 요구합니다.

3. Deadlock 해결 방법

1) Prevention(예방)



- 좌측의 그림처럼 환경 대기가 곧 발생할 것 같으면 OS가 미리 발생을 차단시켜 버립니다.
- 상호배제 , 점유대기 , non-preemption도 허용하지 않습니다. 완전히 보수적이어서 모든 원칙도 차단시켜 버립니다.
- OS는 thread가 어떤 자원을 사용할지 알 수 없으므로 말이 되지 않는 해결방법입니다.
- 결론적으로 'Prevention'은 말이 되지 않는 해결 방법입니다.

2) Avoidance(회피)

= 프로세스가 어떤 자원을 얻으려고 할 때 마다 OS는 교착상태가 발생할 가능성이 있는지 조사하고 , 발생할 가능성이 있다면 자원을 할당하지 않거나 프로세스를 시작시키지 않습니다.

- 상호배제 , 점유대기 , non-preemption은 허용합니다. 현실적으로 이 3가지는 무조건 허용해야합니다.
- Prevention과 구분하기가 매우 어렵습니다. 교수님은 avoidance에 더 중점을 두고 설명하셨습니다.
- OS가 현재 자원의 가용 개수와 프로세스의 자원 요구량을 미리 알고 있어야 합니다. 프로세스가 어떤 자원을 얼마나 사용할지 알 수 없으므로 말이 되지 않는 해결방법입니다.
- 시스템의 성능 저하를 유발하는 문제점이 많아서 구현을 하면 안 되는 방법이고 현대 OS도 구현하지 않습니다. (=구현 불가능)

① 프로세스 시작 거부

자원 = $R = (R_1, R_2, \dots, R_m)$	시스템에 존재하는 자원의 전체 개수
가용 = $V = (V_1, V_2, \dots, V_m)$	시스템에 존재하는 자원 중 현재 사용가능한 자원의 개수
요구 = $C = \begin{bmatrix} C_{11} & C_{12} & \dots & C_{1m} \\ C_{21} & C_{22} & \dots & C_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \dots & C_{nm} \end{bmatrix}$	C_{ij} 는 프로세스 i 가 자원 j 를 요구하고 있음을 의미
할당 = $A = \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1m} \\ A_{21} & A_{22} & \dots & A_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ A_{n1} & A_{n2} & \dots & A_{nm} \end{bmatrix}$	A_{ij} 는 프로세스 i 가 자원 j 를 할당받아 점유하고 있음을 의미

- $R = \{3, 2, 5, 1\}$: 자원_1은 3개 , 자원_2은 2개 , 자원_3는 5개 , 자원_4은 1개
- $V = \{3, 2, 5, 1\}$: (사용가능)자원_1은 3개 , (사용가능)자원_2는 2개 , (사용가능)자원_3은 5개 , (사용가능)자원_4는 1개
- $R \geq V$
- C21 : 2번 프로세스가 자원1을 요구하고 있습니다.
- A32 : 3번 프로세스가 자원2를 점유하고 있습니다.
- 할당은 **현재** 프로세스가 점유하고 있는 자원의 개수

$$R_i = V_i + \sum_j A_{ij}$$

= 모든 자원의 합 = 가용 자원의 합 + 할당된 자원의 합

$$C_{ij} \leq R_i$$

= 시스템에 존재하는 자원 보다 요구하는 자원이 많을 수 없습니다.

ex) $R = \{2, 4, 3, 1\}$ / C의 특정 행 = $\{3, 4, 4, 3\}$ -> 자원_1이 2개 있는데 자원_1을 3개 요구할 수 없습니다.

$$A_{ij} \leq C_{ij}$$

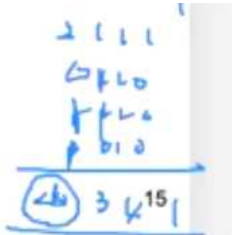
= 요구한 자원 보다 할당된 자원이 많을 수 없습니다.

= C의 특정 행 $\{3, 4, 3, 2\}$ / A의 같은 행 = $\{4, 4, 3, 2\}$ -> 자원_1을 3개 요구했는데 자원_1을 4개 할당 할 수 없습니다.

- C에서 열을 모두 더해서 만든 행이 R보다 크면 deadlock 발생합니다. (같아도 됩니다.) 그러므로 C에서 열을 모두 더해서 만든 행이 R보다 작거나 같을 때만 새로운 프로세스의 시작을 허가 합니다.

- 새로운 프로세스가 요구할 때 마다 검사하고 만약 R보다 크다면 프로세스의 시작을 거부합니다. 하지만 다른 프로세스의 자원 할당이 종료 되어 가용 자원이 늘어났을 때의 검사결과가 R보다 작거나 같으면 프로세스의 시작을 허가합니다.

<C에서 열을 모두 더해서 만든 행>



- 모든 프로세스들이 동시에 최대 자원을 요구하면 새로운 프로세스의 시작이 되지 않을 것이고 , 매 번 프로세스들의 자원 관련 내용을 검사하는 것은 매우 큰 과부하를 발생시킵니다.

② 자원 할당 거부(Banker's Algorithm)

- 시스템에 고정된 수의 프로세스와 자원이 존재한다고 가정합니다.
- 은행원 알고리즘
- 안전하면 돈(자원)을 빌려주고 , 불안전하면 돈을 빌려주지 않습니다.
- 안전 상태 : 자원을 할당할 수 있는 진행 경로(중요)가 존재함을 의미합니다.

<Example_1 : Deadlock이 발생하지 않음>

	R1	R2	R3
P1	3	2	2
P2	0	1	3
P3	3	1	4
P4	4	2	2
Claim matrix C			
	R1	R2	R3
P1	1	0	0
P2	6	1	2
P3	2	1	1
P4	0	0	2
Allocation matrix A			
	R1	R2	R3
P1	2	2	2
P2	0	0	1
P3	1	0	3
P4	4	2	0
C - A			

	R1	R2	R3
Resource vector R	9	3	6

	R1	R2	R3
Available vector V	0	1	1

(a) Initial state

	R1	R2	R3
P1	3	2	2
P2	0	0	0
P3	3	1	4
P4	4	2	2
Claim matrix C			
	R1	R2	R3
P1	1	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2
Allocation matrix A			
	R1	R2	R3
P1	2	2	2
P2	0	0	0
P3	1	0	3
P4	4	2	0
C - A			

	R1	R2	R3
Resource vector R	9	3	6

	R1	R2	R3
Available vector V	6	2	3

(b) P2 runs to completion

	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	3	1	4
P4	4	2	2
Claim matrix C			
	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	2	1	1
P4	0	0	2
Allocation matrix A			
	R1	R2	R3
P1	0	0	0
P2	0	0	0
P3	1	0	3
P4	4	2	0
C - A			

	R1	R2	R3
Resource vector R	9	3	6

	R1	R2	R3
Available vector V	7	2	3

(c) P1 runs to completion

① (a)

= 초기상태

- (C - A) : 프로세스 마다 얼마만큼의 자원을 더 할당받으면 프로세스의 모든 일을 수행하고 자원을 반납할 수 있음.

- Available Vector V : OS가 할당해 줄 수 있는 자원의 총량. 현재 (0,1,1)이므로 C-A를 확인해 보면 P2는 (0,1,1)보다 작거나 같기 때문에 할당 해줄 수 있습니다.

② (b) : p2가 일을 마치고 모든 자원을 반납한 시점.

- P2는 OS로 부터 (0,0,1) 만큼 할당받고 자신의 일을 모두 수행합니다. 그 후 , 자신이 가지고 있던 모든 자원을 OS에게 반환합니다.

- P2의 할당 자원은 (6,1,2) 상태에서 (0,0,1)를 할당 받아서 (6,1,3)이 되고 , Available vector V는 (0,1,1)에서 P2에게 (0,0,1)을 주어서 (0,1,0)이 됩니다. 그러므로 P2에게 자원을 돌려받으면 Available vector V는 (0,1,0) + (6,1,3) = (6,2,3)이 됩니다.

③ (c)

- (6,2,3)으로 C-A에서 줄 수 있는 프로세스를 찾습니다. 모든 프로세스에게 줄 수 있지만 책에서는 p1에게 프로세스를 줍니다.

④ 결론

= 프로세스에게 자원을 원활하게 할당하였으므로 Deadlock이 발생하지 않았습니다.

<Example_2 : Deadlock이 발생함>

안전하지 않은 상태 예) 그림 6.8

	R1	R2	R3
P1	3	2	2
P2	6	1	3
P3	3	1	4
P4	4	2	2

Claim matrix C

	R1	R2	R3
P1	1	0	0
P2	5	1	1
P3	2	1	1
P4	0	0	2

Allocation matrix A

	R1	R2	R3
P1	2	2	2
P2	1	2	2
P3	1	0	3
P4	4	2	0

C - A

	R1	R2	R3
P1	9	3	6

Resource vector R

	R1	R2	R3
	1	1	2

Available vector V

(a) Initial state

(e)

- Available vector V가 (1,1,2)이고 C-A의 모든 프로세스가 Available vector V보다 **크기 때문에** 어떤 프로세스에게도 자원을 할당할 수 없습니다.

- 그러므로 Deadlock이 발생합니다.

- 한 번 더 교수님께서 강조하셨습니다. 모든 프로세스에 대해서 OS가 매순간 위의 작업들을 하는 것은 OS에게 매우 큰 부담을 줌과 동시에 구현이 어려우므로 현대 OS는 하지 않습니다.

팁 : 항상 완벽을 추구하지는 말자(Tom West's Law)

컴퓨터 산업에 고전이 된 새로운 기계의 영혼(Soul of a New Machine) [K81]이라는 책의 제목만큼 유명한 Tom West는 “해야하는 한 모든 일이 모두 다 잘해야 하는 일은 아니다.”라는 공학적으로 아주 훌륭한 명언을 남겼다. 만약 안 좋은 일이 아주 가끔 일어난다면 그 일을 방지하기 위해서 아주 많은 시간을 들일 필요가 전혀 없다. 특히, 그 안 좋은 일에 대한 대가가 작다면 더 그렇다. 반면에, 우주 비행선을 만든다고 했을 때, 잘못되었을 때 그 비행기가 폭발할 수 있다고 하면, 그때는 이 조언을 무시해야 할 것이다.

3) Detection(발견)

- = 주기적으로 존재하는 모든 프로세스에서 발생하는 환경 대기 를 계속 추적해서 찾습니다. 그 결과 OS에게 엄청난 부담을 줍니다.
- Windows에서는 deadlock의 발생확률이 1/100만이라고 합니다. 다시 말해 deadlock은 1년에 한두 번 발생합니다.
- 극히 낮은 확률이므로 현대 OS는 대부분 deadlock을 해결하지 않습니다. 보통 사용자가 Kill로 끝내버립니다.
- 이것도 실제로 구현하기 매우 어렵습니다.

<Deadlock 발견 알고리즘>

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

- Request Matrix Q : 프로세스 마다 요구한 자원의 양
 - Allocation Matrix A : 프로세스 마다 할당된 자원의 양
 - Resource Vector : 시스템에 존재하는 자원의 양
 - Available Vector : 시스템이 할당해 줄 수 있는 자원의 양
- ① Allocation matrix A에서 모든 자원의 할당량이 0인 프로세스(P4)를 찾고 마크합니다.
 - ② Available Vector(0,0,0,0,1)를 W로 하고 , W를 주면 해결할 수 있는(자원의 요구량이 W보다 작거나 같은) 프로세스를 Q(P3)에서 찾습니다.
 - ③ 조건에 맞는 프로세스가 없다면 종료합니다.
 - ④ 조건에 맞는 프로세스를 찾는다면 해당 프로세스(P3)도 마크하고 Available Vector에 요구 자원량을 더해서 (0,0,0,1,1)로 갱신합니다.
 - ⑤ (② ~ ④)를 반복하는 과정에서 Available Vector 보다 작거나 같은 프로세스가 존재하지 않으므로 P1 , P2에서 교착상태가 발생합니다.

< Deadlock 회복 알고리즘>

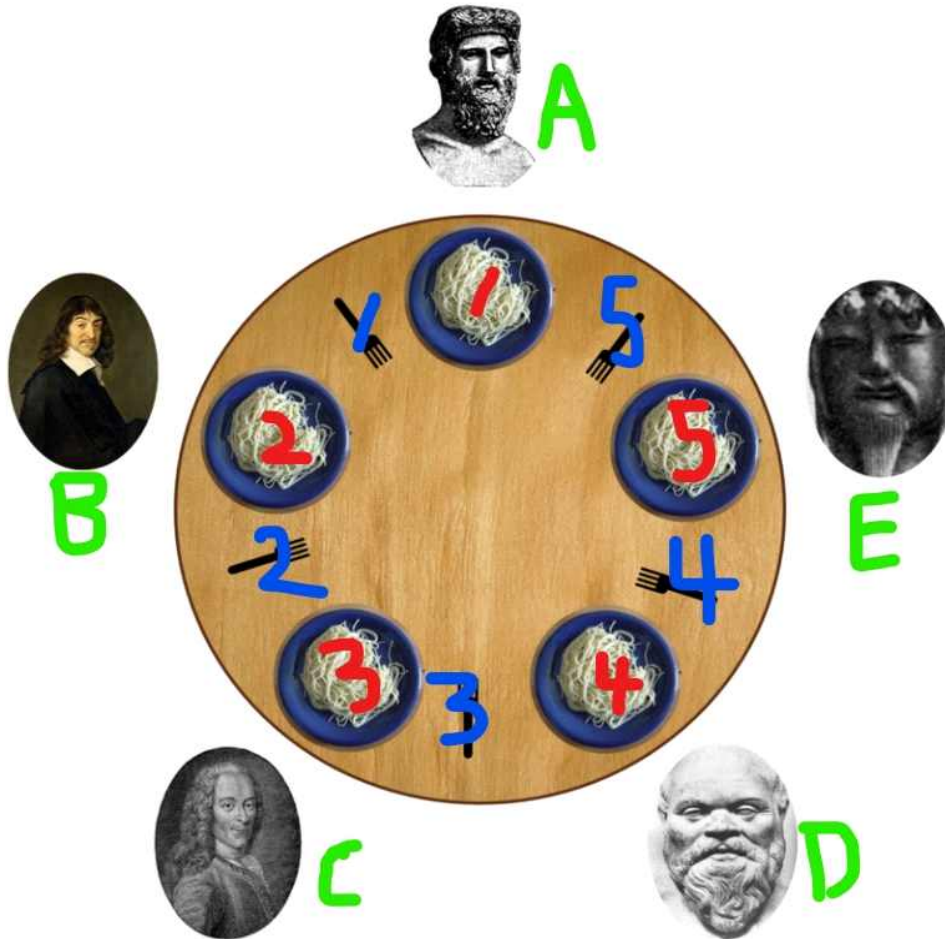
① Deadlock 관련 프로세스를 모두 중지.

② Deadlock 관련 프로세스들을 롤백.(장시간 돌아가는 프로세스들은 중간 중간 저장합니다.)

③ Deadlock이 없어질 때 까지 Deadlock에 포함된 프로세스들을 하나씩 종료합니다.

- 결국 어떤 프로세스는 victim이 되어 죽어야 deadlock이 종료됩니다.

4. Dinning Philosopher Problem(식사하는 철학자 문제)



1) 식사하는 철학자 문제의 정의

- 5명의 철학자(A~E)가 식당에 왔습니다. 원탁에 음식이 5개 있고, 포크가 5개 있습니다. 철학자들은 반드시 음식 기준 양쪽의 포크 2개를 모두 사용해서 음식을 먹어야합니다.
- 철학자는 프로세스이며, 포크는 공유 변수입니다.
- 식사를 시작하는 경우 먼저 철학자 기준 오른쪽 포크를 소유하고 그 후 왼쪽 포크를 소유합니다.

2) Deadlock이 발생하지 않는 경우

- A가 가장 어르이라 먼저 식사를 하는 경우, A가 음식을 먹기 위해 포크_1을 소유하고 그 다음 포크_5를 소유하면 B와 E는 기다리지만 C와 D 둘 중 한명은 음식을 먹을 수 있습니다.

3) Deadlock이 발생하는 경우

- 5명 모두 동시에 식사를 하려는 경우 (A,1), (B,2), (C,3), (D,4), (E,5)인 경우가 되고, 왼쪽 포크를 소유하려는 순간 아무도 왼쪽 포크를 소유할 수 없게 되고 다른 철학자가 식사를 마치길 무한정 기다립니다.

4) Deadlock 해결 방법

- ① 포크를 더 구매 합니다.
- ② 포크 한 개로 식사하는 방법을 가르쳐 줍니다.
- ③ 5명이 동시에 먹으려고 하는 경우에 문제가 발생하므로 최대 4명만이 동시에 식사를 할 수 있도록 Semaphore 변수를 설정합니다.

<과제 5번 공부> 11/12 ~ 11/18

1. Stack vs Heap

- 과거에는 Heap을 두려워했고 편리한 Stack만을 사용했습니다. 하지만 사실 둘은 메모리에 존재하는 위치만 다르지 똑같은 메모리의 저장장소입니다.

~

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6
7     //1. heap array
8     int* arr = (int*)malloc(sizeof(int)*10);
9     arr[1] = 1;
10    printf("Heap array : %08x\n" , arr);
11    printf("%08x" , &arr[1]);
12    printf("%d\n" , arr[1]);
13
14    //2. stack array
15    int arr2[4];
16    arr2[1] = 1;
17    printf("Stack array : %08x\n" , arr2);
18    printf("%08x" , &arr2[1]);
19    printf("%d\n" , arr2[1]);
20
21    //3. heap variable
22    int* value = (int*)malloc(sizeof(int));
23
24    value = 2; // make WARNING
25    printf("Heap value : %08x\n" , value);
26    printf("%d\n" , value);
27
28    //4. stack variable
29    int value2;
30    value2 = 2;
31    printf("stack value : %08x\n" , &value2);
32    printf("%d\n" , value2);
33
34
35    return 0;
36 }
```

<Stack에 저장되는 객체 vs Heap에 저장되는 객체>

- 우선 Stack은 사용자가 지정한 크기로 배열을 만들고 , 배열의 index마다 원하는 값을 저장합니다. 그리고 배열의 이름이 배열의 주소(스택영역)를 나타냅니다.
- Heap도 똑같습니다. Heap 또한 사용자가 지정한 크기로 배열을 만들고 , 배열의 index 마다 원하는 값을 저장합니다. 그리고 배열의 이름이 배열의 주소(힙 영역)를 나타냅니다.

1) 동일한 점

- Stack은 메모리의 스택 영역에 저장하고 Heap은 메모리의 힙 영역에 저장하는 차이만 존재하지 메모리에 저장되는 방식은 동일합니다.

2) 다른 점

- Stack은 한 번 지정한 배열의 크기를 바꿀 수 없지만 Heap은 메모리를 할당하고 해제할 수 있기 때문에 배열의 크기를 바꿀 수 있는 동적인 성질을 갖고 있습니다.
- Stack은 메모리의 할당 및 해제를 알아서 해주지만 , Heap은 메모리의 할당 및 해제를 사용자가 직접해주어야 합니다.
- 포인터를 사용하기 때문에 힙에 단일 변수(배열X)를 저장할 때에는 스택 변수처럼 값을 직접 지정하면 WARNING이 발생합니다.
- 이제는 Stack 과 Heap을 생각할 때 서로 다른 특징이 있지만 메모리에 저장되는 방식은 저장되는 메모리의 위치만 다르고 똑같다고 생각합니다.

```

User@DESKTOP-BAGIJS2 ~
$ gcc a1.c
a1.c: In function 'main':
a1.c:24:8: warning: assignment to 'int *' from 'int' makes pointer from integer without a cast [-Wint-conversion]
    24 |     value = 2; // make WARNING
        |         ^
User@DESKTOP-BAGIJS2 ~
$ ./a.exe
Heap array : 000003c0
000003c4  1
Stack array : ffffcc20
ffffcc24  1
Heap value : 00000002
2
stack value : ffffcc1c
2

```

2. Malloc 공부

1)



```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int* arr = (int*)malloc(sizeof(int) * 100000);
7     printf("%d\n", sizeof(arr));
8
9     return 0;
10 }

```

```

User@DESKTOP-BAGIJS2 ~
$ gcc a2.c

```

```

User@DESKTOP-BAGIJS2 ~
$ ./a.exe
8

```

- 10만 * 4의 크기를 힙 메모리에 할당 했지만 arr의 크기는 8입니다.
- 그림에서 sizeof(arr)는 크기가 40만인 힙 영역을 가리키는 **포인터 변수의 크기**이므로 64비트 컴퓨터에서 8인 것이 당연합니다.

2) 배열의 크기를 넘어서 할당(C, C++)



```
1 #include <stdio.h>
2
3 int main()
4 {
5     int arr[5];
6     for(int i=-2; i<10; i++)
7     {
8         arr[i] = 77;
9     }
10    for(int i=-2; i<10; i++)
11    {
12        printf("%d : %d , %08x\n" , i , arr[i], &arr[i]);
13    }
14
15    return 0;
16 }
17
```

```
User@DESKTOP-BAGIJS2 ~
$ gcc b1.c
./a
User@DESKTOP-BAGIJS2 ~
$ ./a.exe
-2 : 77 , ffffcc18
-1 : 0 , ffffcc1c
0 : 77 , ffffcc20
1 : 77 , ffffcc24
2 : 77 , ffffcc28
3 : 77 , ffffcc2c
4 : 77 , ffffcc30
5 : 77 , ffffcc34
6 : 6 , ffffcc38
7 : 78 , ffffcc3c
8 : -13088 , ffffcc40
9 : 0 , ffffcc44
```

- C언어와 C++은 그냥 연속된 메모리에 저장시켜버립니다. 값은 자유롭게 적히는 것 같습니다.
- c++의 container(vector)는 크기를 넘어서면 에러를 출력합니다.

http://1st.gamecodi.com/board/zboard.php?id=GAMECODI_Talkdev&no=2453

<모르는 내용 및 학습 키워드>

```
uint8_t* first = mmap(NULL, PAGE_SIZE, PROT_READ | PROT_WRITE,  
                      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);  
  
uint8_t* second = mmap(NULL, PAGE_SIZE, PROT_READ | PROT_WRITE,  
                      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
```

- uint8_t는 그냥 normal한 자료형이니까 상관없고

- 6개의 매개변수

① 1번 매개변수

= 메모리 할당 시작주소

- NULL : OS가 아무데나 적당한 곳에 메모리를 할당해주세요

- 사용자 지정 주소 : 내가 지정한 곳에 메모리를 할당해주세요. 그러나 PAGE 단위가 4K이므로 하위 12비트는 000이 됩니다.

```
mmap((void*)0xFEEDBEEF, PAGE_SIZE,  
     MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
```

```
tracing $ ./a.out  
First: 0xfeedb000
```

② 2번 매개변수

= 할당할 메모리의 사이즈 (byte단위)

- 과제에서는 4K이므로 4096

③ 3번 매개변수

= protection

- 메모리를 어떤 용도로 쓰겠습니까.

PROT_READ : 읽기 가능한 페이지
PROT_WRITE : 쓰기 가능한 페이지
PROT_EXEC : 실행 가능한 페이지
PROT_NONE : 접근할 수 없는 페이지

④ 4번 매개변수

= Mapping 유형과 동작 구성 요소

⑤ 5번 매개변수 & 6번 매개변수

= 디스크의 파일과 메모리를 연결(?)하는 것 같은데 과제에서는 디스크의 파일이 연관되지 않기 때문에 중요하지 않은 매개변수입니다.

- 디스크의 파일을 쓰지 않는다면 각각 -1과 0을 적어줍니다.

를 거쳐서 실제 입출력을 하게된다. 하지만 mmap()을 이용하여 메모리 맵 방식으로 파일을 연결하게 되면 버퍼를 이용하는 것이 아니라 '페이지(page)'를 이용하여 데이터 처리가 가능해진다. 상대적으로 크기가 작은 버퍼에 비해 보통 4KB의 크기를 가지는 페이지를 이용하

- Start 는, Kernel이 아무 곳이나 지정해도 좋다면 NULL을 입력한다.
- Offset 은, 보통 0으로 둔다. (MAP_ANONYMOUS 또는 MAP_ANON Flag가 있는 경우엔 무시된다)
- Fd 는, 연결할 파일 디스크립터를 지정한다. (MAP_ANONYMOUS 또는 MAP_ANON Flag를 통해서 '파일로 사용하지 않는다' 라고 한다면 -1을 넣어줘야 한다. (사실 안 넣어주고 무시해도 되지만 몇몇 구현에서는 넣어야 한다 카더라)

<39강_Interlude : File and Directory> 11/18 ~ 11/20

1. File

= 디스크에 기록된 관련 정보의 집합으로 정의할 수 있습니다. 사용자 입장에서 파일은 논리적 보조 저장 장치의 가장 작은 할당 요소입니다.

- 파일은 단순히 읽거나 쓸 수 있는 순차적인 바이트의 배열입니다. 키보드로 입력할 수 있는 값들이 최소 단위이며 이들은 모두 ASCII코드로 나타낼 수 있습니다. 그러므로 1 byte로 표현할 수 있습니다. 한글의 경우 유니코드이므로 2 byte로 표현 됩니다.
- 디스크 상에서 연속적으로 저장 될 필요는 없습니다.
- 사용자 관점에서의 File : file_name(ex : foo.txt)
- 시스템 관점에서의 File : inode의 번호(숫자)를 이름으로 갖는데 사용자는 이를 알 수 없습니다.

2. Directory

1) Overview

- 파일과 마찬가지로 inode 번호를 갖습니다.
- 계층 구조(트리)를 이룹니다.

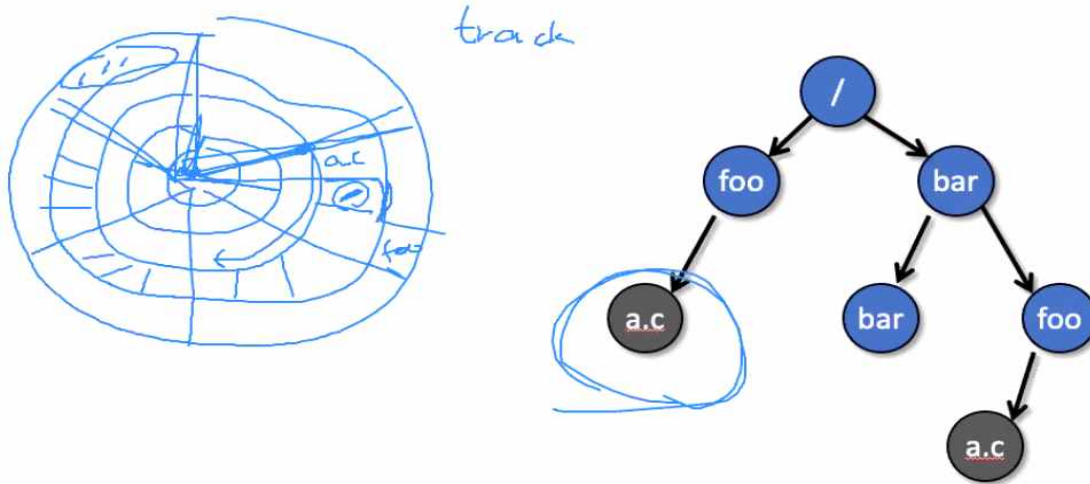
2) 계층 구조

= Directory를 트리 형태로 계층적으로 저장합니다.

- 원하는 파일을 찾는 것이 느립니다. Directory가 많으면 많을수록 더 느려집니다.

ex) Windows , Linux , Android(Linux 기반)는 계층 구조로 Directory가 구성됩니다.

<느린 이유>



- foo -> a.c는 트리 구조상 매우 근처에 있지만 디스크 내부에서는 다른 트랙에 존재합니다. 이런 경우 디스크 헤더가 트랙 단위로 움직여야 하기 때문에 오래 걸립니다.
- 최악의 경우 디스크 헤더는 전체 디스크를 모두 탐색해야 합니다.
- 하드 디스크의 발전으로 회전 속도는 빨라졌지만 헤더의 이동은 한계가 있습니다.

2) Index 기법

= Root Directory 아래에 모든 파일을 한 곳에 모아서 저장합니다.

- 계층 구조가 아니기 때문에 원하는 파일을 찾는 것이 **매우 빠릅니다.**

- 사용자는 파일이 어디에 저장되어 있는지 알 수 없습니다. 요즘은 대략적인 위치정도는 알 수 있다고 합니다.

ex) BSD 기반인 IOS는 Index 기법을 사용하여 Directory를 구성합니다.

3. File Descriptor Table & File Table

1) File Descriptor Table

= 프로세스 마다 존재하는 배열(Table)입니다.

- File Descriptor들이 배열의 원소입니다.
- File Descriptor는 local 주소입니다.
- File Descriptor Table은 프로세스가 생성 될 때 자동으로 생성되며 default File Descriptor로 0 , 1 , 2를 생성합니다.
- Open(시스템 콜)할 때 마다 생겨서 open Table이라고도 합니다.

2) File Table

- 같은 File Descriptor 값을 가지면 File Table의 같은 곳을 가리킵니다.
- 전역적인 구조체(시스템 전체에 1개만 존재합니다.)
- Offset과 Counter를 기록합니다.

4. File System_Call

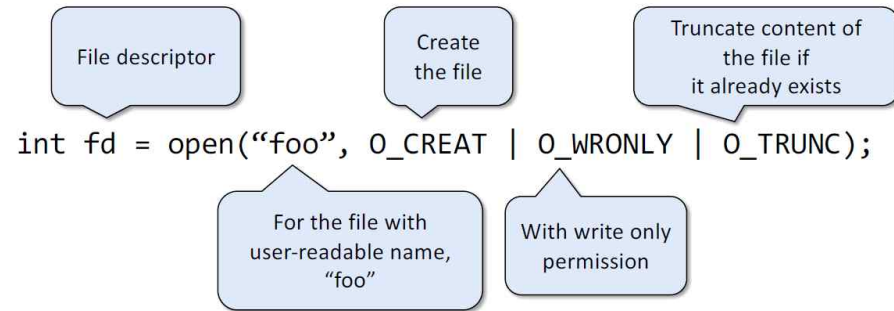
- 파일 관련 시스템 콜(open)과 C언어 라이브러리 함수(fopen)는 유사합니다.
- 라이브러리 함수가 익숙하므로 시스템 콜은 참고로 알아둡시다.
- C언어 라이브러리 함수는 <stdio.h> 헤더에 모두 포함되어 있습니다.

SYSTEM CALL VERSUS LIBRARY CALL

SYSTEM CALL	LIBRARY CALL
A request by the program to the kernel to enter kernel mode to access a resource	A request made by the program to access a function defined in a programming library
The mode changes from user mode to kernel mode	There is no mode switching
Not portable	Portable
Execute slower than library calls	Execute faster than system calls
System calls have more privileges than library calls	Library calls have less privileges than system calls
fork() and exec() are some examples for system calls	fopen(), fclose(), scanf() and printf() are some examples for library calls
	Visit www.PEDIAA.com

- 라이브러리의 속도가 더 빠릅니다. 그러므로 라이브러리를 사용해야 합니다.

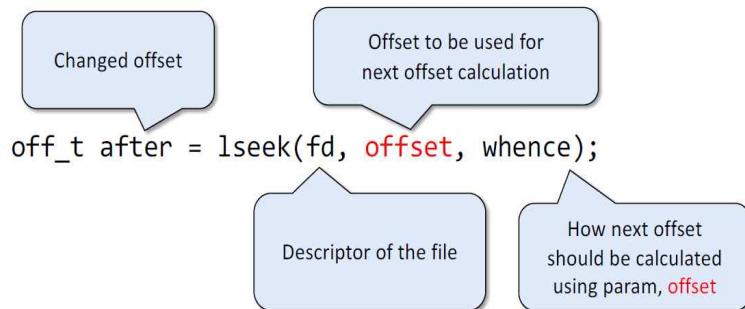
1) open()



= 파일 생성 / 파일 열기

- 무조건 create를 하고 open을 해야 합니다.

2) lseek()



= 파일의 offset 변경

- Whence : 어디서부터 시작할지의 기준점

3) read()

Number of bytes
read() has read

Buffer that read content
should be stored inside

```
ssize_t bytes = read(fd, buffer, length);
```

Descriptor of the file

Size of the buffer

= 파일을 읽고 , 읽은 내용을 buffer에 저장합니다.

4) write()

Number of bytes
write() has written

Buffer that
data to be written is inside

```
ssize_t bytes = write(fd, buffer, count);
```

Descriptor of the file

Number of bytes
issued to be written

- = Count 만큼의 byte를 파일에 적습니다.
- 디스크에 바로 적지 않고 우선 buffer에 적습니다.

<Delayed Write(지연 쓰기)>

- 지금 HWP에 작성하는 것처럼 필기 하는 순간 마다 디스크에 바로 적는다고 생각해보시다. I/O가 발생하므로 매우 느립니다. 그렇기 때문에 일단 buffer에 모아 놓고 일정 시간 마다 한 번에 디스크에 적어줍니다.
 - 디스크에 바로 쓰는 경우도 가끔 있습니다.
 - Buffer를 Buffer Cash라고 하며 이는 H/W 캐시가 아니라 S/W 캐시 입니다.
 - 보통 10초마다 System-call인 Sync()함수를 사용해서 buffer에서 디스크로 저장된 데이터를 flush(디스크로 데이터를 옮겨서 Buffer를 완전히 비움)합니다. 이 과정에서 전기가 나가면 파일의 일부를 잃을 수 있습니다.
 - 사용자가 직접 C 라이브러리 함수인 fsync() 또는 명령어로 sync()를 사용해서 강제로 sync()를 호출할 수 있습니다.
- ex) \$sync \$sync -> 어떠한 이유로 인해 2번 적어주어야 합니다.

5) rename()

0 if rename() success,
-1 else

New name of the file
to be changed

```
int res = rename(old_name, new_name);
```

Old name of the file
to be changed

= 파일 이름 변경

6) mkdir()

0 if mkdir() success,
-1 else

Permission of the directory

```
int res = mkdir("foo", 0777);
```

Name of directory
to be made

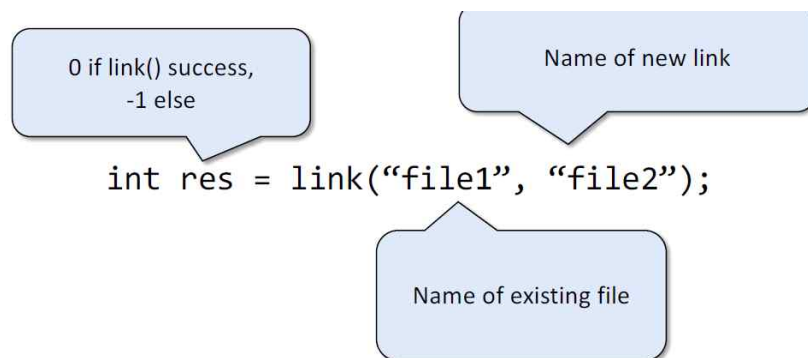
= Directory 생성

7) copy() & link()

① copy

- cp A B
- Link와 관련 있는 내용입니다.
- 두 파일 A , B는 다른 inode를 가집니다.
- A의 내용을 변경해도 B의 내용은 변경되지 않습니다.

② link()



= 동일한 파일에 대한 새로운 쌍을 생성합니다.

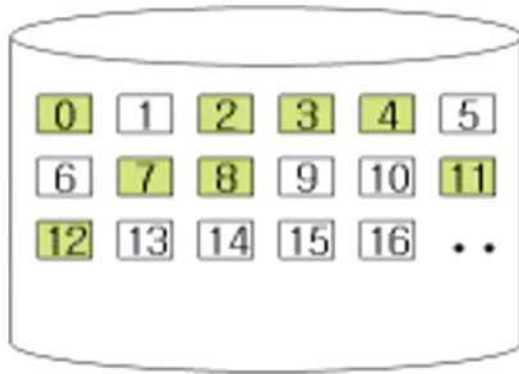
- link A B
- 두 파일 A , B는 같은 inode를 가집니다.
- A를 지워도 B 때문에 살아있고 , A의 내용을 변경하면 B의 내용도 변경됩니다.
- Hard link는 link와 동일하고 , Hard link의 단점을 보완한 Symbolic link는 바로 가기와 동일합니다.

8) mount()

= 파일 시스템이 다른 디스크를 루트의 하위 directory로 연결하여 사용 가능하게 해주는 명령어입니다.

<40강_File System Implementation> 11/25 ~ 11/27

1. 파일 시스템의 종류



- 그림은 디스크를 나타냅니다.
- 각 번호에 해당 하는 객체들은 블록을 나타내며 4KB입니다.
- 녹색은 이미 데이터가 적혀있는 block입니다.
- 흰색은 데이터가 적혀 있지 않은 free block입니다.
- 14KB의 파일을 디스크에 저장해보시다. $14/4 = 3.5$ 이므로 파일을 저장하기 위해서는 4개의 블록이 필요합니다.

<파일 시스템>

- 컴퓨터에서 파일이나 자료를 쉽게 발견 및 접근할 수 있도록 보관 또는 조직하는 시스템 체계
- 소프트웨어입니다.

1) Sequential Allocation(연속 할당)

- 14KB를 저장해야 하므로 4개의 연속된 블록이 필요합니다. 그러므로 13 ~ 16번 블록에 파일을 저장합니다.
- 메타 데이터로 2가지 변수를 저장해야 합니다. (Start block의 번호 , 파일의 크기)
- 테이프(구식)에서 썼던 기술이며 지금은 사용하지 않습니다.

<문제점>

- 연속된 공간이 없으면 압축(compaction)을 해야 합니다.
- 압축을 하면 파일 시스템이 파일들에 대한 정보 기억 내용을 다시 갱신해야

2) Non-Sequential Allocation(불연속 할당)

① Block Chain

- Linked List 구조를 이용하여 블록들을 가리킵니다.

ex) 1번 블록 -> 5번 블록 -> 6번 블록 -> 9번 블록 -> NULL_PTR

- 메타데이터로 1개의 변수만 저장하면 됩니다. (Start block의 번호)

<문제점>

- 16KB의 파일을 저장하려고 하면 4개의 블록이 필요하지만 Linked List를 사용하면 4바이트의 포인터를 저장해야 하므로 5개가 필요합니다.

② FAT(File Allocation Table)

- 테이블(배열)의 index로는 블록의 번호이며 , 해당 index에 저장되는 값이 가리킬 블록의 번호가 됩니다.

<문제점>

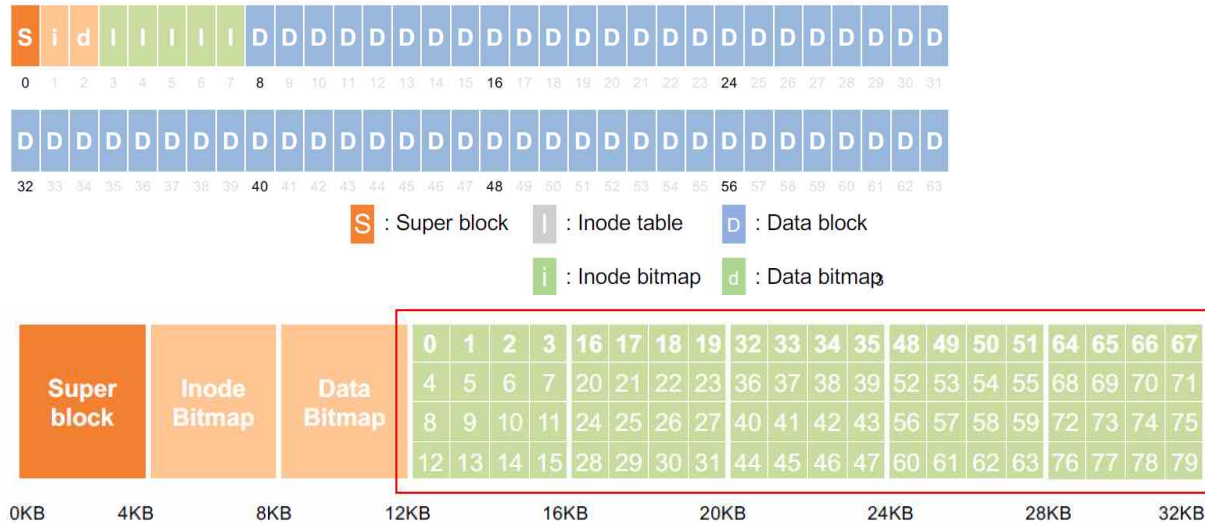
- 포인터를 저장하지 않지만 파일마다 하나씩 FAT을 저장해야 합니다.

③ Index Block

- Inode와 관련된 중요한 개념이므로 아래에서 자세하게 설명하겠습니다.

2. 간단한 파일시스템 구조

- Very Simple File System(vsfs)



<메타 데이터>

= 데이터에 대한 데이터로, 데이터를 빠르게 검색하기 위해 만들어진 데이터.

- 파일의 종류, 크기, 할당된 블록 수, 파일이 저장된 Data Block의 위치, 소유자, 접근 권한, 변경 시간 등과 같은 정보를 가진 데이터
- 사용자 데이터(디스크의 대부분을 차지)가 아닌 기타 정보를 통틀어서 메타데이터라고 부릅니다.

- 그림의 파일시스템은 4KB 크기의 블록이 64개가 존재하고, 1개의 Super Block, 1개의 Inode Bitmap, 1개의 Data Bitmap, 5개의 Super Block, 56개의 Data Block으로 구성되어 있습니다.
- 파일은 자신의 데이터와 메타데이터로 구성되어 있고, Data Block은 데이터(사용자가 파일에 저장한 모든 내용, cat 명령어로 확인할 수 있음)를 저장하고, Super Block과 Inode Block은 메타 데이터를 저장합니다.
- Inode Block이 5개이고, 이는 80개의 Inode로 구성되어 있으므로 그림의 파일시스템은 최대 80개의 파일을 만들 수 있습니다.
- inode 영역에 있는 inode들의 사용 여부를 표현하기 위해 inode 비트맵(inode bitmap)을 사용합니다.
- Data 영역에 있는 블록들의 사용 여부를 표현하기 위해 데이터 비트맵(data bitmap)을 사용합니다.

<과제 6번 : 간단한 파일시스템>

1) Super Block

```
struct superblock_t
{
    char name[MAX_NAME_STRLEN];
    char inode_freelist[NUM_INODES];
    char datablock_freelist[NUM_DATA_BLOCKS];
};
```

- Super Block은 파일 시스템 전체에 대한 정보를 구조체를 사용해서 담고 있습니다.
- 과제에서는 파일 시스템의 이름 , 전체 아이노드 개수(과제에서는 비어있는 아이노드 개수), 전체 데이터 블록 개수(과제에서는 비어있는 데이터 블록 개수)
- Super Block이 훼손되면 파일 시스템이 깨지는 것이므로 대부분의 파일 시스템은 Super Block을 몇 개 복사해둡니다.

2) Inode (Index_Node)

```
struct inode_t
{
    int status;
    char name[MAX_NAME_STRLEN];
    int file_size;
    int direct_blocks[MAX_FILE_SIZE];
};
```

- Inode Block을 구성하는 inode는 구조체를 사용해서 정보를 담고 있습니다.
 - 과제에서 inode struct는 현재 inode의 상태(비어 있는지), inode의 이름 , inode가 가리키는 사용자 데이터의 크기 , inode가 가리키는 사용자 데이터의 데이터 블록에서의 저장 위치를 저장합니다.
 - 결론적으로 , inode는 구조체를 사용해서 자신이 담당하는 파일의 데이터에 대한 정보를 담고 있습니다.
- ex) 0번_inode 블록에 포함된 6번_inode가 담당하는 400 bytes 크기의 사용자 file은 n번_data block 에 저장합니다.

① 정의

= **파일의 메타데이터를 저장하기 위한** 자료 구조

- Inode도 저장을 위해 디스크 공간이 필요합니다.
- Inode를 사용자가 관리하기 힘들기 때문에 stat 구조체(inode의 일부분을 추출한 구조체)를 사용합니다.

② Inode Block에 저장된 inode



- 그림의 디스크에서는 4KB 블록 5개를 inode 들을 저장하기 위해 할당해주었습니다. 보통 inode 1개는 256 bytes 정도 이므로 4KB 블록에 16개 정도가 들어갑니다.
- **0 ~ 79까지 총 80개의 inode가 디스크에 저장**되어 있습니다. inode가 80개가 존재하므로 이 파일 시스템에서는 파일을 최대 80개를 만들 수 있습니다.
- 0 ~ 79를 각각 inumber(=저수준 이름)라고 부릅니다.

3) Data

= 자신을 담당하는 Inode가 지정해준 Data Block에 파일의 데이터를 모두 저장합니다.

3. Inode가 파일의 블록 위치를 표현하는 방법

- 직접 포인터만으로는 큰 파일을 표현할 수 없기 때문에 간접 포인터 개념을 도입했습니다.
- 간접 포인터로도 부족하여 원리는 같은 이중 간접 포인터 , 삼중 간접 포인터 개념이 추가되었습니다.

1) 직접 포인터

- Linux는 12개 UNIX는 10개 정도 존재합니다.
- 직접 가리키기 때문에 블록의 크기가 4KB면 $4 * 10 = 40\text{KB}$ 이므로 고작 40KB 크기의 파일만 저장할 수 있게 됩니다.

2) 간접 포인터(특수 포인터)

- 간접 포인터는 직접적으로 데이터 블록을 가리키지 않고 데이터 블록을 가리키는 포인터들이 저장된 블록을 가리킵니다.

ex) 블록 크기가 4KB이고 포인터 변수의 크기가 4Byte라면 하나의 블록에는 1024개의 포인터 변수를 저장할 수 있으므로 , 1개의 간접 포인터는 1024개(4KB / 4Byte)의 블록을 가리킬 수 있습니다. 1024개의 블록은 $4\text{KB} * 1024 = 4\text{MB}$ 이므로 , 최대 4MB 크기의 파일을 디스크에 저장할 수 있습니다.

3) 이중 간접 포인터

- 간접 포인터를 이중으로 사용한 개념입니다.

ex) 이중 간접 포인터를 사용하면 간접 포인터에서 가리킨 1024개의 블록에서 각각 1024개의 포인터 변수를 가리킬 수 있으므로 $1024 * 1024$ 개의 블록을 가리킬 수 있습니다. 그러므로 $4\text{KB} * 1024 * 1024 = 4\text{GB}$ 이므로 , 최대 4GB 크기의 파일을 디스크에 저장할 수 있습니다.

4) 삼중 간접 포인터

- 같은 원리로 4TB 크기의 파일을 디스크에 저장할 수 있습니다.

<기출 문제_1>

- 10개의 직접 데이터 블록 포인터
- 1개의 간접 데이터 블록 포인터
- 1개의 이중간접 데이터 블록 포인터
- 1개의 삼중간접 데이터 블록 포인터
- 디스크의 블록의 크기는 1KB 일때. 이 시스템에서 가질 수 있는 파일의 최대 크기는?

- 직접 포인터 : $10 * 1KB$
- 간접 포인터 : $1 * 256 * 1KB$
- 이중 간접 포인터 : $1 * 256 * 256 * 1KB$
- 삼중 간접 포인터 : $1 * 256 * 256 * 256 * 1KB$
- 4 가지 경우를 모두 더하면 시스템에서 가질 수 있는 파일의 최대 크기를 알 수 있습니다.
- 파일의 최대 크기를 주어주고 역으로 포인터들의 크기를 구하는 심화 문제가 나올 수 있습니다.

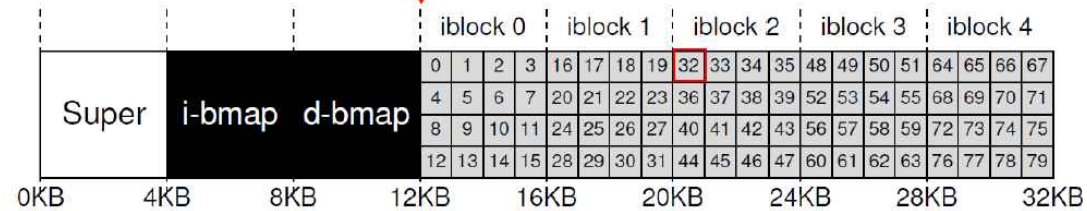
<기출문제_2>

inode의 위치 계산

- $\text{blk} = (\text{inumber} * \text{sizeof}(\text{inode_t})) / \text{blockSize};$
- $\text{sector} = ((\text{blk} * \text{blockSize}) + \text{inodeStartAddr}) / \text{sectorSize};$

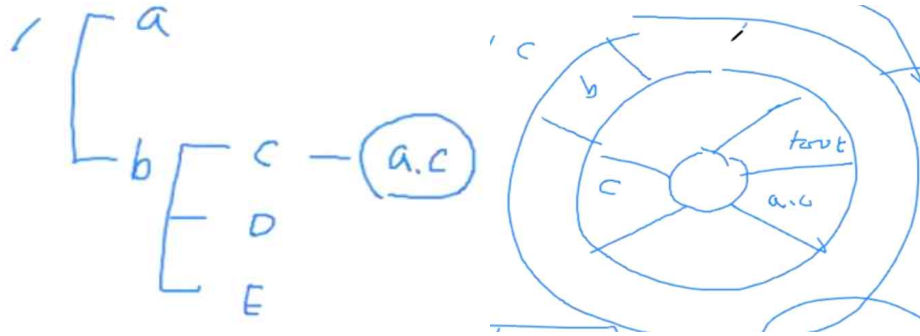
Inode number	32
Inode size	256 B
Inode start address	12 KB
Block size	4 KB
Sector size	512 B

iblock	=	$(i_number * inode_size) / block_size$
	=	$32 * 256B / 4KB$
	=	2
sector	=	$((iblock * block_size) + i_start_addr) / sector_size$
	=	$(2 * 4KB + 12KB) / 512B$
	=	40



4. 실행 흐름 : 파일의 읽기와 쓰기

1) 파일 읽기



- a.c 파일의 경로 : '/b/c/a.c'입니다.

<'a.c'를 찾아서 읽는 과정>

- ① Root의 inode로 가서 root의 데이터 블록으로 갑니다.
- ② Root의 데이터 블록에는 자신 아래에 폴더_a와 폴더_b의 inode에 대한 정보가 저장되어있습니다.
- ③ 폴더_b의 inode로 가서 폴더_b의 데이터 블록으로 갑니다.
- ④ 폴더_b의 데이터 블록에는 자신 아래에 폴더_c와 폴더_d와 폴더_e의 inode에 대한 정보가 저장되어있습니다.
- ⑤ 폴더_c의 inode로 가서 폴더_c의 데이터 블록으로 갑니다.
- ⑥ 폴더_c의 데이터 블록에 'a.c'가 존재합니다.

- Inode는 디스크에 연속적으로 존재합니다. 사실 inode는 메모리에 올려놓기 때문에 디스크 search 과정에서 무시할 만큼 빠릅니다.
- Inode가 존재하는 블록과 데이터 블록은 명확하게 구분해야 합니다.
- 그림에서 root , b , c , 'a.c'는 데이터 블록입니다.
- 데이터 블록들이 디스크 내에서 흩어져 있으므로 경로가 길어질수록 찾는 데 시간이 많이 걸릴 것입니다.
- Inode 파일시스템에서 inode를 잘 찾아주면 , 데이터 블록 찾는 과정은 간단하므로 폴더에는 inode_number와 파일이름을 저장합니다.
- 요약하면 , 트리구조에서 원하는 파일을 찾는 방법은 inode에서 데이터 블록 위치를 찾고 , 데이터 블록에 저장된 inode를 찾아가는 과정을 반복하는 것입니다.

5. 파일 쓰기

= 파일 읽기와 동일한 방식입니다.

<directory>

- 특수 형태의 파일
- 하나의 inode와 데이터 블록들을 갖고 있음.
- direct.h에 inode 번호와 name은 반드시 들어가야 합니다.

inum	reclen	strlen	name
5	4	2	.
2	4	3	..
12	4	4	foo
13	4	4	bar
24	8	7	foobar

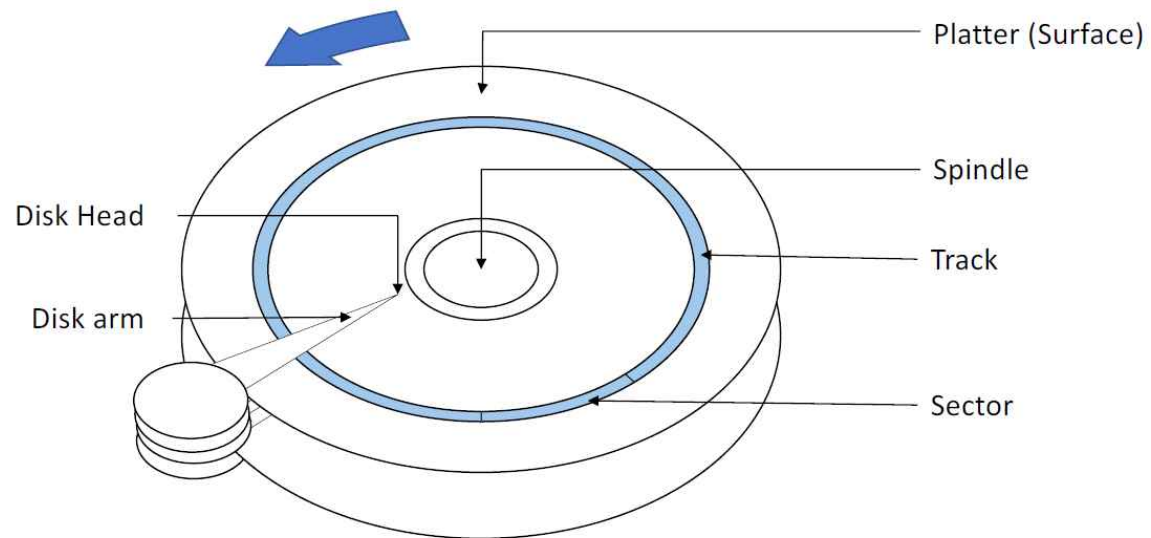
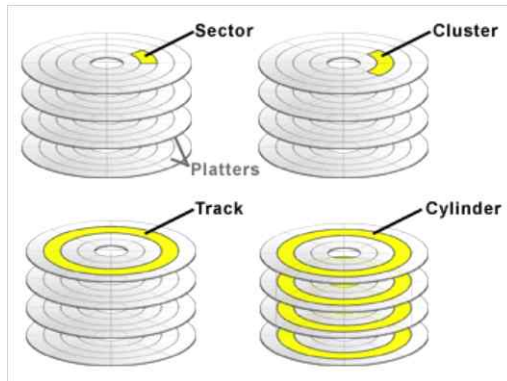
<37강_HDD Drives> 11/27

1. HDD

- = 영속적인 데이터 저장소
- 하드 인터페이스 SATA
- 근본회사 : Western_Digital vs Seagate

1) HDD 구조

- Sector가 전통적으로 512bytes이므로 보통 8개의 sector를 모아서 block(4kb)을 구성합니다.
- 회전수(RPM) : 요즘은 10,000rpm



Platter

- One or more platters
- Two surfaces per a platter

Disk Head

- Reading/Writing from the surface

Disk Arm

- Locates the disk head over the desired track

Track

- Concentric circle of sectors
- Two surfaces per a platter

Sector

- Numbered from 0 to n-1
- 0 to n-1 is address space of the drive
- e.g., 512 bytes

Spindle

- Spins the platters at a constant rate



Source : <http://en.wikipedia.org/wiki/File:Laptop-hard-drive-exposed.jpg>

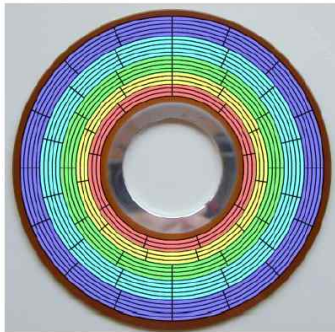
2) HDD에서 데이터를 읽는 시간

- Seek time + Rotational delay + Transfer time
- Seek time(탐색 시간) : Disk Arm을 원하는 데이터가 존재하는 섹터의 **트랙으로 이동**시킵니다.
- Rotational time(회전 지연 시간) : Disk Arm을 이동한 트랙에서 원하는 데이터가 존재하는 **섹터로 이동**시킵니다.
- Transfer time(전송 시간) : 섹터에서 데이터를 읽습니다.

3) Zone

= 연속 된 트랙의 집합

- 지름에 따라 원의 크기가 다르므로 트랙마다 존재하는 섹터의 개수가 다릅니다.
- HDD의 외부 구역에 내부구역 보다 더 많은 섹터가 존재합니다.
- 섹터의 크기는 모두 동일합니다.
- 그림처럼 트랙마다 섹터의 분할 부위가 모두 다르므로 트랙 이동시에 **Track Skew(왜곡)**이 발생해서 미세하게 헤더를 조정해주어야 합니다.



4) 추가 내용

- 디스크에서 섹터를 읽을 때 트랙전체를 읽어서 메모리에 저장해서 후속 요청에 대비합니다.
- Write Back : 데이터를 우선 메모리에 먼저 쓰고 , 후에 디스크에 씁니다. 메모리에 먼저 쓰는 경우에도 디스크는 쓰기가 완료되었다고 인식합니다. 약간의 위험한 방식입니다.

2. 디스크 관련 개념

1) $I/O \text{ Time} = \text{seek time} + \text{rotation time} + \text{transfer time}$

2) Workload(작업량)

= sequential workload vs random workload

- 제대로 된 workload 계산은 random workload입니다.

3. Disk Scheduling

- Seek time에만 초점을 두고 Rotational time & Transfer time은 무시합니다.

- 헤더의 이동거리로 성능을 판단합니다.

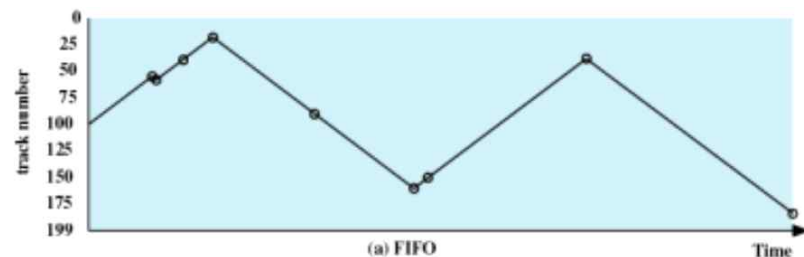
ex) 55번 트랙에서 58번 트랙으로 이동하면 헤더의 이동거리는 3, 58에서 39로 이동하면 헤더의 이동거리는 19가 됩니다.

<Example : 트랙_55에 저장된 데이터가 가장 먼저 들어왔고, 트랙_184에 저장된 데이터가 가장 나중에 들어왔을 때의 Disk Scheduling>

200개의 트랙을 지닌 디스크에서 처음에는 디스크 헤드가 트랙 100에 있고 무작위 요청을 가정

55, 58, 39, 18, 90, 160, 150, 38, 184

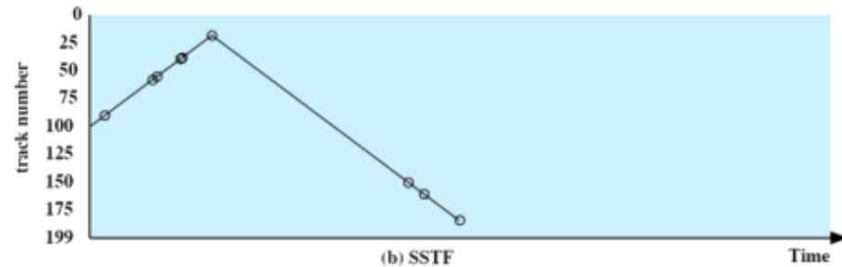
1) FIFO



= 들어온 순서부터 이동합니다.

- 55 -> 58 -> 39 -> 18 -> 90 -> 160 -> 150 -> 38 -> 184

2) SSTF(Shortest Seek Time First)



= 현재 트랙에서 가장 가까운 트랙부터 이동합니다.

- 이동거리는 FIFO에 비해 적어서 성능은 좋아지지만 특정 범위에서만 이동하므로 양 극(0근처 200근처)에 있는 놈들은 서비스를 받지 못하는 starving(기아 현상)이 발생합니다.

- 90 -> 58 -> 55 -> 39 -> 38 -> 18 -> 150 -> 160 -> 184

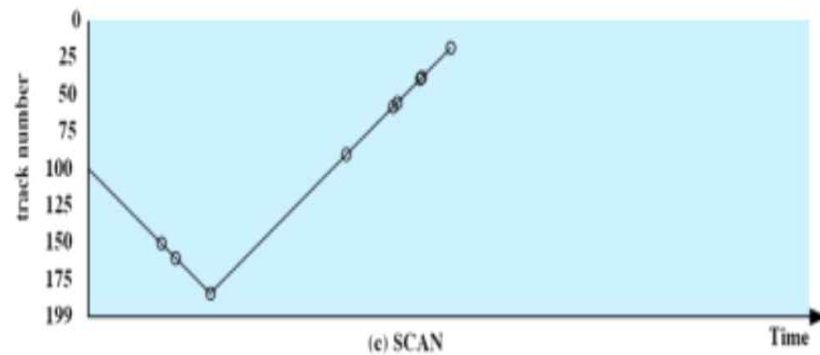
3) SCAN

- 현대에 많이 쓰는 Disk Scheduling 방식입니다.

- Arm Stickiness(암 고정 현상)가 발생합니다. 양극에 있는 놈들의 서비스 요청에 대해서 극단적으로 기아현상이 발생하지는 않지만 서비스 받는 시간이 길어질 수 있습니다.

- 아래 두 경우 모두 처음에는 큰 트랙으로 이동하는 경우로 설명하겠습니다.

① SCAN

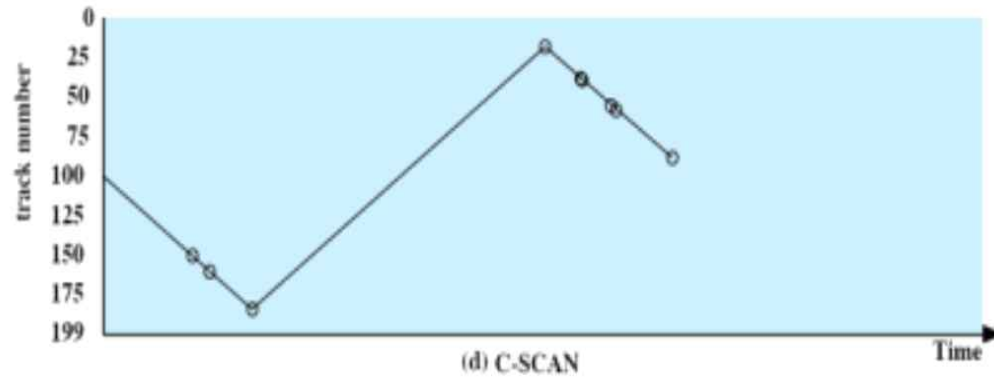


= 엘리베이터의 작동방식과 동일하게 한 방향으로만 이동합니다.

- 양 극에 있는 놈들까지 도달하는데 시간이 많이 걸리므로 서비스를 빠르게 받기가 어렵습니다.

- 150 -> 160 -> 184 -> 90 -> 58 -> 55 -> 39 -> 38 -> 18

② C-SCAN(Look)



= 가장 큰 번호인 트랙에 도달했을 때 , 무조건 가장 낮은 번호인 트랙으로 이동해서 다시 올라갑니다.

- SCAN보다 양 극에 있는 놈들이 덜 불리하지만 여전히 불리합니다.

- 150 -> 160 -> 184 -> 18 -> 38 -> 39 -> 55 -> 58 -> 90

- 일반적인 C-SCAN은 양쪽 끝(0 OR 199)까지 다녀와야 하지만 , 강의에서 배운 C-SCAN(Look)은 양쪽 끝 방문 요청이 없으면 양쪽 끝을 가지 않습니다.

③ N-step-SCAN

- 트랙들을 묶어서 segment 단위로 만듭니다.
- 여러 개의 Queue를 사용합니다.

④ FSCAN

= 두 개의 Queue를 사용합니다. 스캔을 시작하기 전에 모든 트랙들을 1번 Queue에 저장하고 , 스캔 하는 도중 요청된 트랙들은 2번 Queue에 저장합니다.

4. Disk Scheduling 정리

(a) FIFO (starting at track 100)		(b) SSTF (starting at track 100)		(c) SCAN (starting at track 100, in the direction of increasing track number)		(d) C-SCAN (starting at track 100, in the direction of increasing track number)	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
Average seek length	55.3	Average seek length	27.5	Average seek length	27.8	Average seek length	35.8

<과제 6번 공부> 11/29

<구조체 포인터>

- 구조체 포인터를 이용하면 이미 만들어 놓은 구조체의 객체를 쉽게 사용할 수 있습니다.
- 어떤 객체를 만들 때 struct str obj로 만들면 stack영역에 정적 할당이 됩니다.
- 어떤 객체를 만들 때 struct str* obj + malloc으로 만들면 Heap영역에 동적 할당이 됩니다.
- 둘 다 함수 내부에 선언하면 해당 함수에서만 사용할 수 있는 지역성을 갖고 , 저장되는 영역과 정적/동적의 차이만 있을 뿐 둘 다 객체를 나타냅니다.

2) Heap에 만든 구조체 객체 obj1을 구조체 포인터로 가리키기



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct str
5 {
6     int a;
7     int b;
8 };
9
10 int main()
11 {
12     struct str* obj1 = (struct str*)malloc(sizeof(struct str));
13     obj1->a = 1;
14     obj1->b = 2;
15
16     struct str* obj2 = obj1;
17     printf("%d %d\n" , obj2->a , obj2->b);
18
19     return 0;
20 }
```

```
User@DESKTOP-BAGIJS2 ~
$ ./a.exe
1 2
```


2) Stack에 만든 구조체 객체 obj1을 구조체 포인터로 가리키기



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct str
5 {
6     int a;
7     int b;
8 };
9
10 int main()
11 {
12     struct str obj1;
13     obj1.a = 3;
14     obj1.b = 4;
15     struct str* obj2 = &obj1;
16     printf("%d %d\n" , obj2->a , obj2->b);
17
18
19     return 0;
20 }
```

```
User@DESKTOP-BAGIJS2 ~
$ ./a.exe
3 4
```

1. 1번 하기 전 선행 ssufs_formatdisk()로 ssufs라는 binary file을 초기화하고 hexdump로 찍어보자>

```
00000000 7373 6675 0073 6900 7878 7878 7878 7878
00000010 7878 7878 7878 7878 7878 7878 7878 7878
00000020 7878 7878 7878 7878 7878 7878 7878 0000
00000030 4a98 716d 7fb7 0000 0000 0000 0000 0000
00000040 0078 0000 0000 0000 0000 0000 0000 0000
00000050 ffff ffff ffff ffff ffff ffff ffff ffff
00000060 0078 0000 0000 0000 0000 0000 0000 0000
00000070 ffff ffff ffff ffff ffff ffff ffff ffff
00000080 0078 0000 0000 0000 0000 0000 0000 0000
00000090 ffff ffff ffff ffff ffff ffff ffff ffff
000000a0 0078 0000 0000 0000 0000 0000 0000 0000
000000b0 ffff ffff ffff ffff ffff ffff ffff ffff
000000c0 0078 0000 0000 0000 0000 0000 0000 0000
000000d0 ffff ffff ffff ffff ffff ffff ffff ffff
000000e0 0078 0000 0000 0000 0000 0000 0000 0000
000000f0 ffff ffff ffff ffff ffff ffff ffff ffff
00000100 0078 0000 0000 0000 0000 0000 0000 0000
00000110 ffff ffff ffff ffff ffff ffff ffff ffff
00000120 0078 0000 0000 0000 0000 0000 0000 0000
00000130 ffff ffff ffff ffff ffff ffff ffff ffff
00000140 000a
00000141
```

- 00 ~ 30 : 64바이트의 super block

<superblock 분석>

① name = 8바이트를 "ssufs"로

② free_inode = 8바이트로 구성되며 1바이트당 1개의 inode가 free한지를 나타내줍니다.

즉, 3번_inode의 free여부는 6900다음의 7878 7878 7878 7878 중 빨간 것(hexdump는 2바이트중 우측 것부터 표기) -> 0번_inode가 할당되어서 사용불가가 되니까 7831 7878 7878 7878 나옴.

③ free_data_block = 30바이트로 구성되며 30바이트를 free_inode와 동일한 원리로 data_block 관리

④ 쓰레기 값 = 64바이트 중 8 + 8 + 30만 사용하므로 나머지 18바이트는 쓰레기 값.

- 40 ~ 140 : 32바이트의 inode block 8개

<inode 분석>

① status = 'x'로 하고 이는 78입니다. int이므로 4바이트를 차지하므로 나머지는 0으로 초기화됨 0078 0000

② name = 8바이트를 0으로 초기화

③ file_size = int 변수를 0으로 초기화 했으므로 4바이트를 0으로 초기화

④ direct_blocks[i] = int[4]의 배열을 -1로 초기화 했으므로 16바이트를 -1로 초기화 : -1은 ff인 듯.

- 'ssufs'라는 파일을 메모리에서 관리합니다. ops.c에서 조작을 하여 ssufs를 채워 넣는 것입니다. hexdump ssufs로 값을 계속 찍어봅시다.
- inode에 추가했을 때 반드시 superblock도 갱신해야함.
- 내가 완벽히 이해한 함수는 ssufs-disk.h에 주석으로 씀

2. 1번

- 파일을 만들기만 하면 0byte이니까 디스크에서 데이터 블록을 할당할 필요가 없습니다.

- inode는 존재해서 디스크가 할당은 해줬는데 데이터는 없는 파일이 0바이트 파일인가?
- 일단 검토까지 완료

```

46 //update superblock
47 struct superblock_t *superblock = (struct superblock_t *)malloc(sizeof(struct superblock_t));
48 memcpy(superblock->name, "ssufs", 8);
49 for(int i=0; i<NUM_INODES; i++)
50 {
51     superblock->inode_freelist[i] = INODE_FREE;
52 }
53 for(int i=0; i<NUM_DATA_BLOCKS; i++)
54 {
55     superblock->datablock_freelist[i] = DATA_BLOCK_FREE;
56 }
57 superblock->inode_freelist[my_inode_num] = INODE_IN_USE;
58 ssufs_writeSuperBlock(superblock);
59
60 return 0;
61 }
62

```

생각 X

3. 5번

- = 블록 단위가 64바이트 이므로 64보다 작아도 64 이용.
- 테스트에서 2번쓰게 하면 2번 적히지만 실행을 2번한다고 해서 2번쓰지는 않음.
- 1) superblock 참고해서 빈 데이터 블록 찾기
- 2) inode 갱신 : 지금 내가 열어놓은 파일의 inode를 갱신해야 합니다.
- 3) 빈 데이터 블록에 데이터 적기

4. 3번

- open을 하면 빈 handler에 현재 이름을 가진 놈의 inode를 저장합니다. offset은 언제나 0으로 설정하라고 교수님이 말씀하심.
- 1) file_handle_array에서 빈 부분을 찾기
- 2) 빈 부분의 인덱스를 리턴하는게 목적이지만 , 빈 부분의 인덱스가 구조체이므로 offset은 0으로 설정하고 , inode에 저장된 filename과 인자를 비교해서 같은 inode의 번호를 저장

<38강_RAID> 12/01 ~ 12/02

1. RAID Overview

= Redundant Arrays of inexpensive Disks

= 개별적인 다수의 디스크를 중복해서 사용하는 기술입니다.

- RAID는 현재 0번부터 시작해서 9번까지 나왔습니다, 모두 독립적인 개념이기 때문에 구분해서 공부해야 합니다.
- 데이터 중복과 성능향상이 목적입니다.

2. RAID 특징

- 다수의 디스크를 사용하기 때문에 몇 개의 디스크에 결함이 발생해도 시스템이 안전하게 돌아갈 수 있습니다.
- 병행적으로 사용하기 때문에 빠릅니다.
- 디스크의 개수가 많이 필요합니다.
- 신뢰성이 높아짐
- 소프트웨어를 많이 바꾸지 않고 쉽게 RAID를 사용할 수 있습니다.

3 Raid 평가 척도

1) 용량

- n개의 디스크가 b개의 블록으로 이루어져 있을 때를 가정해봅시다.
- 데이터 중복이 없는 경우의 용량 : $b * n$
- 데이터 중복이 있는 경우의 용량 : $b * n / 2$

2) 신뢰성

① 신뢰성 vs 가용성

- 신뢰성을 $R(t)$ 라고 하며 , 지금 시스템이 살아있을 확률을 의미합니다.
 - 가용성을 $A(t)$ 라고 하며 , 언제부터 언제까지 시스템이 살아있을 확률을 의미합니다.
 - 신뢰성과 가용성을 모두 올려야 합니다.
 - 신뢰성을 올리려면 디스크에 데이터를 중복시켜서 저장해야 합니다.
 - 은행은 신뢰성이 중요하기 때문에 데이터를 중복시켜서 용량을 감소시키지만 신뢰성을 확보합니다.
- ex) 은행에서 client의 데이터를 여러 디스크에 중복해서 저장합니다.

② 결함 허용

= Fault-Tolerance

= 디스크에 결함이 발생해도 시스템이 문제없이 돌아가는 경우.

- 디스크 1개의 결함 문제 허용 : 여러 개의 디스크 중 1개가 깨져도 문제없이 시스템이 돌아감.
- 디스크 n개의 결함 문제 허용 : 여러 개의 디스크 중 n개가 깨져도 문제없이 시스템이 돌아감.

3) 성능

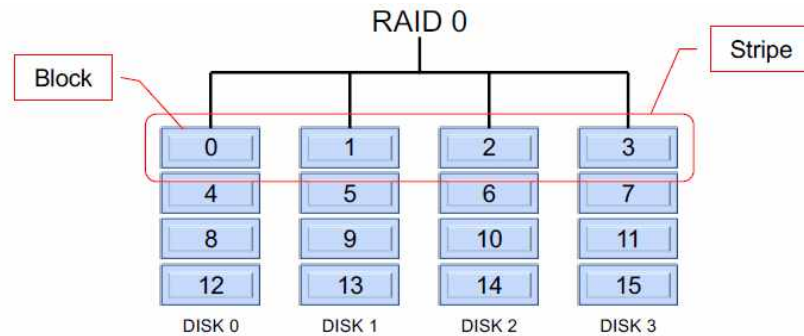
- JBOD(=스패닝) : 내 컴퓨터처럼 여러 개의 디스크를 c , d , e 처럼 사용하는 방법

4. 버전별 RAID

- 버전의 번호는 성능과 관련이 없습니다.
- RAID 2는 RAID 0 . RAID 1의 기능을 모두 갖고 있지 않고 , 업그레이드 버전도 아닙니다.
- RAID끼리는 모두 독립적인 특징을 가지고 있습니다.
- 모든 RAID는 **striping** 기법을 사용합니다.
- RAID의 분석 부분 관련 ppt는 시험에 나오지 않습니다.

1) RAID 0

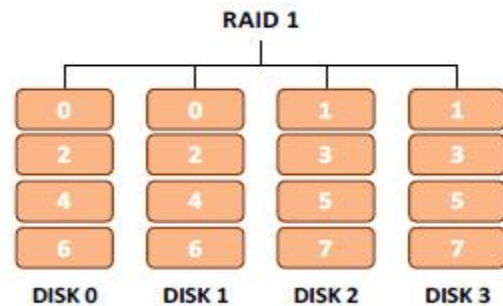
= 핵심기술 : **Striping**



- Striping : 디스크가 4개지만 각각의 디스크의 블록을 구분 짓지 않고 4개의 디스크를 하나의 디스크라 생각하고 블록을 사용합니다.
- 용량 : JBOD와 차이가 없습니다.
- Reliability : JBOD와 차이가 없습니다.
- 성능은 좋아집니다.

2) RAID 1

= 핵심기술 : Mirroring



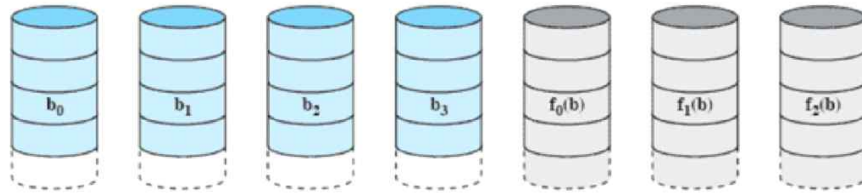
- Mirroring : 기존에 존재했던 **디스크의 복사본**들을 각각 만들어서 저장하는 기법입니다.
- Disk 0의 복사본이 Disk 1 , Disk 2의 복사본이 Disk 3입니다.
- 복사본을 1개씩 만들기 때문에 기존의 방식은 n 개의 디스크가 필요하지만 RAID 1에서는 백업용 디스크로 인해 $2n$ 개의 디스크가 필요합니다. 그러므로 $2n$ 개의 디스크가 존재하지만 실질적인 데이터는 n 개의 분량입니다.
- 같은 내용을 가진 디스크가 존재하기 때문에 복사본 디스크가 살아있다면 원본이 박살나도 괜찮습니다.
- 디스크를 많이 사서 가격이 비쌉니다.
- 용량은 $1/2$ 로 줄어듭니다.

3) RAID 2 & RAID 3

= 핵심기술 : Parity(=P(b))

- $P(b)$ 는 같은 층의 블록에서 0의 개수가 홀수면 1이고 , 0의 개수가 짝수면 0입니다.
- RAID 2와 RAID 3은 비슷합니다.
- RAID 2와 RAID 3은 **작은 스트립**을 사용합니다.
- RAID 2가 디스크를 더 많이 사용합니다.
- RAID 3을 RAID 2보다 많이 사용합니다.

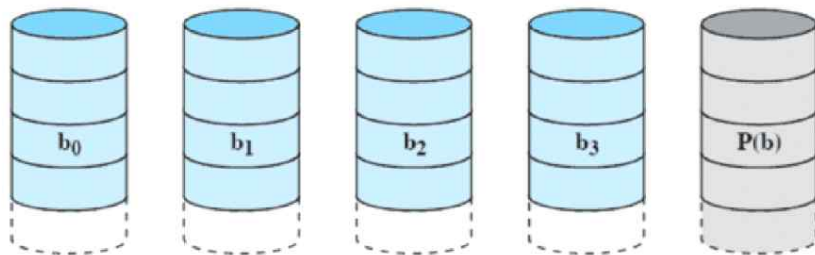
① RAID 2



(c) RAID 2 (redundancy through Hamming code)

- 디스크가 n 개 있으면 $n-1$ 개의 $P(b)$ 를 사용합니다.

② RAID 3



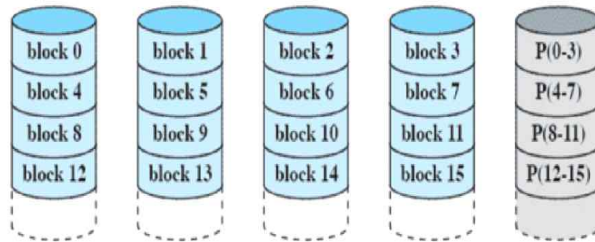
(d) RAID 3 (bit-interleaved parity)

- RAID 2에서 $p(b)$ 가 $n-1$ 개나 필요한지에 대한 의문으로 고안된 방식입니다.
- 디스크의 개수와 상관없이 $P(b)$ 를 단 1개만 사용합니다.
- 비트 단위

4) RAID 4 ~ RAID 6

- RAID 4 ~ RAID 6은 **큰 스트립**을 사용합니다.

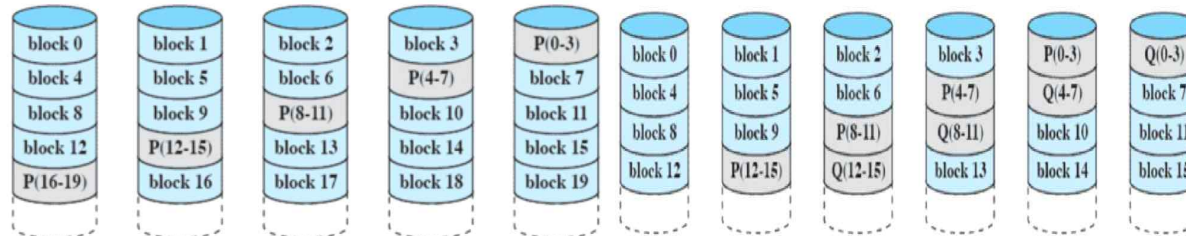
① RAID 4



(e) RAID 4 (block-level parity)

- RAID 3과 RAID 4는 거의 모든 면에서 동일하지만 **RAID 4는 블록 단위**입니다.
- 속도의 측면에서 블록단위가 비트단위보다 빠릅니다.

② RAID 5



(f) RAID 5 (block-level distributed parity)

(g) RAID 6 (dual redundancy)

- RAID 4와 RAID 5는 거의 모든 면에서 동일하지만 RAID 5는 **p(b)**를 **전체 디스크에 분산**해서 저장합니다.

③ RAID 6

- RAID 3 ~ RAID 5는 임의의 디스크 1개가 박살나는 것은 괜찮으나 2개가 박살나는 것은 문제가 생깁니다. 하지만 RAID 6은 2개까지는 디스크가 박살나는 것을 허용해줍니다.
- N개의 디스크가 필요한 경우 N+2개의 디스크가 필요합니다.
- 쓰기의 경우가 RAID 5보다 30% 이상 성능의 저하가 발생하는 문제점이 존재합니다.

5. 교수님 여담

- NAS 장비로 RAID를 사용할 수 있습니다.
- RAID는 랜섬웨어 방어에 효과적이기 때문에 인기를 끌었습니다.
- 랜섬웨어에 걸리면 풀 방법은 없다고 생각합니다.
- 최고의 랜섬웨어 방지법 : FreeNAS를 빈 디스크에 다운받고 , 네트워크를 끊고 해당 디스크에 따로 백업하기

*** 시험문제 : RAID 0 ~ 6의 간단한 특징 + 알고 요구되는 디스크 개수 ***

카테고리	레벨	설명	요구되는 디스크	데이터 가용성	큰 입출력 데이터 전송 능력	작은 입출력 요구율
스트라이핑	0	중복 없음	N	단일 디스크보다 낮음	매우 높음	읽기, 쓰기 모두에서 매우 높음
미러링	1	미러됨	2N, 3N, etc.	RAID 2,3,4,5 보다 높고, 6 보다 낮음	읽기는 단일 디스크보다 높고, 쓰기는 단일 디스크와 비슷함	읽기는 단일 디스크의 최대 두 배, 쓰기는 단일 디스크와 비슷함
병렬접근	2	해밍코드에 의한 중복	N+M	단일 디스크보다 상당히 높고, RAID 3,4,5 보다 높음	모든 레벨들 중 가장 높음	단일 디스크의 거의 두 배임
	3	비트 인터리브드 패리티	N+1	단일 디스크보다 상당히 높고, RAID 2,4,5와 대등	모든 레벨들 중 가장 높음	단일 디스크의 거의 두 배임
독립접근	4	블록 인터리브드 패리티	N+1	단일 디스크보다 상당히 높고, RAID 2,3,5와 대등	읽기는 RAID 0과 비슷함. 쓰기는 단일 디스크보다 상당히 낮음	읽기는 RAID 0 와 비슷함 쓰기는 단일 디스크보다 상당히 낮음
	5	블록 인터리브드 분산 패리티	N+1	단일 디스크보다 상당히 높고, RAID 2,3,4와 대등	읽기는 RAID 0과 비슷함. 쓰기는 단일 디스크보다 낮음	읽기는 RAID 0 와 비슷함 쓰기는 일반적으로 단일 디스크보다 낮음
	6	블록 인터리브드 이중 분산 패리티	N+2	모든 레벨들 중 가장 높음	읽기는 RAID 0과 비슷함 쓰기는 RAID 5보다 낮음	읽기는 RAID 0 와 비슷함 쓰기는 RAID 5 보다 상당히 낮음

<41강_Locality and Fast File System> 12/02 ~ 12/06

1. 기존 UNIX의 단점

- 1) 내부 단편화 발생
- 2) 디스크로부터의 데이터 전송이 비효율적입니다.
- 3) 블록의 크기(512 Bytes)가 작습니다.
- 4) inode와 데이터 간의 거리가 디스크 관점에서는 매우 멀리 있습니다.

2. Fast File System (=FFS) 탄생

- BSD(기업)에서 UNIX에서 만든 File System의 단점들을 개선하기 위해 빠른 파일시스템을 만들었습니다.
- 속도가 빠릅니다.
- **디스크 블록의 크기가 큼니다.**
- 대용량 영화나 음악 파일 관리에 적합합니다.

3. FFS의 성능개선

- UNIX에서 만든 기존의 File System과 비교해서 개선된 FFS의 성능.

11:15 ~ 11:24 화장실

1)

- 그룹으로 나누어서 superblock을 두고 , 디스크와 inode의 거리를 줄여서 file system을 구성
- 단점 : 다수의 superblock으로 인해 용량을 많이 차지함 , 블록의 크기가 크므로 , 내부단편화가 많이 생김.
- 즉 , 용량의 손해는 크지만 속도는 빠르게 단순한 FFS의 특징
- 그래서 FFS를 쓰는 IOS는 큰 용량의 디스크 OR Flashmemory를 사용
- BSD : 윈도우처럼 계층적 디렉토리를 사용하지 말고 , 내가 알아서 할테니까 계층적 구조를 만들지 말자. 아무리 ssd라도 계층적 구조는 좋지 않아. file의 종류(pdf , mp3)끼리 모아서 저장 하고 index를 통해 내가(BSD기반 소프트웨어) 바로 찾아줌

2) Large-File Exception

- 아주 큰 파일은 예외적으로 분산 시켜서 저장함 , 이렇게 해도 FFS는 여전히 빠름

3) 기타 성능 향상

① Parameterized Placement

= 기술의 발전으로 디스크의 회전 속도가 지나치게 빠르다 보니 헤더가 원하는 섹터를 지나쳐 버리는 단점이 생깁니다. 그러므로 연속된 데이터를 순차적으로 저장하지 않고 , 조금씩 떨어뜨립니다.



(FFS Standard vs. Parameterized Placement)

② Sub-Blocks

= 내부 단편화가 많이 발생하는 것을 방지하기 위해 항상 4KB , 16KB를 블록 크기로 하지 말고 512byte의 블록도 허용합니다.

③ 긴 파일 이름

④ Symbolic Link : 바로가기

⑤ System-Call : rename()

- 기존에는 파일 이름을 변경할 때 깨질까봐 '변경할 이름을 tmp에 저장 -> 변경 -> tmp 삭제'의 로직으로 변경하였습니다.
- 현대에는 rename()을 사용해서 모든 과정을 생략하고 한 번에 파일 이름을 변경합니다.

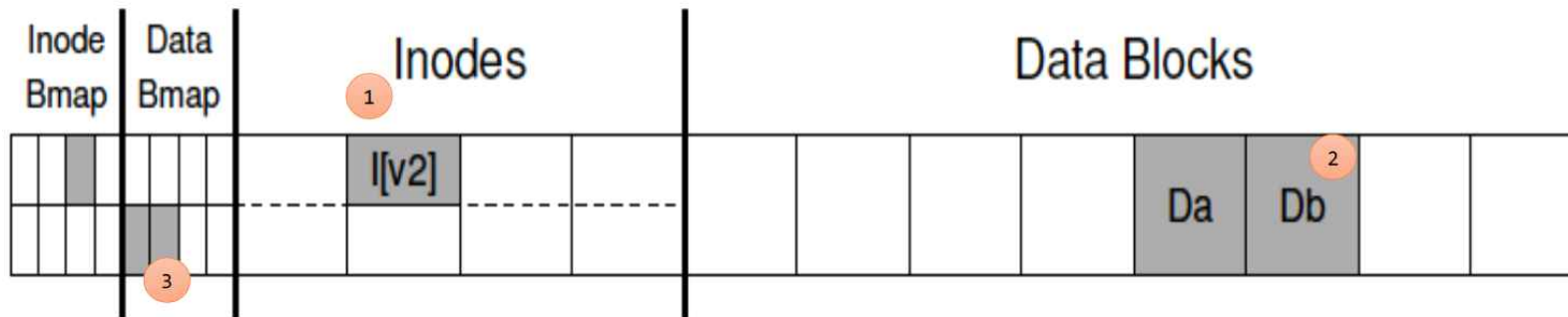
<42강_Crash Consistency : FSCK and Journaling> 12/08 ~ 12/10

1. Overview

- 전력 손실이나 System Crash가 발생해도 안전하게 디스크 상의 내용을 저장하고 갱신해야 합니다.
- 디스크에서 크래시가 발생하면 동일해야 하는 데이터가 다른 Crash Consistency(깨짐 일관성)가 발생할 수 있습니다.
- Disk Crash가 발생해서 생기는 다양한 문제를 FSCK OR journaling 기법을 사용하여 해결 합니다.

2. 중요 개념

<Example : Inode(v2)에 새로운 데이터 블록(Db)을 할당하는 과정에서 6가지 경우의 Crash가 발생합니다.>



1) Inode Bmap (=Inode Bitmap)

= Inode 블록에 Inode가 할당되었는지를 0과 1로 관리합니다.

2) Data Bmap (=Data Bitmap)

= Data 블록에 Data가 할당되었는지를 0과 1로 관리합니다.

- 사용자가 디스크에 데이터를 추가 하는 경우 Inodes를 확인하지 않고 Data Bitmap을 확인해서 빈 공간(0)을 찾아서 할당합니다.

3) Inodes

= 블록마다 Inode가 저장되며 , Inode 블록은 Data가 Data Block 어디에 있는지 저장합니다.

```
struct inode_t
{
    int status;
    char name[MAX_NAME_STRLEN];
    int file_size;
    int direct_blocks[MAX_FILE_SIZE];
};
```

4) Data Blocks

= 블록마다 Data가 저장됩니다.

5) Consistent State

= Data Block에 새로운 데이터를 저장해도 일관성이 유지됩니다.

6) Inconsistent State

= Data Bitmap과 관련 있는 개념입니다.

① inode가 bitmap에 할당되지 않은 데이터 블록을 참조하는 경우

② 사용되지 않은 데이터 블록을 위해 비트맵 할당

ex) Data Block에 Db가 저장되어 있음에도 불구하고 Data Bitmap이 '0000/1000' 인 경우

ex) Data Block에 Da , Db만 저장되어 있음에도 불구하고 Data Bitmap이 '0010/1100' 인 경우

7) Inconsistent state의 결과

① 디스크 누수 현상(디스크에 저장되지 않았는데 bitmap에 써서 쓰지 못하는 슬픔)이 일어납니다.

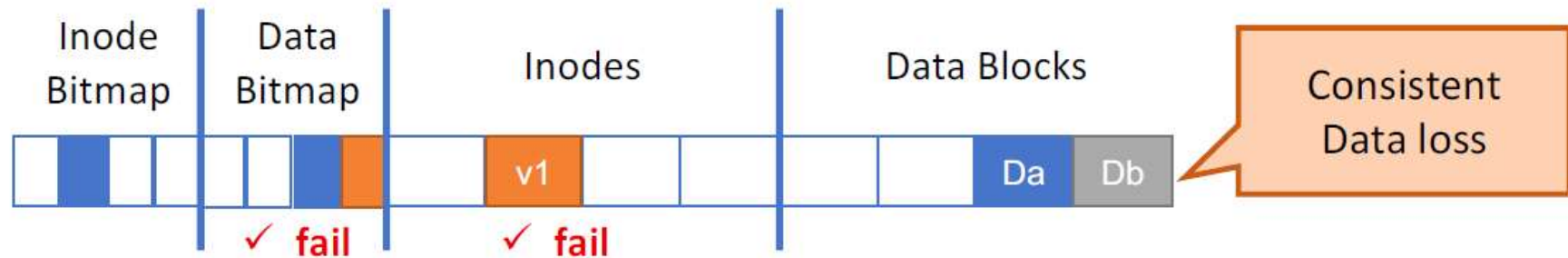
② inode가 Db를 가리키는데 0000/1000이면 다른 상황에서 6번째가 비어있다고 생각하므로 , Db자리에 데이터를 가리키도록 함. 가비지 데이터를 읽게 됨.

③ 고아 데이터 : DB에는 있는데 Data Bitmap에는 없는 데이터

3. 디스크에 데이터를 저장하는 경우에 발생하는 6가지 Crash(디스크 깨짐)

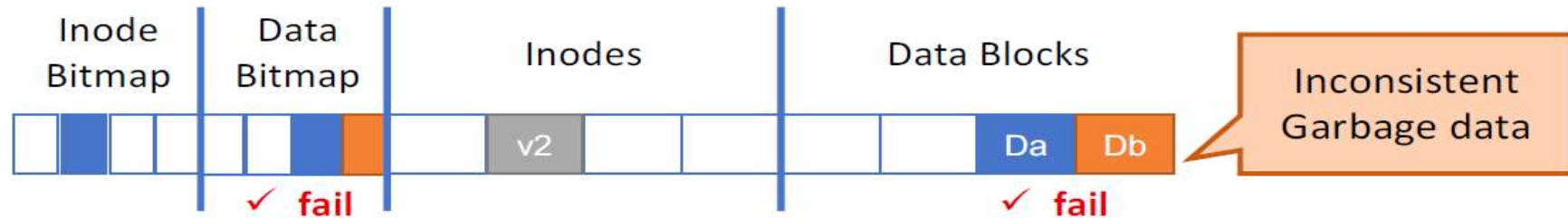
- 디스크에 데이터를 저장할 때 3가지 저장소 {Bitmap(Inode Bitmap & Data Bitmap), Inodes, Data Blocks}에 모두 저장해야 합니다.
- 3가지 저장소 중에서 1 곳이라도 저장하지 않으면 Crash가 발생합니다.
- 1) ~ 3) : 저장소 1개만 성공 , 나머지 2개 저장소 실패
- 4) ~ 6) : 저장소 2개 성공 , 나머지 1개 저장소 실패
- 교수님 말씀 : 이 내용은 너무 중요하지만 완벽하게 이해가 도저히 되지 않는다면 아래의 consistent , inconsistent 조건을 암기하시오.
- Consistent : Bitmap & Inodes가 모두 적혀있거나 모두 적혀있지 않는 경우
- Inconsistent : 6가지 중 Consistent인 경우를 제외한 4가지 경우
- consistent는 복구가 쉽고 inconsistent는 복구가 어렵습니다.

1) Data Blocks 저장



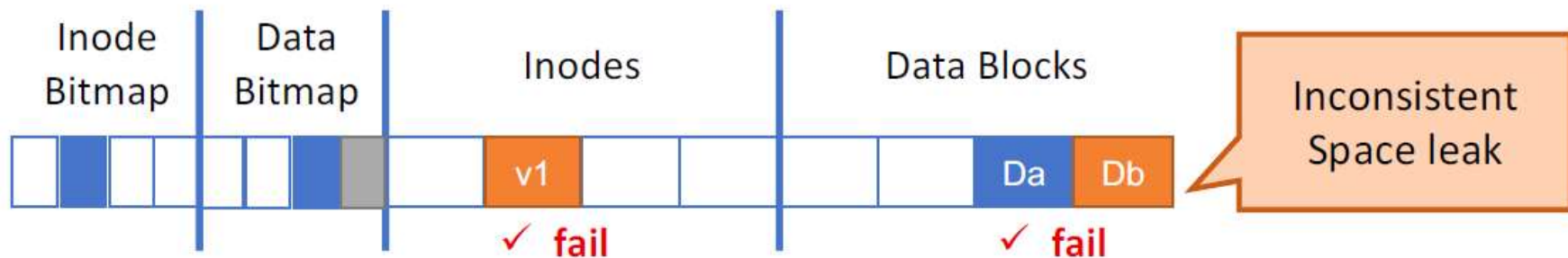
- Data Blocks에 Da , Db를 적었지만 Data Bitmap과 Inodes에는 적지 않았습니다.
- Data Bitmap은 Data Blocks의 어떤 블록이 할당 되어 있는지 알려주지 않고 Da와 Db는 Inode로도 연결되어 있지 않습니다. 그러므로 데이터는 존재하지만 연결이 되어있지 않아 의미 없는 데이터가 되어 버립니다. 하지만 어차피 bitmap이 fail 되어 내용이 적혀있지 않으므로 추후에 다른 데이터로 덮어쓰고 inode를 갱신시킬 수 있습니다.

2) Inodes 저장

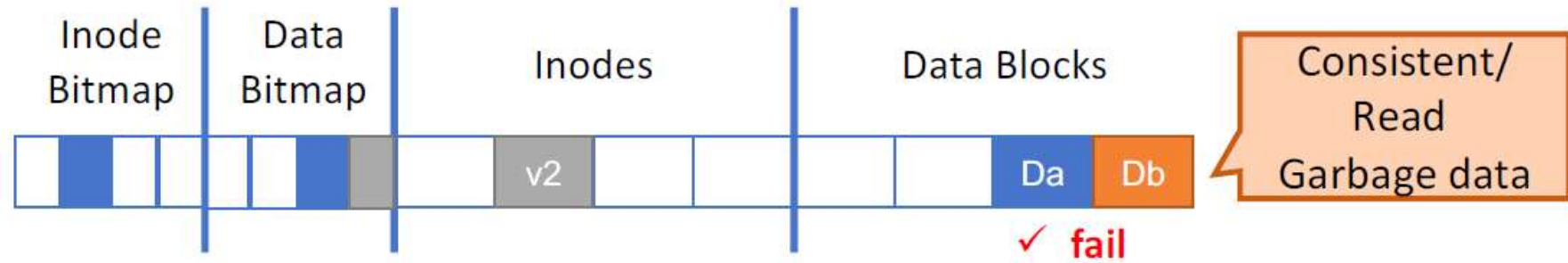


- Inode는 적었으므로 v2는 Da(ex : 0001), Db(ex : 0101)를 포인터로 가리킵니다.
- Data Bitmap과 Data Blocks는 디스크에 저장되지 않았습니다.
- Data Bitmap가 fail 되어 적혀있는 내용이 없으므로 추후에 Db자리에 다른 데이터(ex : 1010)를 저장할 수 있습니다.
- 그러므로 v2가 가리키는 Db에는 v2가 원하지 않는 다른 데이터(1010)가 저장되게 됩니다. 이 때 Db에 저장된 데이터를 garbage라고 합니다.
- v2가 기존에 저장한 제대로 된 데이터(0101)와 garbage(1010)는 다른 값을 갖기 때문에 일관성이 없습니다. -> inconsistent

3) Bitmap 저장

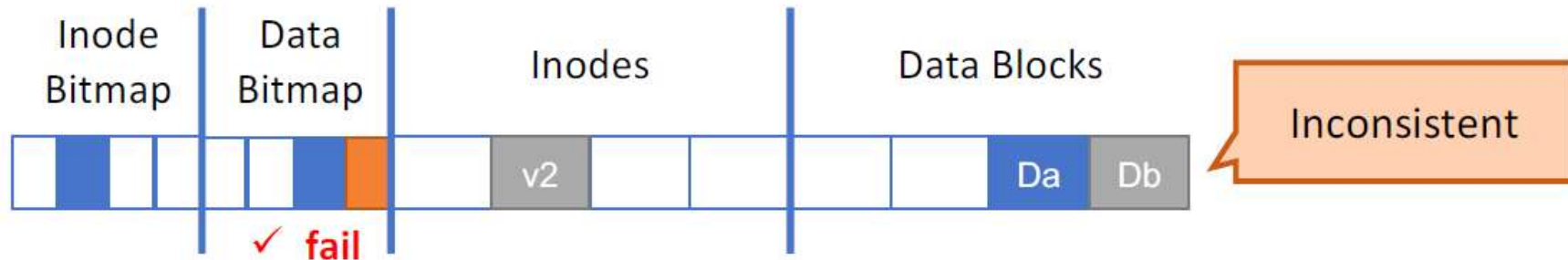


4) Bitmap & inodes 저장



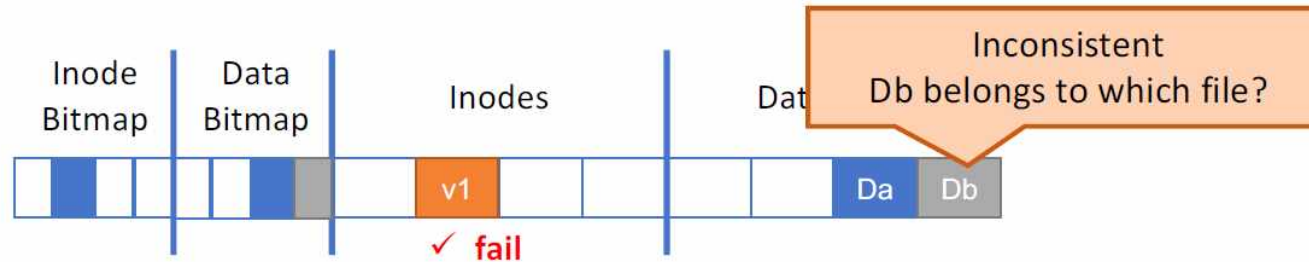
- Inode가 제대로 할당 되어 v2가 올바른 Data Block을 가리키지만 , Data Blocks의 fail로 인해 올바른 데이터가 아닌 garbage 데이터가 저장되어 있어 garbage 데이터를 읽게 됩니다.

5) inodes & Data Blocks 저장



- 올바른 데이터를 Inode가 제대로 가리킵니다.
- 하지만 Bitmap의 fail로 인해 할당 여부를 알지 못하므로 Data Blocks를 덮어쓸 수 있습니다.

6) Bitmap & Data Blocks 저장



4. Solution_1 : FSCK(File System Checker)

1) 정의 및 특징

- = Consistent states는 간단하게 해결할 수 있으므로 어려운 inconsistent states를 해결하는 방법입니다.
- inconsistent states의 일관성을 유지하도록 변경합니다.
- 4가지 inconsistent states를 모두 해결하지는 못합니다. (특히, Inodes Crash는 해결하기 어렵습니다.)
- Journaling은 write 작업이 존재하지만, FSCK는 write 작업이 존재하지 않습니다.
- 간단한 해결책이지만 너무 느리다는 단점이 존재합니다.

2) 기본동작

검사 항목			
Super Block 오류	Free Block	Inode 상태	할당된 Inode 개수
중복 포인터	Bad Block	Directory	

- 최악의 경우 Inode를 초기화 시켜버리고 Bitmap을 갱신합니다.
- Bitmap과 Inode가 초기화 되어서 둘 다 아무것도 적히지 않은 상태가 되므로 inconsistent states 가 consistent states로 바뀝니다.

5. Solution_2 : Journaling

1) Journaling Overview

- Bitmap , Inodes , DB에 정보를 적을 때 마다 관련 내용을 Log로 따로 저장하고 , Crash가 발생하면 Log를 통해 해결합니다.
- Bitmap , Inodes , DB에 정보를 적는 요청이 발생했을 때 바로 해당 블록에 적지 않고 Log에 먼저 기록을 합니다.
- Log와 Transaction은 같은 개념으로 생각해도 무방하며 , Journal Section은 여러 개의 Log(=Transaction)들로 구성이 되어있습니다.
- Log도 추가로 디스크에 저장해야 하므로 디스크의 일부분을 차지하게 됩니다. 그러므로 디스크의 용량이 기하급수적으로 커진 현대에 각광받는 기술입니다.
- Log로 인해 시간은 더 소요되지만 크게 문제가 되지는 않습니다.
- Journaling 기법을 Data Journaling 과 MetaData Journaling 으로 구분할 수 있지만 우리는 Data Journaling 만 배웁니다.

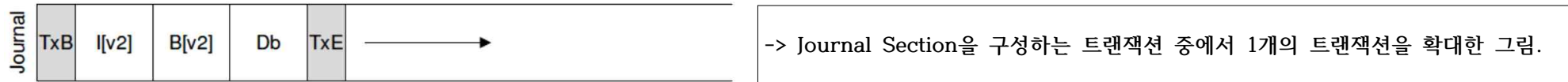
2) Journal

<그림_1 : 디스크 전체 그림에서의 Journal Section(붉은색)>



-> 기존의 File System에서 Journal Section을 추가했습니다.

<그림_2 : 그림_1의 Journal Section(붉은색)은 여러 개의 트랜잭션이 모인 공간입니다.>



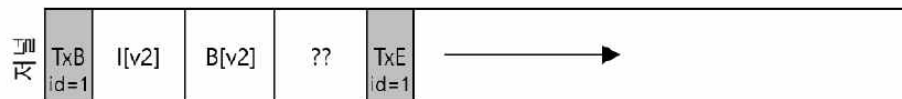
- Journal Section에 여러 개의 트랜잭션을 저장합니다. 그러므로 트랜잭션을 작성하고 다음 블록에 또 다른 트랜잭션을 작성할 수 있습니다.
- 트랜잭션(Transaction) : 디스크에 저장된 내용을 Read 나 Write를 통해 변화시키는 일련의 행위입니다.
- 트랜잭션은 $Txb + I[v2] + B[v2] + Db + TxE$ 으로 구성됩니다.
- TxB : 트랜잭션 시작 블록이며 , 갱신될 블록들에 대한 정보를 기록합니다.
- I[v2] : 미리 기록한 Inodes 정보
- B[v2] : 미리 기록한 Bitmap 정보
- Db : 미리 기록한 Data Blocks 정보
- TxE : 트랜잭션 종료 블록이며 , 트랜잭션 작성의 성공을 의미합니다.
- TxB와 TxE는 같은 id를 갖습니다.

<Check Pointing>

= Journal에 기록된 내용을 실제로 디스크에 반영.

3) Journal 기록 과정에서 발생한 Crash 해결 방법

<Example : 트랜잭션에서 Inode 로그를 보고 디스크에서 inode를 변경합니다. 트랜잭션에서 Bitmap 로그를 보고 디스크에서 Bitmap을 변경합니다. 트랜잭션에서 디스크 블록을 변경하려는 도중 Crash가 발생하여 디스크 블록 로그에 garbage(??)가 저장되었습니다.>



① 방법_1

- 이전에 변경한 Inode , Bitmap도 변경 이전으로 모두 되돌립니다.
- 확실한 방법이지만 너무 느립니다.

② 방법_2

- = 트랜잭션에서 잘못된 데이터 블록 로그 영역의 정보를 write 하면 되돌릴 수 없습니다. 그러므로 트랜잭션을 구성하는 5개의 로그 정보와 완벽하면 한 번에 디스크에서 변경합니다.
- TxE를 적는 순간 5개의 로그 정보가 완벽함을 의미합니다.
- 디스크 쓰기 연산의 원자성을 보장합니다. 디스크에 쓰기 연산을 하는 경우 반드시 섹터 단위(512byte)로 쓰게 하고 트랜잭션의 크기를 512byte 이내로 만들어서 한 번에 옮길 수 있도록 만듭니다.
- 트랜잭션의 크기가 512byte를 넘기면 압축하여 512byte 이내로 만듭니다.
- 한 번에 변경하기 때문에 파일 시스템에서 의도한 순서로 디스크에 write 하지 않고 디스크에 의해 순서를 결정하여 write 합니다.

*** 시험범위 : #22번 PPT의 17 Page 까지 ***